User Manual

for S32K3 ADC Driver

Document Number: UM34ADCASR4.4 Rev0000R2.0.0 Rev. 1.0

1 Revision History **2**

2 Introduction **3**

2.1 Supported Derivatives . . . 3
2.2 Overview . . . 4
2.3 About This Manual . . . 4
2.4 Acronyms and Definitions . . . 5
2.5 Reference List . . . 5

3 Driver **6**

3.1 Requirements . . . 6
3.2 Driver Design Summary . . . 6
3.3 Hardware Resources . . . 7
3.4 Deviations from Requirements . . . 7
3.5 Driver Limitations . . . 17
3.6 Driver usage and configuration tips . . . 17
3.6.1 Calibration function . . . 17
3.6.2 Self-Test function . . . 18
3.6.3 BCTU control mode . . . 18
3.6.4 BCTU triggered mode . . . 19
3.6.5 DMA transfer . . . 20
3.6.6 Injected conversions . . . 22
3.6.7 Set Channel Optimization . . . 22
3.6.8 Enable/Disable channel . . . 23
3.6.9 Conversion Time Once . . . 24
3.6.10 Bypass Abort Chain Check . . . 24
3.6.11 Set Clock Mode . . . 25
3.6.12 Analog Watchdog feature . . . 26
3.6.13 Without Interrupts group . . . 27
3.6.14 Without Dma group . . . 29
3.6.15 External DMA channel feature . . . 29
3.6.16 Optimize DMA streaming groups . . . 30
3.6.17 TempSense functionality . . . 33
3.7 Runtime errors . . . 33
3.8 Symbolic Names Disclaimer . . . 34

4 Tresos Configuration Plug-in **35**

4.1 Module Adc . . . 40
4.2 Container AdcConfigSet . . . 40
4.3 Container AdcHwUnit . . . 41
4.4 Parameter AdcHwUnitId . . . 41

4.5 Parameter AdcLogicalUnitId . . . 42
4.6 Parameter AdcTransferType . . . 42
4.7 Parameter AdcClockSource . . . 43
4.8 Parameter AdcPrescale . . . 43
4.9 Parameter AdcAltPrescale . . . 44
4.10 Parameter AdcCalibrationPrescale . . . 44
4.11 Parameter AdcHighSpeedEnable . . . 45
4.12 Parameter AdcAltHighSpeedEnable . . . 46
4.13 Parameter AdcPowerDownDelay . . . 46
4.14 Parameter AdcAltPowerDownDelay . . . 47
4.15 Parameter AdcMuxDelay . . . 47
4.16 Parameter AdcAutoClockOff . . . 48
4.17 Parameter AdcBypassSampling . . . 48
4.18 Parameter AdcHwUnitOverwriteEn . . . 48
4.19 Parameter AdcPresamplingInternalSignal0 . . . 49
4.20 Parameter AdcPresamplingInternalSignal1 . . . 49
4.21 Parameter AdcPresamplingInternalSignal2 . . . 50
4.22 Parameter AdcHwUnitUsrOffset . . . 51
4.23 Parameter AdcHwUnitUsrGain . . . 51
4.24 Parameter AdcHwUnitResolution . . . 52
4.25 Parameter AdcHwUnitBypassResolution . . . 52
4.26 Parameter AdcHwUnitDmaClearSource . . . 53
4.27 Parameter AdcSarHwUnitVref . . . 53
4.28 Reference AdcDmaChannelId . . . 55
4.29 Reference AdcCountingDmaChannelId . . . 55
4.30 Reference AdcHwUnitEcucPartitionRef . . . 56
4.31 Container AdcNormalConvTimings . . . 56
4.32 Parameter AdcHardwareAverageEnable . . . 57
4.33 Parameter AdcHardwareAverageSelect . . . 57
4.34 Parameter AdcSamplingDurationNormal0 . . . 58
4.35 Parameter AdcSamplingDurationNormal1 . . . 59
4.36 Parameter AdcSamplingDurationNormal2 . . . 59
4.37 Container AdcAlternateConvTimings . . . 60
4.38 Parameter AdcHardwareAverageEnableAlternate . . . 60
4.39 Parameter AdcHardwareAverageSelectAlternate . . . 60
4.40 Parameter AdcSamplingDurationAlt0 . . . 61
4.41 Parameter AdcSamplingDurationAlt1 . . . 62
4.42 Parameter AdcSamplingDurationAlt2 . . . 62
4.43 Container AdcChannel . . . 63
4.44 Parameter AdcLogicalChannelId . . . 63

4.45 Parameter AdcChannelName . . . 64
4.46 Parameter AdcChannelId . . . 64
4.47 Parameter AdcChannelConvTime . . . 65
4.48 Parameter AdcChannelLimitCheck . . . 65
4.49 Parameter AdcChannelHighLimit . . . 66
4.50 Parameter AdcChannelLowLimit . . . 66
4.51 Parameter AdcChannelRangeSelect . . . 67
4.52 Parameter AdcChannelRefVoltsrcHigh . . . 67
4.53 Parameter AdcChannelRefVoltsrcLow . . . 68
4.54 Parameter AdcChannelResolution . . . 68
4.55 Parameter AdcChannelSampTime . . . 69
4.56 Parameter AdcEnablePresampling . . . 69
4.57 Parameter AdcEnableThresholds . . . 70
4.58 Parameter AdcWdogNotification . . . 70
4.59 Reference AdcThresholdRegister . . . 71
4.60 Container AdcGroup . . . 71
4.61 Parameter AdcGroupAccessMode . . . 72
4.62 Parameter AdcGroupConversionMode . . . 72
4.63 Parameter AdcGroupConversionType . . . 73
4.64 Parameter AdcGroupId . . . 73
4.65 Parameter AdcGroupPriority . . . 74
4.66 Parameter AdcGroupReplacement . . . 74
4.67 Parameter AdcGroupTriggSrc . . . 75
4.68 Parameter AdcHwTrigSignal . . . 75
4.69 Parameter AdcHwTrigTimer . . . 76
4.70 Parameter AdcNotification . . . 76
4.71 Parameter AdcExtraNotification . . . 77
4.72 Parameter AdcStreamingBufferMode . . . 77
4.73 Parameter AdcEnableOptimizeDmaStreamingGroups . . . 78
4.74 Parameter AdcEnableHalfInterrupt . . . 79
4.75 Parameter AdcStreamingNumSamples . . . 79
4.76 Parameter AdcStreamResultGroup . . . 80
4.77 Parameter AdcEnableChDisableChGroup . . . 80
4.78 Parameter AdcWithoutInterrupts . . . 81
4.79 Parameter AdcWithoutDma . . . 81
4.80 Parameter AdcExtDMAChanEnable . . . 82
4.81 Reference AdcGroupHwTriggerSource . . . 82
4.82 Reference AdcGroupDefinition . . . 83
4.83 Reference AdcGroupEcucPartitionRef . . . 83
4.84 Container AdcGroupConversionConfiguration . . . 84

4.85 Parameter AdcGroupHardwareAverageEnable . . . 84
4.86 Parameter AdcGroupHardwareAverageSelect . . . 85
4.87 Parameter AdcSamplingDuration0 . . . 85
4.88 Parameter AdcSamplingDuration1 . . . 86
4.89 Parameter AdcSamplingDuration2 . . . 86
4.90 Container AdcAlternateGroupConvTimings . . . 87
4.91 Parameter AdcGroupAltHardwareAverageEnable . . . 87
4.92 Parameter AdcGroupAltHardwareAverageSelect . . . 88
4.93 Parameter AdcAltGroupSamplingDuration0 . . . 88
4.94 Parameter AdcAltGroupSamplingDuration1 . . . 89
4.95 Parameter AdcAltGroupSamplingDuration2 . . . 89
4.96 Container AdcThresholdControl . . . 90
4.97 Parameter AdcThresholdControlRegister . . . 90
4.98 Parameter AdcHighThreshold . . . 91
4.99 Parameter AdcLowThreshold . . . 91
4.100 Container AdcHwTrigger . . . 92
4.101 Parameter AdcHwTrigSrc . . . 92
4.102 Container BctuHwUnit . . . 93
4.103 Parameter BctuLowPowerMode . . . 94
4.104 Parameter BctuGlobalHwTriggers . . . 94
4.105 Parameter BctuNewDataDMAEnableMask . . . 95
4.106 Parameter BctuFifoDmaRawData . . . 95
4.107 Parameter BctuTriggerNotification . . . 95
4.108 Container BctuInternalTrigger . . . 96
4.109 Parameter BctuTriggerLoop . . . 96
4.110 Parameter BctuDataDestination . . . 97
4.111 Parameter BctuHwTriggerEnable . . . 97
4.112 Parameter BctuTriggerConversionMode . . . 98
4.113 Parameter BctuAdcTargetMask . . . 98
4.114 Parameter BctuConversionListStartIndex . . . 99
4.115 Reference BctuTriggerSource . . . 99
4.116 Reference BctuAdcChannelSingle . . . 99
4.117 Container BctuAdcNotifications . . . 100
4.118 Parameter BctuAdcNewDataNotification . . . 100
4.119 Parameter BctuDataOverrunNotification . . . 101
4.120 Parameter BctuListLastConversionNotification . . . 101
4.121 Reference BctuAdcNotificationsAdcIndex . . . 102
4.122 Container BctuListItems . . . 102
4.123 Parameter BctuAdcChannelList . . . 102
4.124 Parameter BctuNextChannelWaitOnTrig . . . 103

4.125 Parameter BctuLastChannel . . . 104
4.126 Container BctuResultFifos . . . 104
4.127 Parameter BctuResultFifoIndex . . . 104
4.128 Parameter BctuWatermarkValue . . . 105
4.129 Parameter BctuFifoNotificationsEnable . . . 105
4.130 Parameter BctuWatermarkNotification . . . 106
4.131 Parameter BctuUnderrunNotification . . . 106
4.132 Parameter BctuOverrunNotification . . . 107
4.133 Parameter BctuFifoDmaEnable . . . 107
4.134 Parameter BctuFifoDmaBuffer . . . 108
4.135 Reference BctuFifoDmaChannelId . . . 108
4.136 Container AdcGeneral . . . 108
4.137 Parameter AdcDeInitApi . . . 110
4.138 Parameter AdcDevErrorDetect . . . 110
4.139 Parameter AdcEnableLimitCheck . . . 111
4.140 Parameter AdcEnableQueuing . . . 111
4.141 Parameter AdcPriorityQueueMaxDepth . . . 112
4.142 Parameter AdcEnableStartStopGroupApi . . . 112
4.143 Parameter AdcGrpNotifCapability . . . 113
4.144 Parameter AdcHwTriggerApi . . . 113
4.145 Parameter AdcPriorityImplementation . . . 114
4.146 Parameter AdcReadGroupApi . . . 115
4.147 Parameter AdcResultAlignment . . . 115
4.148 Parameter AdcVersionInfoApi . . . 116
4.149 Parameter AdcLowPowerStatesSupport . . . 116
4.150 Parameter AdcPowerStateAsynchTransitionMode . . . 117
4.151 Reference AdcEcucPartitionRef . . . 117
4.152 Reference AdcKernelEcucPartitionRef . . . 117
4.153 Container AdcPowerStateConfig . . . 118
4.154 Parameter AdcPowerState . . . 118
4.155 Parameter AdcPowerStateReadyCbkRef . . . 119
4.156 Container AdcInterrupt . . . 119
4.157 Parameter AdcInterruptSource . . . 120
4.158 Parameter AdcInterruptEnable . . . 120
4.159 Container AdcPublishedInformation . . . 121
4.160 Parameter AdcChannelValueSigned . . . 121
4.161 Parameter AdcGroupFirstChannelFixed . . . 122
4.162 Parameter AdcMaxChannelResolution . . . 122
4.163 Container CommonPublishedInformation . . . 122
4.164 Parameter ArReleaseMajorVersion . . . 123

4.165 Parameter ArReleaseMinorVersion . . . 123
4.166 Parameter ArReleaseRevisionVersion . . . 124
4.167 Parameter ModuleId . . . 124
4.168 Parameter SwMajorVersion . . . 125
4.169 Parameter SwMinorVersion . . . 125
4.170 Parameter SwPatchVersion . . . 126
4.171 Parameter VendorApiInfix . . . 126
4.172 Parameter VendorId . . . 127
4.173 Container AutosarExt . . . 127
4.174 Parameter AdcTimeoutMethod . . . 128
4.175 Parameter AdcTimeoutVal . . . 128
4.176 Parameter AdcSarIpDevErrorDetect . . . 129
4.177 Parameter BctuIpDevErrorDetect . . . 129
4.178 Parameter AdcMulticoreSupport . . . 130
4.179 Parameter AdcEnableGroupDependentChannelNames . . . 130
4.180 Parameter AdcBypassAbortChainCheck . . . 131
4.181 Parameter AdcConvTimeOnce . . . 131
4.182 Parameter AdcOptimizeOneShotHwTriggerConversions . . . 132
4.183 Parameter AdcOptimizeDmaStreamingGroups . . . 132
4.184 Parameter AdcPreSamplingOnce . . . 133
4.185 Parameter AdcEnableInitialNotification . . . 134
4.186 Parameter AdcEnableDmaTransferMode . . . 134
4.187 Parameter AdcUseSoftwareInjectedGroups . . . 135
4.188 Parameter AdcUseHardwareNormalGroups . . . 135
4.189 Parameter AdcEnableUserModeSupport . . . 136
4.190 Parameter AdcSetHwUnitPowerModeApi . . . 136
4.191 Parameter AdcEnableChDisableChApi . . . 137
4.192 Parameter AdcGetInjectedConvStatusApi . . . 137
4.193 Parameter AdcEnableThresholdConfigurationApi . . . 138
4.194 Parameter AdcEnableCtuTrigAutosarExtApi . . . 138
4.195 Parameter AdcCtuHardwareTriggerOptimization . . . 139
4.196 Parameter AdcEnableCtuControlModeApi . . . 139
4.197 Parameter CtuEnableDmaTransferMode . . . 140
4.198 Parameter AdcEnableWatchdogApi . . . 141
4.199 Parameter AdcEnableSetChannel . . . 141
4.200 Parameter AdcEnableDualClockMode . . . 142
4.201 Parameter AdcEnableCalibration . . . 142
4.202 Parameter AdcEnableSelfTest . . . 142
4.203 Parameter AdcEnableReadRawDataApi . . . 143
4.204 Parameter AdcEnableGroupStreamingResultReorder . . . 143

4.205 Parameter AdcSarEnableTempsenseApi . . . 144
4.206 Parameter AdcSarPowerOnTempSense . . . 144
4.207 Parameter AdcSarTempSenseVsupply . . . 145

5 Module Index **146**
5.1 Software Specification . . . 146

6 Module Documentation **147**
6.1 Adc driver . . . 147
6.1.1 Detailed Description . . . 147
6.1.2 Data Structure Documentation . . . 153
6.1.3 Macro Definition Documentation . . . 156
6.1.4 Types Reference . . . 166
6.1.5 Enum Reference . . . 167
6.1.6 Function Reference . . . 171
6.1.7 Variable Documentation . . . 194
6.2 Adc Sar IPL . . . 195
6.2.1 Detailed Description . . . 195
6.2.2 Data Structure Documentation . . . 198
6.2.3 Macro Definition Documentation . . . 205
6.2.4 Types Reference . . . 205
6.2.5 Enum Reference . . . 206
6.2.6 Function Reference . . . 211
6.3 Bctu IPL . . . 232
6.3.1 Detailed Description . . . 232
6.3.2 Data Structure Documentation . . . 234
6.3.3 Macro Definition Documentation . . . 238
6.3.4 Enum Reference . . . 238
6.3.5 Function Reference . . . 239

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	31.03.2022	NXP RTD Team	Prepared for release RTD S32K3 2.0.0

Chapter 2

Introduction

- Supported Derivatives
- Overview
- About This Manual
- Acronyms and Definitions
- Reference List

This User Manual describes NXP Semiconductor AUTOSAR ADC for S32K3XX. AUTOSAR ADC driver configuration parameters and deviations from the specification are described in Driver chapter of this document. AUTOSAR ADC driver requirements and APIs are described in the AUTOSAR ADC driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k344_mapbga257
- s32k344_mqfp172
- s32k324_mapbga257
- s32k324_mqfp172
- s32k314_mapbga257
- s32k314_mqfp172
- s32k312_mqfp172
- s32k312_mqfp100
- s32k322_mqfp172
- s32k322_mqfp100
- s32k342_mqfp172
- s32k342_mqfp100
- s32k341_mqfp172
- s32k341_mqfp100

All of the above microcontroller devices are collectively named as S32K3.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
ADC	Analog to Digital Converter
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CS	Chip Select
CTU	Cross Trigger Unit
BCTU	Body Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Signifigant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of ADC Driver	AUTOSAR Release 4.↩ 4.0
2	S32K3XX Reference Manual	Rev. 3 - 10/2021
3	S32K3xx Data Sheet	Rev. 3 - 10/2021
4	S32K344, Mask Set Errata for Mask 0P55A/1P55A	Rev. 30/Sep/2021
5	S32K312, Mask Set Errata for Mask 0P09C	Rev. 10/Jan/2022
6	S32K342, Mask Set Errata for Mask 0P97C	Rev. 13 Dec 2021

Chapter 3

Driver

- Requirements
- Driver Design Summary
- Hardware Resources
- Deviations from Requirements
- Driver Limitations
- Driver usage and configuration tips
- Runtime errors
- Symbolic Names Disclaimer

3.1 Requirements

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See Table Reference List).

It has vendor-specific requirements and implementation.

3.2 Driver Design Summary

The ADC Driver initializes and controls the internal Analog to Digital Converter Unit(s) of the microcontroller. It provides services to start and stop a conversion respectively to enable and disable the trigger source for a conversion. Furthermore it provides services to enable and disable a notification mechanism and routines to query the status and result of a conversion. The ADC Driver shall work on so called ADC channels. An ADC channel combines an analog input pin, the needed ADC circuitry itself and a conversion result register into an entity that can be individually controlled and accessed via the ADC Driver. The driver provides a service for Streaming management results and for De-Initialization of circuits.

3.3 Hardware Resources

This device provides three General-purpose ADC :ADC HW Unit 0 (ADC0), ADC HW Unit 1 (ADC1), ADC HW Unit 2(ADC2). The number of ADC hardware units and channels are derivative specific, so please consult the reference manual.

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR ADC Driver software specification in some places. The table below identifies the AUTOSAR requirements that are not implemented or out of scope for the ADC Driver.

Term	Definition
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently or out of scope for the ADC driver.

Requirement	Status	Description	Notes
SWS_Adc_00082	N/F	Service name: - IoHwAb_Adc↩ Notification - Syntax: - void IoHw↩ Ab_AdcNotification(void) - Sync/↩ Async: - Synchronous - Reentrancy↩ : - Non Reentrant - Parameters (in): - None - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - Will be called by the ADC Driver when a group conversion is completed for group . - Available via: - IoHwAb↩ _Adc.h -	"IoHwAb_AdcNotification<#group↩ ID>" is not fully implemented, because callback notification with this prototype can be configured per group, according with SWS_Adc_00084 and also in agreement with this statement from the 'Specification of ADC Driver AUTOSAR 4.4.0': "In this chapter all interfaces are listed where the target function could be configured. The target function is usually a call-back function. The names of this kind of interfaces are not fixed because they are configurable."
SWS_Adc_00345	N/S	The ADC module's priority mechanism shall allow suspending and resuming of channel group conversions.	Only abort/restart is supported
SWS_Adc_00341	N/S	If the priority mechanism is supported by the hardware: The ADC module shall support the static configuration option ADC_PRIORITY_HW to enable the priority mechanism using only the hardware priority mechanism.	HW prioritization mechanism is not available on platform
SWS_Adc_00339	N/S	If hardware priority mechanism is supported and selected: The ADC module shall allow the mapping of the configured priority levels (0-255) to the available hardware priority levels.	HW prioritization mechanism is not available on platform

Requirement	Status	Description	Notes
SWS_Adc_00337	N/S	The hardware prioritization mechanism shall be used in case of hardware triggered conversion requests.	HW prioritization mechanism is not available on platform
SWS_Adc_00488	N/S	In case development error reporting is activated:The API shall report the DET error ADC_E_POWER_STATE_NOT_SUPPORTED in case this API is called with an unsupported power state or the peripheral does not support low power states at all.	Adc_SetPowerState does not have the Adc_PowerStateType parameter. This parameter is passed and checked (for unsupported power state) through the AdcPreparePowerState function that is called before calling Adc_SetPowerState function.
SWS_Adc_00489	N/S	The API shall report a runtime error ADC_E_TRANSITION_NOT_POSSIBLE in case the requested power state cannot be directly reached from the current power state.	ADC hardware supports only two power states (full power and low power) and all transitions are valid; there is no need to report ADC_E_TRANSITION_NOT_POSSIBLE error.
SWS_Adc_00498	N/S	The API shall report a runtime error ADC_E_TRANSITION_NOT_POSSIBLE in case the requested power state cannot be directly reached from the current power state.All asynchronous operation, needed to reach the target power state, can be executed in background in the context of Adc_Main_PowerTransitionManager.	ADC hardware supports only two power states (full power and ow power) and all transitions are valid; there is no need to report ADC_E_TRANSITION_NOT_POSSIBLE error.
SWS_Adc_00479	N/S	Service name: - Adc_Main_PowerTransitionManager - Syntax: - void Adc_Main_PowerTransitionManager(void) - Service ID[hex]: - 0x14 - Description: - This API is cyclically called and supervises the power state transitions, checking for the readiness of the module and issuing the callbacks IoHwAb_Adc_NotifyReadyForPowerState (see AdcPowerStateReadyCbkRef configuration parameter). - Available via: - SchM_Adc.h -	Asynchronous Power State transition mode is not needed. It's not supported by hardware but still available in driver design and code (without any functionalities) since it's an Autosar API.
SWS_Adc_00499	N/S	This API executes any non-immediate action needed to finalize a power state transition requested by Adc_PreparePowerState().	Asynchronous Power State transition mode is not needed. It's not supported by hardware.
SWS_Adc_00500	N/S	The rate of scheduling shall be defined by Adc MainSchedulePeriod and shall be variable, as the function only needs to be called if a transition has been requested	Asynchronous Power State transition mode is not needed. It's not supported by hardware.
SWS_Adc_00501	N/S	This API shall also issue callback notifications to the eventually registered users (IoHwAbs) as configured, only in case the asynch mode is chosen.	Asynchronous Power State transition mode is not needed. It's not supported by hardware.

Requirement	Status	Description	Notes
SWS_Adc_00502	N/S	In case the ADC module is not initialized, this function shall simply return without any further elaboration. This is needed to avoid to elaborate uninitialized variables. No DET error shall be entered, because this condition can easily be verified during the startup phase (tasks started before the initialization is complete).Rationale: during the startup phase it can happen that the OS already schedules tasks, which call main functions, while some modules are not initialised yet. This is no real error condition, although need handling, i.e. returning without execution.Although the transition state monitoring functionality is mandatory, the implementation of this API is optional, meaning that if the HW allows for other ways to deliver notification and watch the transition state the implementation of this function can be skipped.	Asynchronous Power State transition mode is not needed. It's not supported by hardware.
SWS_Adc_00480	N/S	Service name: - IoHwAb_Adc_NotifyReadyForPowerState - Syntax: - void IoHwAb_Adc_NotifyReadyForPowerState(void) - Sync/Async: - Synchronous - Reentrancy: - Non Reentrant - Parameters (in): - None - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - The API shall be invoked by the ADC Driver when the requested power state preparation for mode is completed. - Available via: - IoHwAb_Adc.h -	Asynchronous Power State transition mode is not needed. It's not supported by hardware but still available in driver design and code (without any functionalities) since it's an Autosar API.
SWS_Adc_00460	N/S	These requirements are not applicable to this specification.	Not a requirement.

Requirement	Status	Description	Notes
SWS_Adc_91004	N/S	Service name: - Adc_Disable↩ GroupNotification (draft) - Syntax: - void Adc_DisableGroupNotification (Adc_GroupType Group) - Service ID[hex]: - 0x08 - Sync/Async: - Asynchronous - Reentrancy: - Reentrant - Parameters (in): - Group - Numeric ID of requested ADC Channel group. - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - Disables the notification mechanism for the requested ADC Channel group.Tags: atp.Status=draft - Available via: - Adc.h -	[17/05/2019] The function is sync instead of async because the multicore implementation is done with type 2 instead of type 4.
SWS_Adc_91002	N/S	Service name: - Adc_Disable↩ HardwareTrigger (draft) - Syntax: - void Adc_DisableHardwareTrigger (Adc_GroupType Group) - Service ID[hex]: - 0x06 - Sync/Async: - Asynchronous - Reentrancy: - Reentrant - Parameters (in): - Group - Numeric ID of requested ADC Channel group. - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - Disables the hardware trigger for the requested ADC Channel group.Tags: atp.Status=draft - Available via: - Adc.h -	[17/05/2019] The function is sync instead of async because the multicore implementation is done with type 2 instead of type 4.
SWS_Adc_91003	N/S	Service name: - Adc_EnableGroup↩ Notification (draft) - Syntax: - void Adc_EnableGroupNotification (Adc_GroupType Group) - Service ID[hex]: - 0x07 - Sync/Async: - Asynchronous - Reentrancy: - Reentrant - Parameters (in): - Group - Numeric ID of requested ADC Channel group. - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - Enables the notification mechanism for the requested ADC Channel group.Tags: atp.Status=draft - Available via: - Adc.h -	[17/05/2019] The function is sync instead of async because the multicore implementation is done with type 2 instead of type 4.

Requirement	Status	Description	Notes
SWS_Adc_91001	N/S	Service name: - Adc_Enable↩ HardwareTrigger (draft) - Syntax: - void Adc_EnableHardwareTrigger (Adc_GroupType Group) - Service ID[hex]: - 0x05 - Sync/Async: - Asynchronous - Reentrancy: - Reentrant - Parameters (in): - Group - Numeric ID of requested ADC Channel group. - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - Enables the hardware trigger for the requested ADC Channel group.Tags: atp.Status=draft - Available via: - Adc.h -	[17/05/2019] The function is sync instead of async because the multicore implementation is done with type 2 instead of type 4.
SWS_Adc_91000	N/S	Service name: - Adc_SetupResult↩ Buffer (draft) - Syntax: - Std_↩ ReturnType Adc_SetupResultBuffer (Adc_GroupType Group, const Adc↩ _ValueGroupType* DataBufferPtr) - Service ID[hex]: - 0x0c - Sync/Async: - Asynchronous - Reentrancy: - Reentrant - Parameters (in): - Group - Numeric ID of requested ADC channel group. - DataBufferPtr - pointer to result data buffer - Parameters (inout)↩ : - None - Parameters (out): - None - Return value: - Std_ReturnType - E_OK: result buffer pointer initialized correctly E_NOT_OK: operation failed or development error occured - Description: - Initializes ADC driver with the group specific result buffer start address where the conversion results will be stored. The application has to ensure that the application buffer, where DataBufferPtr points to, can hold all the conversion results of the specified group. The initialization with Adc_SetupResultBuffer is required after reset, before a group conversion can be started.Tags: atp.↩ Status=draft - Available via: - Adc.h -	[17/05/2019] The function is sync instead of async because the multicore implementation is done with type 2 instead of type 4.

Requirement	Status	Description	Notes
ECUC_Adc_00461	N/F	Name - AdcPowerState - Parent Container - AdcPowerState↩ Config - Description - Each instance of this parameter describes a different power state supported by the ADC HW. It should be defined by the HW supplier and used by the ADCDriver to reference specific HW configurations which set the ADC HW module in the referenced power state.At least the power mode corresponding to full power state shall be always configured. - Multiplicity - 1 - Type - EcucIntegerParamDef (Symbolic Name generated for this parameter) - Range - 0 .. 18446744073709551615 - - Default value - – - Post-Build Variant Value - false - Value Configuration Class - Precompile time - X - All Variants - Link time - – - - Post-build time - – - - Scope / Dependency - scope↩ : localdependency: This parameter shall only be configured if the parameter AdcLowPowerStatesSupport is set to true.	This requirement shall be fulfilled to avoid VSMD errors.Asynchronous Power State transition mode doesn't support by hardware. Maximum value of this node is restricted from 18446744073709551615 to 9223372036854775807 due to EB Tresos limitation: "Deviating from the AUTOSAR traceable [TPS_GST_00008], the formula language interpreter of E↩ B tresos Studio supports only the signed 64-bit range for integer values, i.e. from 0x8000000000000000 (-9223372036854775808) to 0x7fffffffffffffff (9223372036854775807)." "The formula language parser of EB tresos Studio is implemented in Java. It uses the long primitive type for integer values and therefore can only support the restricted range."

Requirement	Status	Description	Notes
ECUC_Adc_00011	N/F	Name - AdcChannelConvTime - Parent Container - AdcChannel - Description - Configuration of conversion time, i.e. the time during which the analogue value is converted into digital representation, (in clock cycles) for each channel, if supported by hardware.Implementation↩ Type: Adc_ConversionTimeType - Multiplicity - 0..1 - Type - EcucIntegerParamDef - Range - 0 .. 18446744073709551615 - - Default value - – - Post-Build Variant Multiplicity - true - Post-Build Variant Value - true - Multiplicity Configuration Class - Precompile time - X - VARIANT-PRE-↩ COMPILE - Link time - – - - Post-build time - X - VARIANT-P↩ OST-BUILD - Value Configuration Class - Precompile time - X - VARIANT-PRE-COMPILE - Link time - – - - Post-build time - X - VARIANT-P↩ OST-BUILD - Scope / Dependency - scope: local	Maximum value of this node is restricted from 18446744073709551615 to 9223372036854775807 due to EB Tresos limitation: "Deviating from the AUTOSAR traceable [TPS_GST_00008], the formula language interpreter of E↩ B tresos Studio supports only the signed 64-bit range for integer values, i.e. from 0x8000000000000000 (-9223372036854775808) to 0x7fffffffffffffff (9223372036854775807)." "The formula language parser of EB tresos Studio is implemented in Java. It uses the long primitive type for integer values and therefore can only support the restricted range."

Requirement	Status	Description	Notes
ECUC_Adc_00455	N/F	Name - AdcChannelHighLimit - Parent Container - AdcChannel - Description - High limit - used for limit checking. - Multiplicity - 0..1 - Type - EcucIntegerParamDef - Range - 0 .. 18446744073709551615 - - Default value - – - Post-Build Variant Multiplicity - false - Post-Build Variant Value - false - Multiplicity Configuration Class - Pre-compile time - X - All Variants - Link time - – - - Post-build time - – - - Value Configuration Class - Pre-compile time - X - All Variants - Link time - – - - Post-build time - – - - Scope / Dependency - scope↩ : localdependency: AdcEnable↩ LimitCheck: not available if limit checking is not globally enabled. AdcChannelLimitCheck: not available if channel specific limit check is not enabled. AdcChannelLowLimit↩ : has to be greater or equal than AdcChannelLowLimit.	Maximum value of this node is restricted from 18446744073709551615 to 9223372036854775807 due to EB Tresos limitation: "Deviating from the AUTOSAR traceable [TPS_GST_00008], the formula language interpreter of E↩ B tresos Studio supports only the signed 64-bit range for integer values, i.e. from 0x8000000000000000 (-9223372036854775808) to 0x7fffffffffffffff (9223372036854775807)." "The formula language parser of EB tresos Studio is implemented in Java. It uses the long primitive type for integer values and therefore can only support the restricted range."

Requirement	Status	Description	Notes
ECUC_Adc_00454	N/F	Name - AdcChannelLowLimit - Parent Container - AdcChannel - Description - Low limit - used for limit checking. - Multiplicity - 0..1 - Type - EcucIntegerParamDef - Range - 0 .. 18446744073709551615 - - Default value - – - Post-Build Variant Multiplicity - false - Post-Build Variant Value - false - Multiplicity Configuration Class - Pre-compile time - X - All Variants - Link time - – - - Post-build time - – - - Value Configuration Class - Pre-compile time - X - All Variants - Link time - – - - Post-build time - – - - Scope / Dependency - scope↩ : localdependency: AdcEnableLimit↩ Check: not available if limit checking is not globally enabled. AdcChannel↩ LimitCheck: not available if channel specific limit check is not enabled. AdcChannelHighLimit: has to be less or equal than AdcChannelHighLimit.	Maximum value of this node is restricted from 18446744073709551615 to 9223372036854775807 due to EB Tresos limitation: "Deviating from the AUTOSAR traceable [TPS_GST_00008], the formula language interpreter of E↩ B tresos Studio supports only the signed 64-bit range for integer values, i.e. from 0x8000000000000000 (-9223372036854775808) to 0x7fffffffffffffff (9223372036854775807)." "The formula language parser of EB tresos Studio is implemented in Java. It uses the long primitive type for integer values and therefore can only support the restricted range."

Requirement	Status	Description	Notes
ECUC_Adc_00290	N/F	Name - AdcChannelSampTime - Parent Container - AdcChannel - Description - Configuration of sampling time, i.e. the time during which the value is sampled, (in clock cycles) for each channel, if supported by hardware.Implementation↩ Type: Adc_SamplingTimeType - Multiplicity - 0..1 - Type - EcucIntegerParamDef - Range - 0 .. 18446744073709551615 - - Default value - – - Post-Build Variant Multiplicity - true - Post-Build Variant Value - true - Multiplicity Configuration Class - Precompile time - X - VARIANT-PRE-↩ COMPILE - Link time - – - - Post-build time - X - VARIANT-P↩ OST-BUILD - Value Configuration Class - Precompile time - X - VARIANT-PRE-COMPILE - Link time - – - - Post-build time - X - VARIANT-P↩ OST-BUILD - Scope / Dependency - scope: local	Maximum value of this node is restricted from 18446744073709551615 to 9223372036854775807 due to EB Tresos limitation: "Deviating from the AUTOSAR traceable [TPS_GST_00008], the formula language interpreter of E↩ B tresos Studio supports only the signed 64-bit range for integer values, i.e. from 0x8000000000000000 (-9223372036854775808) to 0x7fffffffffffffff (9223372036854775807)." "The formula language parser of EB tresos Studio is implemented in Java. It uses the long primitive type for integer values and therefore can only support the restricted range."

Requirement	Status	Description	Notes
ECUC_Adc_00401	N/F	Name - AdcHwTrigTimer - Parent Container - AdcGroup - Description - Reload value of the ADC module embedded timer (only if supported by ADC hardware).ImplementationType↩ : Adc_HwTriggerTimerType - Multiplicity - 0..1 - Type - EcucIntegerParamDef - Range - 0 .. 1844674407370955161 5 - - Default value - – - Post-Build Variant Multiplicity - true - Post-Build Variant Value - true - Multiplicity Configuration Class - Pre-compile time - X - VARIANT-↩ PRE-COMPILE - Link time - – - - Post-build time - X - VARIANT-P↩ OST-BUILD - Value Configuration Class - Pre-compile time - X - VARIANT-PRE-↩ COMPILE - Link time - – - - Post-build time - X - VARIANT-P↩ OST-BUILD - Scope / Dependency - scope↩ : localdependency: AdcTriggSrcHw: Valid only if the group is configured to be triggered by a hardware event.	Maximum value of this node is restricted from 18446744073709551615 to 9223372036854775807 due to EB Tresos limitation: "Deviating from the AUTOSAR traceable [TPS_GST_00008], the formula language interpreter of E↩ B tresos Studio supports only the signed 64-bit range for integer values, i.e. from 0x8000000000000000 (-9223372036854775808) to 0x7fffffffffffffff (9223372036854775807)." "The formula language parser of EB tresos Studio is implemented in Java. It uses the long primitive type for integer values and therefore can only support the restricted range."

3.5 Driver Limitations

- AdcPreSamplingOnce is always enabled and cannot be modified.
- Unused ADC_IPW_MAX_HARDWARE_TRIGGERS define when CTU hardware trigger API is disabled.

3.6 Driver usage and configuration tips

3.6.1 Calibration function

ADC driver provides Adc_Calibrate() API to eliminate the errors between the actual values and the expected converted values, which might be generated by variations in manufacturing and various runtime environmental effects. The API can be called explicitly by the user at any time after the driver was initialized and while there are no ongoing conversions. Calibration is not automatically started by initialization function.

Configuration-time-prerequisites:

- Enable AdcEnableCalibration in AutosarExt container.



3.6.2 Self-Test function

For safety applications, it is important to verify correct operation at regular intervals. ADC driver provides Adc_SelfTest() API for this purpose to check its components and flags errors, if any are found. The API can be called explicitly by the user at any time after the driver was initialized and while there are no ongoing conversions.

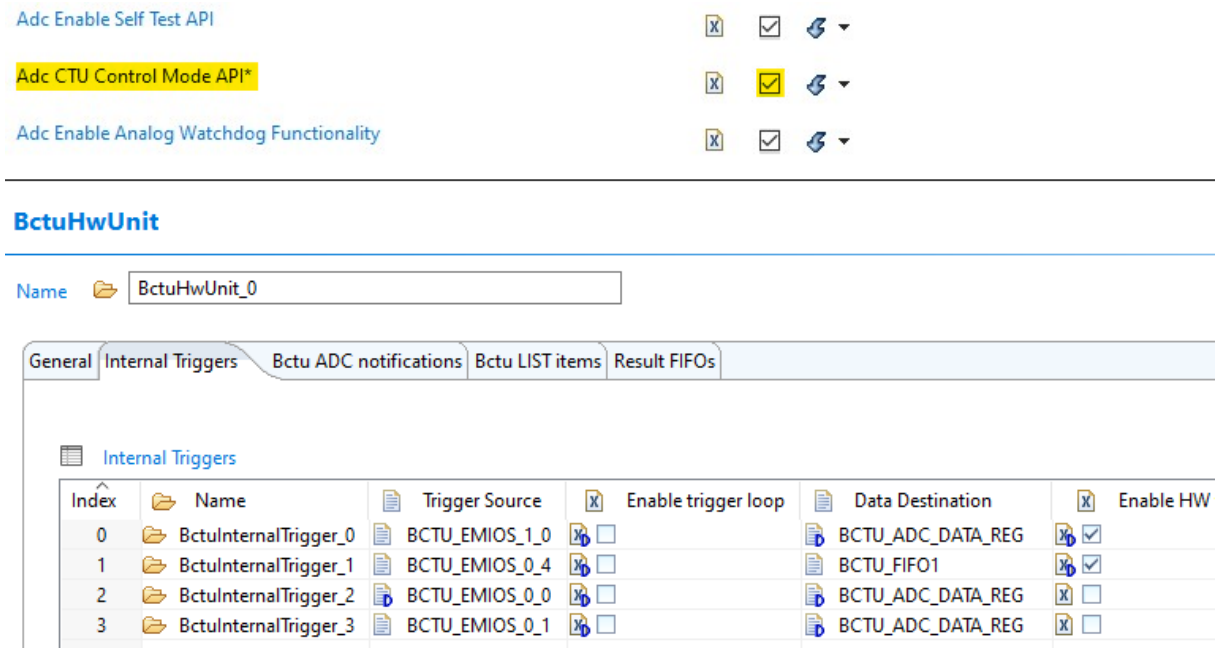
Configuration-time-prerequisites:

- Enable AdcEnableSelfTest in AutosarExt container.



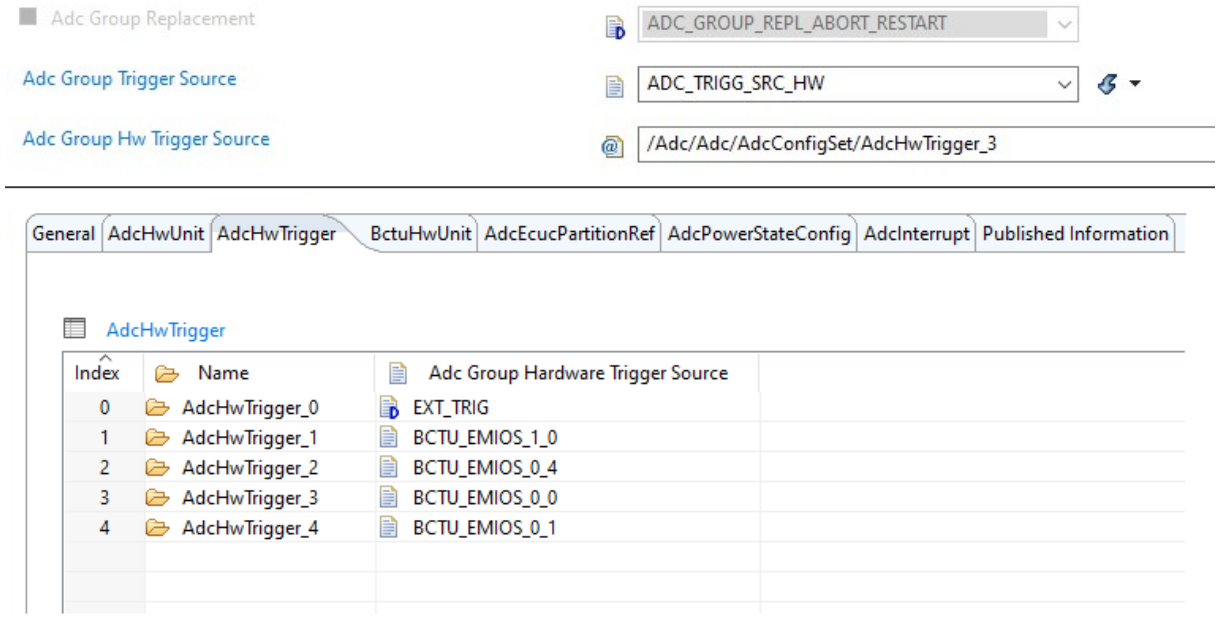
3.6.3 BCTU control mode

- Control mode is used by enabling the control mode API from the configuration and by calling the Adc_EnableCtuControlMode() function at runtime.
- In control mode the BCTU assumes complete control over the ADC module and groups can no longer be used. If enabled, triggered mode cannot be used in parallel.
- Software triggered groups can be used on different ADC units which are not controlled by BCTU
- Also SW triggered conversions can be used on the ADC unit controlled by BCTU using Adc_CtuStartConversion()
- In control mode the BCTU hardware is configured as specified directly by the user in the BCTU configuration container and with dedicated APIs at runtime for certain parameters.
- The data can be obtained by calling functions dedicated for reading the FIFOs or data registers from the notifications, or by DMA transfers
- A notification can be configured for each BCTU event.
- The BCTU is configured only once when calling Adc_EnableCtuControlMode() and not every time a hardware trigger is enabled.
- This mode enables the usage of functions prefixed with Adc_Ctu, e.g. Adc_CtuReadConvData(), Adc_CtuEnableHwTrigger(), etc.
- In Tresos, CTU control mode is enabled by ticking its box and this enables the configuration of the BCTU HW unit(see image)



3.6.4 BCTU triggered mode

- In this mode the BCTU is not directly configurable by the users. The BCTU configuration is generated based on the group configured information
- Trigger mode is the default mode of triggering (if Adc_EnableCtuControlMode() is not called)
- In Trigger mode the BCTU is used strictly as a trigger source for ADC groups and the result data is read from the adc module.
- This is done by configuring the BCTU list to look exactly like the list of channels in the desired group and activating only the trigger source associated to that group.
- In the end of list interrupt coming from the BCTU, the ADC driver copies and process the adc data from the result registers of the adc module and optionally calls a user notification mapped for the active group.
- The BCTU is reconfigured each time Adc_EnableHardwareTrigger() is called, so it triggers the channels in the same sequence as they are defined in the groups.
- In Tresos, the triggers must be configured from a separate tab.

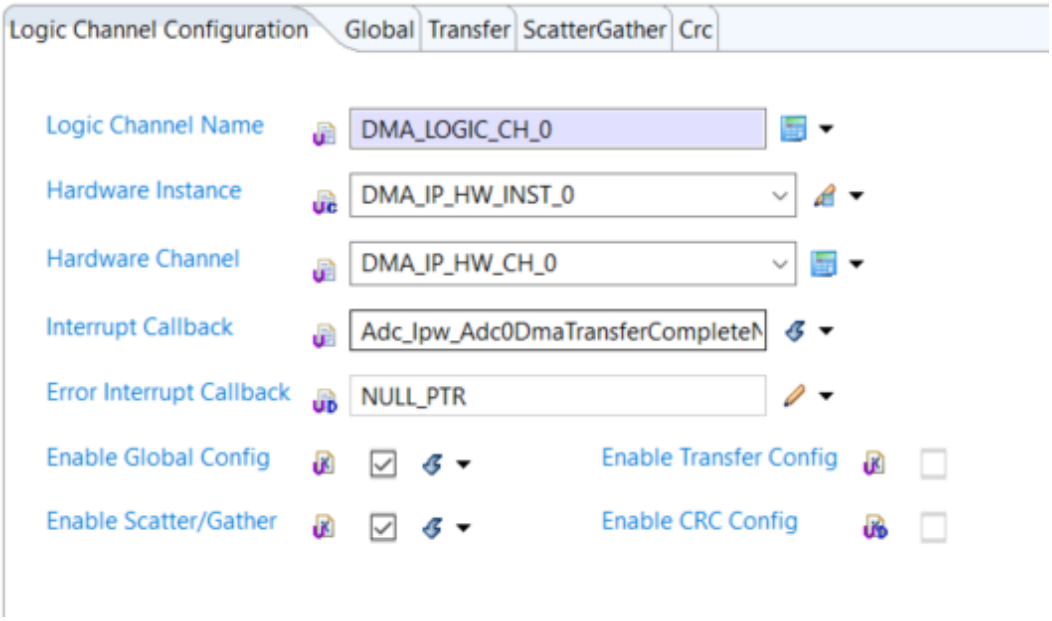


3.6.5 DMA transfer

ADC driver provides the DMA transfer mechanism for transferring the conversion results directly from data registers to system memory via Direct Memory Access (DMA). This is opposed to Interrupt transfer mode, when ADC driver code moves the data from registers to user buffer. This feature can be configured independently for each ADC HW unit. The buffer containing conversion results should be declared within a NO_CACHEABLE data section.

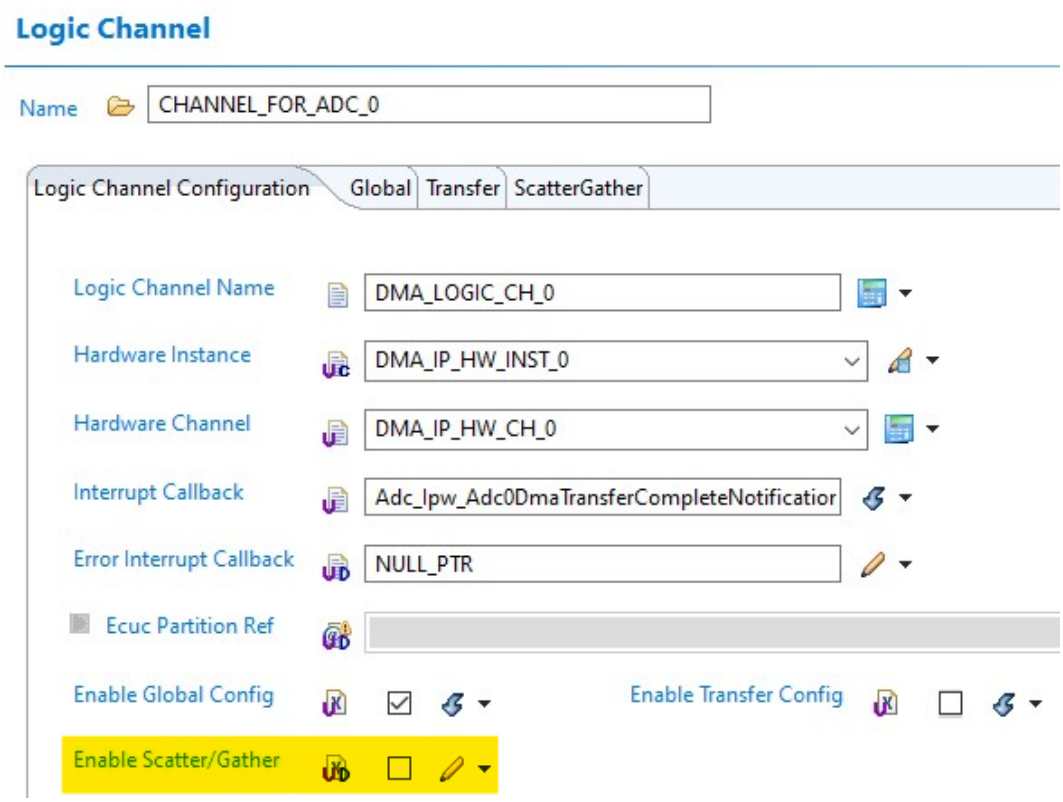
Configuration-time-prerequisites:

- AdcEnableDmaTransferMode needs to be enabled in AutosarExt container
- Configured AdcTransferType as ADC_DMA in AdcHwUnit container
- Mcl configuration for the logical channel used for adc must have Adc_Ipw_AdcXDmaTransferComplete↩ Notification set as interrupt callback (with X defining the Adc instance used, for example Adc_Ipw_Adc0↩ DmaTransferCompleteNotification()), global config and scatter/gather enabled. The DMA source must be configured as the ADC instance used (SarAdc). Also, the number of ScatterGather elements configured for the Mcl logical channel must be equal to the maximum number of AdcChannels configured in Adc groups. The elements in the ScatterGather configuration must be linked consecutively (element n linked to element n+1), except the last element in the list that should be marked as such. The other parameters for the ScatterGather elements will be overwritten at runtime by the ADC driver.



Note

- When using multi-core feature, the DMA channel should have the same partition as the ADC module.
- When using Enable Config Time Support feature, if ADC is configured as VariantPreCompile, Mcl cannot be configured as VariantPostBuild.
- AdcEnableDmaTransferMode shouldn't be configured simultaneously with Adc_SetChannel() or Adc_↩ EnableChannel()/Adc_DisableChannel() because there is a risk that DMA TCD size will be modified at runtime.
- When using DMA, the results transferred to buffers are 15-bit wide, same as Adc CDATA register, for any resolution selected. If results are accessed directly from buffers or via pointer retrieved by Adc_GetStreamLastPointer() results will have the width mentioned previously. If results are retrieved via Adc_ReadGroup(), result values are returned having width according to the selected resolution and alignment.
- For groups having only one channel, the driver does not use the Scatter-Gather mode. So Scatter-Gather must be disabled in DMA Channel Configuration.



3.6.6 Injected conversions

Adc driver supports to configure groups with conversion type Injected for high priority conversions. An injected conversion is a conversion chain that can be injected at any time, including into an ongoing normal conversion chain. Injected conversion can only be run in One-Shot mode, and can only interrupt a normal conversion. When an injected chain is inserted into an ongoing normal chain, the ongoing normal channel is suspended and the injected channel request is processed. After completing the last channel in the injected chain, normal conversion resumes from the channel at which it was suspended. Depending on the hardware capabilities, a Software Normal Group can be active in the Queue at the same time as a Hardware Injected group, or a Software Injected Group can be active at the same time with a Hardware Normal Group, so that 2 groups can be converted almost in parallel. For more details, please refer to the Reference Manual of the microcontroller, chapter Injected conversion.

Configuration-time-prerequisites:

- In AdcGroup container, configure AdcGroupConversionType as ADC_CONV_TYPE_INJECTED, Adc↩ GroupConversionMode as ADC_CONV_MODE_ONESHOT and the AdcGroupPriority must be 255 (highest)
- If it is a Software group, AdcUseSoftwareInjectedGroups needs to be enabled in AutosarExt container

3.6.7 Set Channel Optimization

ADC driver provides an optional configuration parameter that allows the user to change the configuration of a group at runtime. According to Autosar specification, the group definition (list of channels) should be fixed at configuration time; if a different set of channels needs to be converted, another group must be started. However, in some applications stopping the group and starting another takes too long for the given time constraints. For this type of use case, Adc_SetChannel() can be used to quickly change the configuration of a group at runtime: the list of channels, and if applicable on the hardware platform - the set of associated conversion timing information for each channel.

Configuration-time-prerequisites:

- The AdcEnableSetChannel parameter needs to be enabled in AutosarExt container



- If channels need to be updated in ISR notification, AdcEnableInitialNotification parameter can be enabled in AutosarExt container



- The initial notification function needs to be enable for each Group where it needs to be used



Adc_SetChannel() API can be called by the user application whenever it is needed. In the interrupt routine that handled the conversion complete event, Adc driver will check if Adc_SetChannel() has been called and will update the required hardware registers to match the new configuration. The API can be called from the standard Autosar group conversion complete notification, but this notification has the disadvantage that it's called at the very end of ADC interrupt processing, leaving no time to do the updates in ADC hardware registers. So the very next conversion will not be affected by the changes, but the one after that will be.

Note

- If the Adc_SetChannel() is called before ADC receives the first hardware trigger signal and after calling Adc_EnableHardwareTrigger(), the results of this trigger will not be correct since the new list of channels will be updated in the interrupt of first trigger. To start a conversion with the new list, another signal needs to be provided.
- In case of calling Adc_SetChannel() after Adc_StartGroupConversion(), the list that will be converted depends on total conversion time. If the interrupt event happened before executing Adc_SetChannel(), the old list will be converted. Otherwise, if the total conversion time is longer than the time required to execute both functions (Adc_SetChannel() and Adc_StartGroupConversion()), then the results of the new list will be recorded.
- Sample index will be reset and result data will be overwritten starting from the first sample after runtime channels are updated in ISR.
- The feature AdcEnableInitialNotification can be used by the user application to call Adc_SetChannel() API before ADC driver updates the hardware configuration for the next conversion.

3.6.8 Enable/Disable channel

The ADC driver provides optional functions Adc_EnableChannel() and Adc_DisableChannel() to enable/disable one ADC channel of a given software triggered (non-injected) ADC group.

Configuration-time-prerequisites:

- The Adc Enable/Disable Channels API parameter needs to be enabled in AutosarExt container



Note

- The user should use generated symbolic channel name defines (e.g. AdcChannel_0_0), because the API assumes ChannelId to have the following format: logical channel ID on bits until position defined by ADC_CHANNEL_SYMBOLIC_NAME_SHIFT_HW_UNIT_ID_U16, and logical unit ID from that position
- The driver will not update the values in result buffers corresponding to disabled channels, keeping in the buffer the last results from when the channel was enabled.
- Adc Enable/Disable Channels API and Adc Set Channel API cannot be enabled simultaneously

3.6.9 Conversion Time Once

ADC driver provides Conversion Time Once feature to enable/disable configuring conversion time and averaging settings only when initializing the ADC module. If this feature is enabled, these settings will be done only once in Adc_Init() with values configured per unit, in AdcHwUnit container. If disabled, these settings will be done whenever a group is started, using values from AdcGroup container. When the feature is enabled, the latency of APIs that start group conversions will be reduced. If Adc_SetClockMode() API is also used, the driver allows the user to configure alternate timing settings (for ALTERNATE mode), both at Unit and Group level.

Configuration-time-prerequisites:

- Enable AdcConvTimeOnce in AutosarExt container
- Configure the values will be used in AdcNormalConvTiming or AdcNormalAlternateTiming (if Adc_SetClockMode() was invoked before wih Alternate mode) in AdcHwUnit container

Note

- The AdcPreSamplingOnce feature is always enabled from configurator but readonly. Adc_Init() will initialize once the presample for all channels in AdcChannel container per unit.

3.6.10 Bypass Abort Chain Check

When a conversion chain is aborted, ADC driver will block program execution to wait for the conversion chain to be stopped. This wait can be skipped by enabling AdcBypassAbortChainCheck. Doing this will improve performance at the cost of HW-SW coherency no longer being guaranteed. In this case, the user must make sure he does not call an ADC service before the hardware reaches the correct state.

Configuration-time-prerequisites:

- Enable AdcBypassAbortChainCheck in AutosarExt container



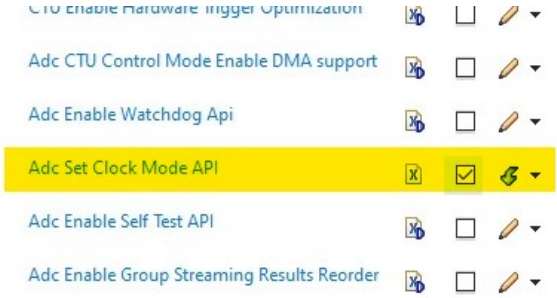
Note

This can be enabled if no queues are being used.

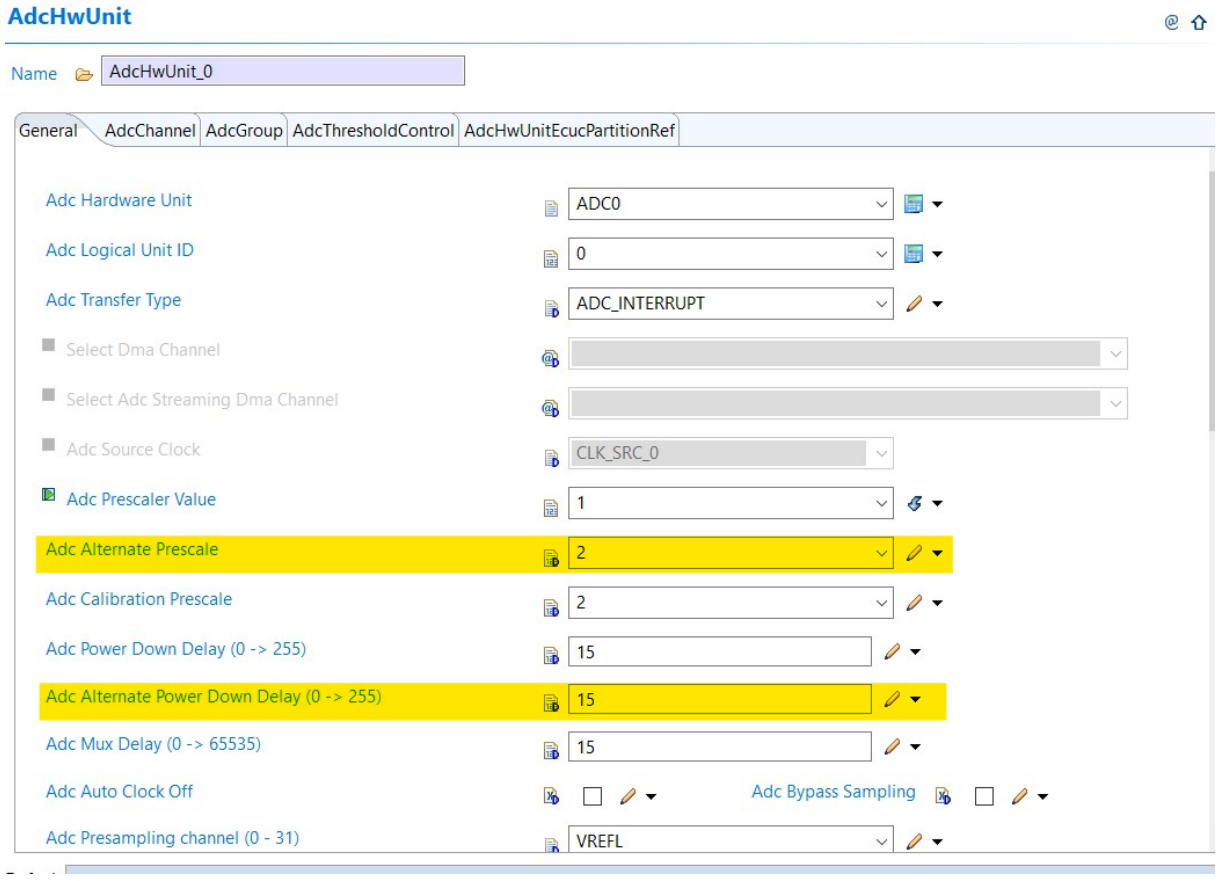
3.6.11 Set Clock Mode

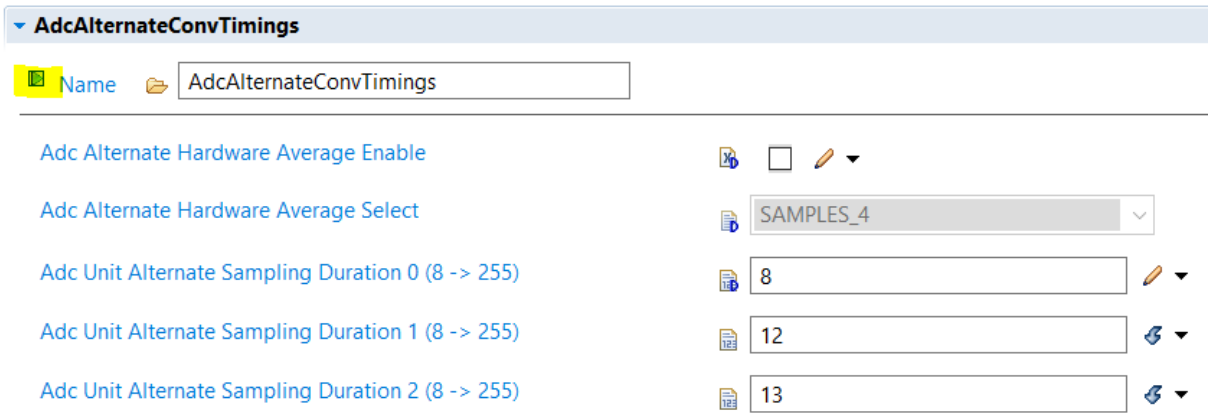
ADC driver provides an optional API Adc_SetClockMode() and configuration parameters to compensate the changes of the input clock of the controlled hardware. By enabling this feature, beside the normal clock configuration set, user can configure an alternate clock configuration set and can switch between normal and alternate at runtime using Adc_SetClockMode(). One clock configuration set includes the ADC parameters affected by clock configuration, such as: ADC power down delay, ADC prescale and sampling duration. Configuration-time-prerequisites:

- To use the set clock mode feature, "Adc Set Clock Mode API" needs to be enabled in AutosarExt container first:

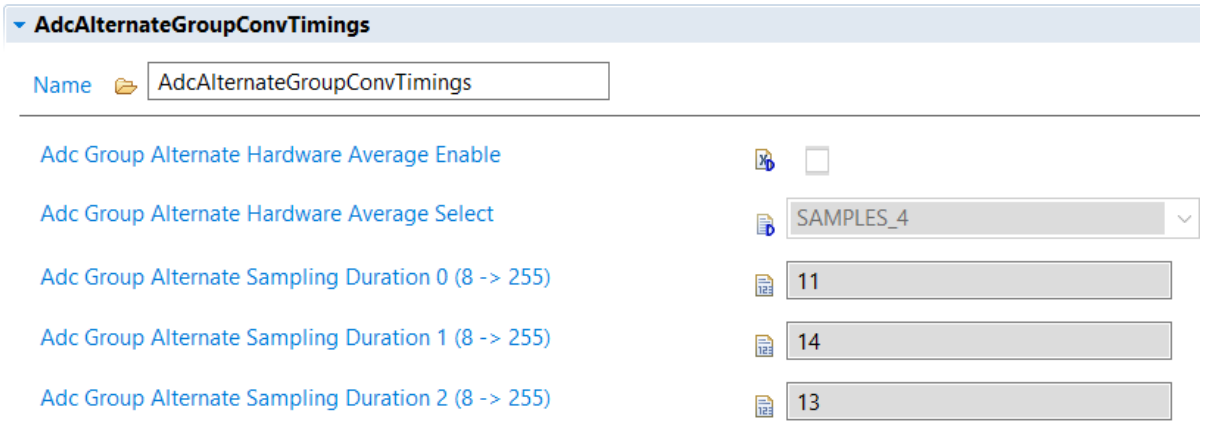


- Enable and configure the alternate clock configuration set at HW unit level. If the Conversion Time Once feature is enabled, normal and alternate sampling duration can be configured as well.





- Enable and configure alternate clock configuration set at group level, if Conversion Time Once feature is disabled.

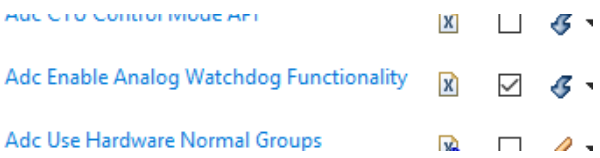


3.6.12 Analog Watchdog feature

ADC driver provides the Analog Watchdog feature for determining whether the result of a conversion lies within a given guard area specified by an upper and a lower threshold value named THRH and THRL, respectively. The upper/lower values are set at configuration time and can be changed at runtime via the API Adc_ConfigureThreshold(). Each channel can be configured to use this feature or not. If the result of a conversion does not lie within the configured range, an interrupt will occur and a user configured notification will be called. For more details about Analog Watchdog feature see the reference manual of the microcontroller. Adc driver also provide optional functions to enable/disable the analog watchdog event notification of each configured ADC channel: Adc_EnableWdgNotification() and Adc_DisableWdgNotification()

Configuration-time-prerequisites:

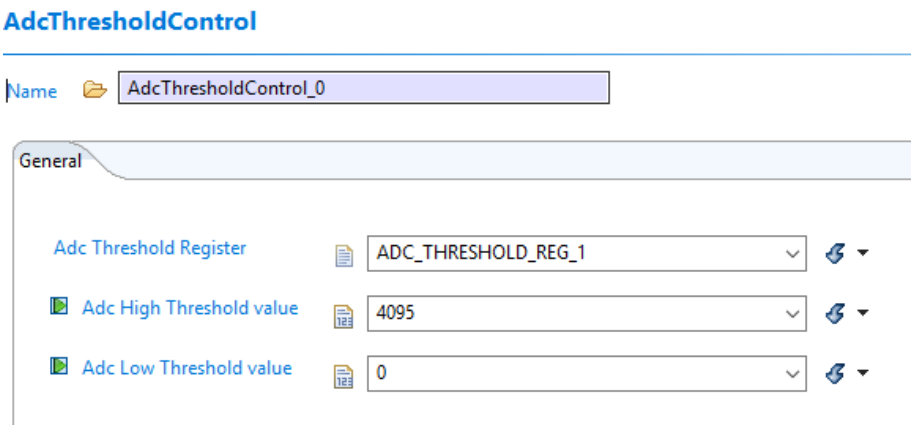
- To use the Analog Watchdog feature, AdcEnableWatchdogFunctionality needs to be enabled in AutosarExt container first:



- Configure the channel to use Analog Watchdog feature in AdcChannel container. Enable AdcEnableThresholds and configure the AdcThresholdRegister and AdcWdogNotification:



- The upper/lower threshold values need to be configured in AdcThresholdControl container:



- To use Adc_ConfigureThreshold(), AdcEnableThresholdConfiguration in AutosarExt container needs to be enabled:



Note

- When using Adc_EnableWdgNotification() and Adc_DisableWdgNotification(), symbolic channel name defines should be used (e.g. AdcChannel_0_0), because these functions assumes that ChannelId has the following format: logical channel ID on bits until position defined by ADC_CHANNEL_SYMBOLIC↩ _NAME_SHIFT_HW_UNIT_ID_U16, and logical unit ID from that position
- When using Adc_ConfigureThreshold(), symbolic threshold control name defines should be used (e.g. AdcThresholdControl_0_0), because the function assumes that ThresholdControlIndex has the following format: logical threshold ID on bits until position defined by ADC_THRESHOLD_SYMBOLIC_NA↩ ME_SHIFT_HW_UNIT_ID_U16, and logical unit ID from that position

3.6.13 Without Interrupts group

ADC driver provides this feature as a mechanism to reduce the CPU load. The basic functionality of ADC driver requires that an interrupt event occurs after each group conversion, during which the software will copy the data from data registers to the user buffer and update the Group status. If Without Interrupts feature is configured, the conversions can run without software intervention (without any interrupt events generated).

Configuration-time-prerequisites:

- Enable AdcWithoutInterrupts for the group using without interrupt feature in AdcGroup container:

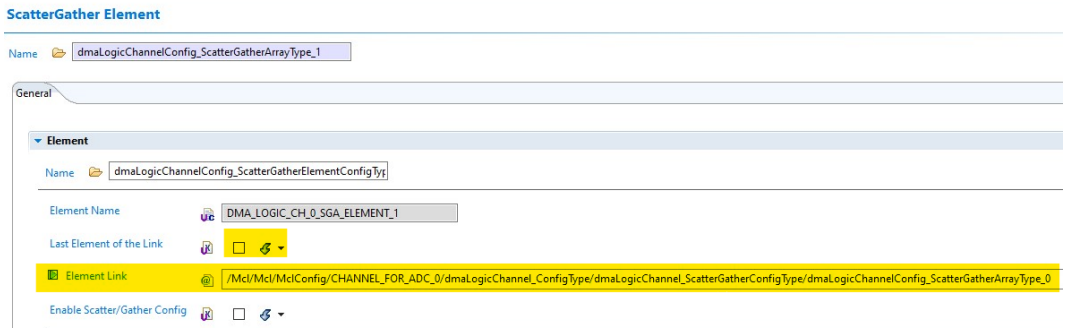


For the case of groups configured with ADC_ACCESS_MODE_SINGLE the application can retrieve the results by calling Adc_ReadGroup(). The result buffer configured by user via Adc_SetupResultBuffer() is no longer used to store the results, but must still be registered to avoid raising DET required by AUTOSAR Standard. If HW unit transfer type is configured as INTERRUPT (not with DMA), Adc_ReadGroup() reads the results directly from hardware registers (if the flags indicate that the group conversion was completed) and updates the group state afterwards. If HW unit transfer type is set to DMA, the results are transferred via DMA into an internal buffer. Adc_ReadGroup() reads the results from this buffer and updates the group state afterwards.

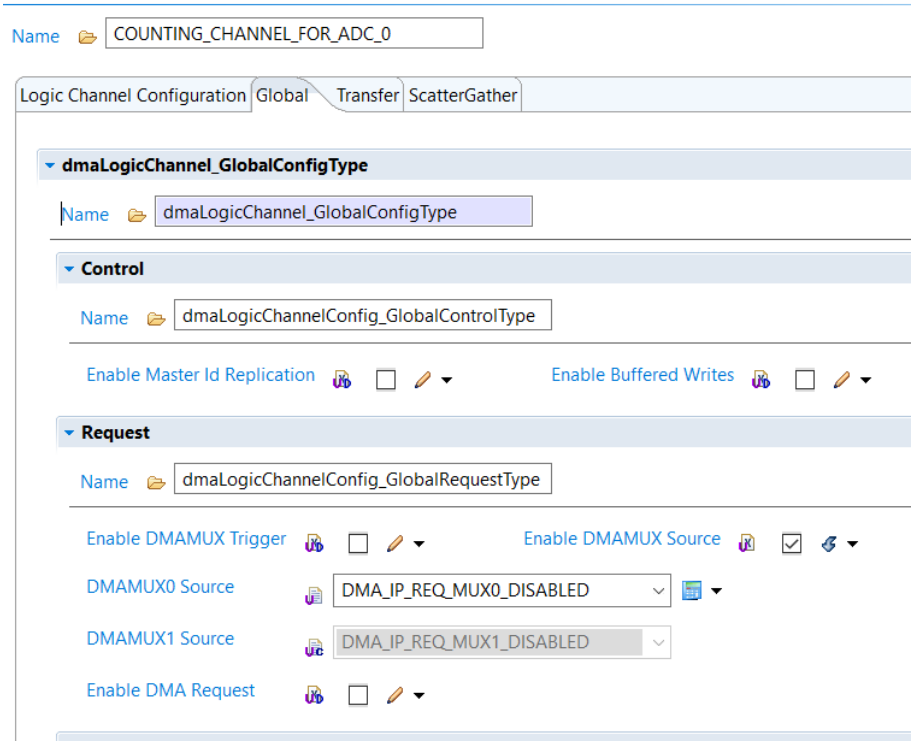
Note

- The Without Interrupt optimization is also supported for groups configured with ADC_ACCESS_M↩ ODE_STREAMING, with following restrictions: HW unit transfer type is DMA, group buffer mode is CIRCULAR and group has enabled Streaming Results Reorder (requires Group Streaming Results Reorder to be enabled in General AutosarExt). In this case a second DMA channel is required to be configured (named Select Adc Streaming Dma Channel) and the results are transferred in 2 stages:

1. Conversion results for 1 sample of all channels are transferred into an internal buffer via the first D↩ MA channel configured with ScatterGather. IMPORTANT NOTE: this channel must be configured as described in DMA transfer, with only one difference: the last ScatterGather element in the array must be linked to the first one, creating a circular scatter gather list. This introduces the limitation that on the same HW unit groups having without interrupt DISABLED cannot be used. Also all groups having without interrupt ENABLED must have the same number of channels as elements in ScatterGather list.



2. Transfer complete of the last scatter gather element (when last ADC result for the current sample is available) triggers Streaming Dma Channel which transfers results for 1 sample from the internal buffer into the buffer registered by user via Adc_SetupResultBuffer(). The ADC driver configures the DMA channel to also support Streaming Number Samples > 1. The configuration required for the Streaming Dma Channel is shown in picture below:



- For hardware triggered groups configured with ADC_ACCESS_MODE_SINGLE, multiple channels and transfer type set to DMA, to support multiple occurrences of a trigger, the ScatterGather elements need to be configured the same as for ADC_ACCESS_MODE_STREAMING (described above, at point 1). For software triggered groups this is not mandatory. Multiple calls to Adc_StartGroupConversion() are supported in both cases, because the TCD is reconfigured in Adc_StartGroupConversion().

3.6.14 Without Dma group

When this feature is disabled and DMA transfer is used, the ADC driver reconfigures the DMA channel at each call to Adc_StartGroupConversion() or Adc_EnableHardwareTrigger(). This feature allows complete bypass of the DMA configuration done by ADC driver for the group, so it will not affect other groups for which the ADC driver has already configured the DMA. This feature can be useful for example when starting a normal software group while an injected hardware trigger with ADC_ACCESS_MODE_STREAMING is already enabled - to allow the hardware triggered group to execute correctly, the DMA configuration for the software triggered group must be bypassed to not reconfigure the DMA channel of the active hardware triggered group. The results for the software group configured as Without Dma can be read with Adc_ReadGroup(), but the channels of the 2 groups must not be overlapped. Otherwise results will not be read, because the valid bit of the result register will be cleared by DMA transfers.
The feature can only be enabled if following conditions are met:

- ADC Unit Transfer Type is ADC_DMA
- Only for software triggered, oneshot, single access mode groups
- Group Without Interrupts is enabled

3.6.15 External DMA channel feature

The ADC driver shall support configuring the DMA channel externally by using a boolean parameter named "↩ External Dma Chan Config" at group configuration level. In this mode, ADC driver will only enable/disable the DMA request for the unit. The user must configure the DMA to transfer data according to his needs.

Configuration-time-prerequisites:

- To use the External DMA channel feature, AdcExtDMAChanEnable needs to be enabled in each group container first:



Note

- When External Dma Chan Config is enabled, the driver will not configure any TCDs for DMA
- The user still needs to register Adc_Ipw_Adc0DmaTransferCompleteNotification() as a notification for DMA, but if External Dma Chan Config is enabled for that group, the function only updates the group status and calls the GroupNotification.
- If a group is in streaming access mode, the code for updating the destination in Adc_Ipw_Adc0Dma↩ TransferCompleteNotification() (ie: update destination address of TCD, enable DMA hardware request) will not be executed for that group so this will need to be handled by the user in the notification
- If this feature is enabled simultaneously with AdcWithoutInterrupts (Without Interrupts group), the user is not required to register Adc_Ipw_Adc0DmaTransferCompleteNotification() as a notification for DMA, because the group status is updated when calling Adc_ReadGroup(). It is required for the application to register a result buffer via Adc_SetupResultBuffer(), but the buffer and DMA transfers are completely managed by the application, so the actual buffer is used only by Adc_ReadGroup(). Adc_ReadGroup() assumes that the registered buffer is one dimensional, with number of elements equal to the number of channels in the group and that it stores the latest conversion result (latest sample). This assumption is required to determine if latest conversion has occurred before updating the group status. Additionally, for groups configured with Ext DMA and Without interrupt, the Adc_SetupResultBuffer() resets the buffer to an invalid value so Adc_ReadGroup() can determine if the result has been transferred and status should be updated. Note that if updating the group status is not required, the user can read directly from the DMA destination buffer without calling Adc_ReadGroup().

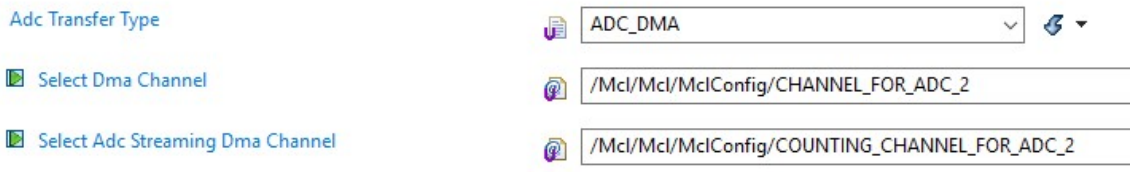
3.6.16 Optimize DMA streaming groups

The ADC driver provides an optional configuration parameter for reducing the number of DMA interrupts required for processing the conversions of ADC groups that are configured with ADC_ACCESS_MODE_STREAMING option. Instead of having an interrupt after every end of chain group conversion, only one or two interrupts will be raised for a stream of N conversions: one after N/2 conversions, and one after all N conversions are completed. With this feature enabled, if a group has more than one channel, a second DMA channel is required, named streaming DMA channel. In this case, the driver will use 2 DMA channels configured: transferring DMA channel and streaming DMA channel. Transferring DMA channel is used for moving data from ADC data register (CDR) to an intermediate buffer. Streaming DMA acts as a hardware counter which has CITER/BITER value equal to group sample as configured by AdcStreamingNumSamples parameter. After every end of chain conversion, ADC hardware will trigger a request to transferring DMA channel to complete its major loop. After that, transferring DMA will trigger streaming DMA by a Major Elink request. Streaming DMA will transfer data from intermediate buffer to user buffer and also decrease DMA CITER count by 1. When all ADC samples are completed, the streaming DMA channel will finish its major loop and automatically disable the hardware request from DMA (this is done by disabling DMA in Adc_Ipw_↩ Adc0DmaTransferCompleteNotification). Both interrupts occurring after half of conversions are done and after all of them are done will be raised by the streaming DMA channel.

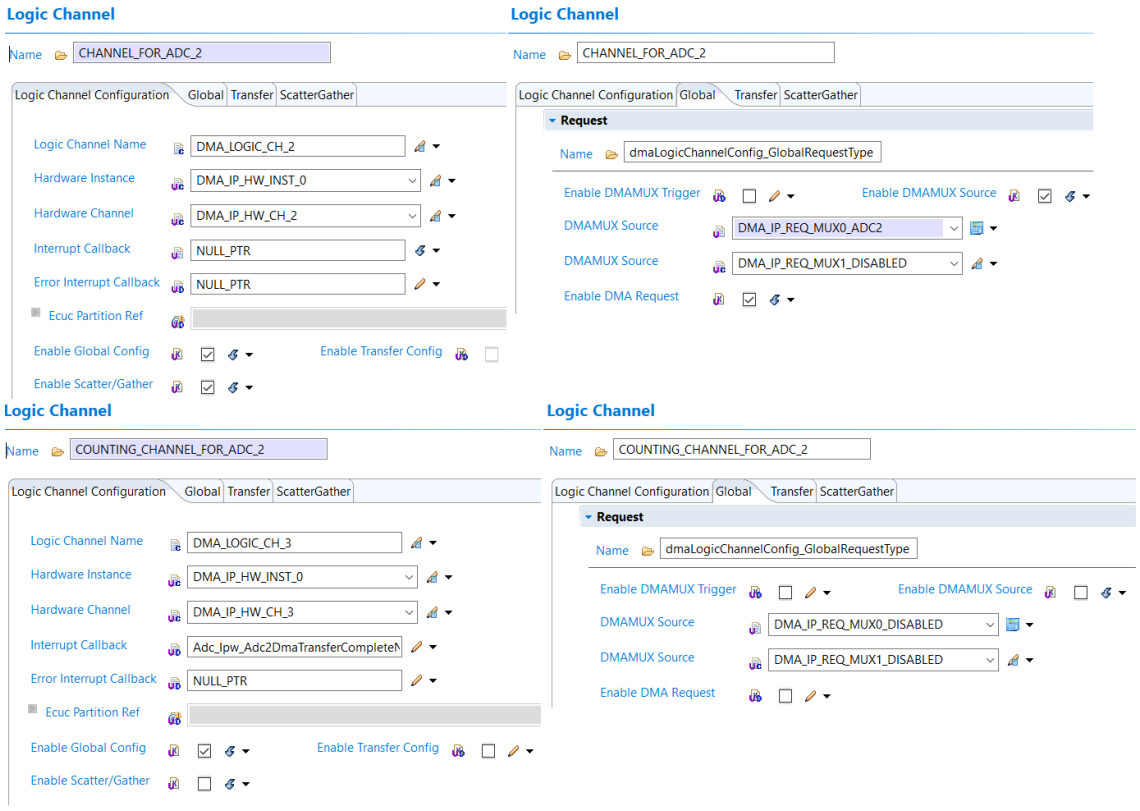
- If this feature is enabled with a group having only a single channel, driver will only use transferring DMA channel.
- The driver implementation is similar to configuring groups without interrupt, streaming access mode with result reordered. (Referring to Without Interrupts group) Configuration-time-prerequisites:
- Enable both Adc Enable DMA support and Adc Optimize DMA Streaming Groups:



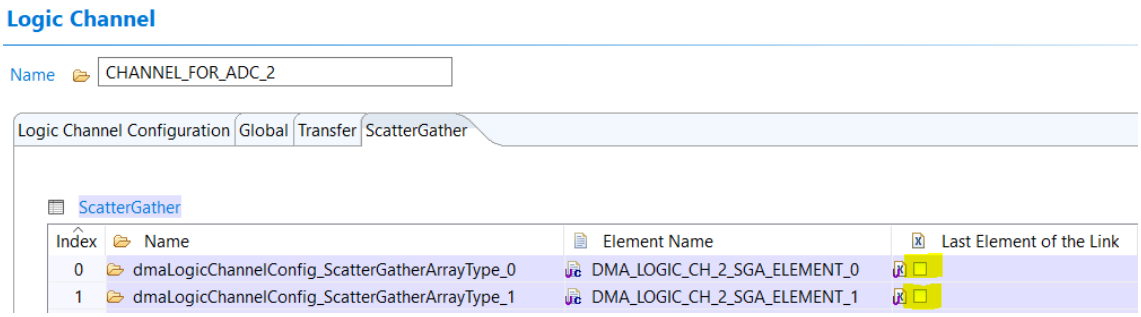
- Configure AdcTransferType as ADC_DMA type and select transferring DMA channel (and also streaming DMA channel if there is at least one ADC group enabled with Optimize DMA streaming groups with more than one ADC channel configured).



- Transferring DMA channel needs to be enabled DMA MUX source (e.g: DMA_IP_REQ_MUX0_ADC2) and streaming DMA channel needs to have Interrupt Callback selected (e.g: Adc_Ipw_Adc2DmaTransfer↩ CompleteNotification()) corresponding to the ADC hardware unit used

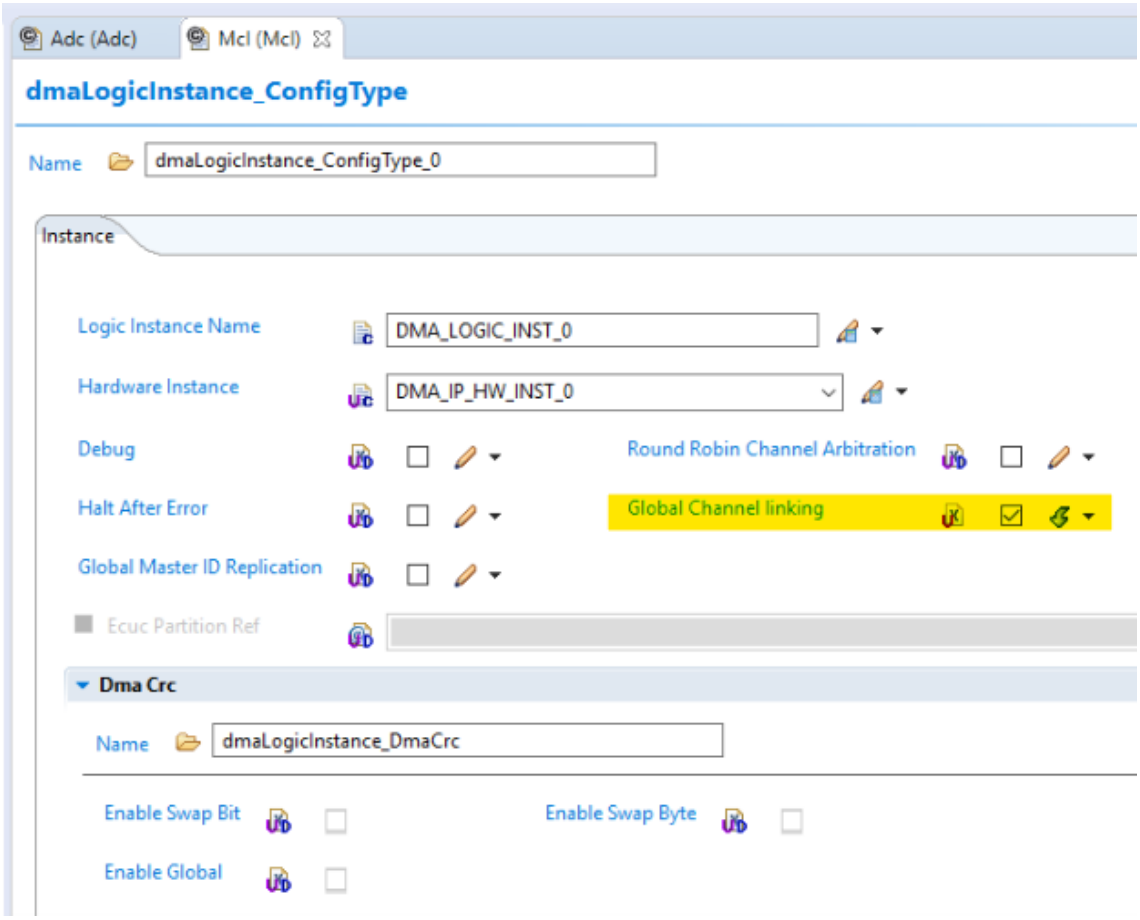


- If a group has multiple channels, transferring DMA channel will be configured in Scatter-Gather mode with the number of elements equal to the number of channels in that group. These elements need to be configured as a ring with no last element configured.

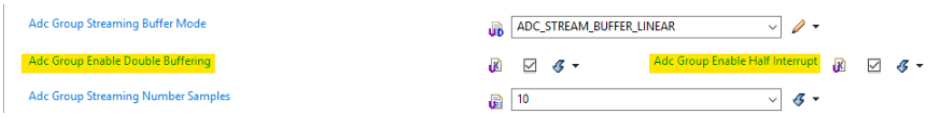


This introduces the limitation that on the same HW unit groups having without interrupt DISABLED cannot be used.

- Global channel linking also need to be enabled for this feature



- At group level, Adc Enable Optimize DMA Streaming Groups (and Adc Group Enable Half Interrupt if half interrupt is used) need to be enabled.



- If a group has configured more than one channel, Adc Group Streaming Results Reorder is also required.



Note

- Adc_SetChannel(), Adc_EnableChannel(), Adc_DisableChannel() and Optimize one-shot hardware trigger cannot be use concurrently with this feature.

3.6.17 TempSense functionality

ADC driver provides an API to interface with the the TempSense module in order to measure the temperature on the chip. The API consists of three functions:

- Adc_TempSenseCalculateTemp() that calculates the temperature based on given data(if read directly using ADC)
- Adc_TempSenseGetTemp() that reads and returns the temperature from the sensor
- Adc_TempSenseSetPowerMode() that turns on or off the module In order to use this functionality, the user must enable AdcSarEnableTempsenseAPI. Additionally, the user can configure TempSense to be powered up or down at startup by setting AdcSarPowerOnTempSense and modify the voltage supply used for TempSense by updating AdcSarTempSenseVsupply.

Configuration-time-prerequisites:

- AdcSarEnableTempsenseAPI enabled in AutosarExt container.



3.7 Runtime errors

The driver generates the following DET runtime errors at runtime.

Function	Error Code	Condition triggering the error
Adc_Init()	ADC_E_TIMEOUT	Driver initialization timed out. ADC hardware could not enter power down or idle state.
Adc_DeInit()	ADC_E_TIMEOUT	Timed out. ADC hardware could not enter power down or idle state.
Adc_StartGroupConversion()	ADC_E_TIMEOUT	Timed out. Ongoing conversion could not be aborted.

Function	Error Code	Condition triggering the error
Adc_StopGroupConversion()	ADC_E_TIMEOUT	Timed out. Ongoing conversion could not be aborted.
Adc_SetClockMode()	ADC_E_TIMEOUT	Timed out. ADC hardware could not enter power down or idle state.
Adc_SetClockMode()	ADC_E_UNINIT	Global pointer is NULL.
Adc_SetClockMode()	ADC_E_INVALID_CLOCK_MO↩ DE	Clock mode is not correctly set.
Adc_SetHwUnitPowerMode()	ADC_E_TIMEOUT	Timed out. Setting power mode timed out.
Adc_Calibrate()	ADC_E_TIMEOUT	Timed out. Calibration operation timed out.
Adc_SelfTest()	ADC_E_TIMEOUT	Timed out. Self test operation timed out.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like:

#define <Mip>Conf_<Container_ShortName>_<Container_ID>

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

Chapter 4

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the driver. All the parameters are described below.

- Module Adc
  - Container AdcConfigSet
    - Container AdcHwUnit
      - Parameter AdcHwUnitId
      - Parameter AdcLogicalUnitId
      - Parameter AdcTransferType
      - Parameter AdcClockSource
      - Parameter AdcPrescale
      - Parameter AdcAltPrescale
      - Parameter AdcCalibrationPrescale
      - Parameter AdcHighSpeedEnable
      - Parameter AdcAltHighSpeedEnable
      - Parameter AdcPowerDownDelay
      - Parameter AdcAltPowerDownDelay
      - Parameter AdcMuxDelay
      - Parameter AdcAutoClockOff
      - Parameter AdcBypassSampling
      - Parameter AdcHwUnitOverwriteEn
      - Parameter AdcPresamplingInternalSignal0
      - Parameter AdcPresamplingInternalSignal1
      - Parameter AdcPresamplingInternalSignal2
      - Parameter AdcHwUnitUsrOffset
      - Parameter AdcHwUnitUsrGain
      - Parameter AdcHwUnitResolution
      - Parameter AdcHwUnitBypassResolution
      - Parameter AdcHwUnitDmaClearSource
      - Parameter AdcSarHwUnitVref
      - Reference AdcDmaChannelId
      - Reference AdcCountingDmaChannelId
      - Reference AdcHwUnitEcucPartitionRef

- Container AdcNormalConvTimings
- Parameter AdcHardwareAverageEnable
- Parameter AdcHardwareAverageSelect
- Parameter AdcSamplingDurationNormal0
- Parameter AdcSamplingDurationNormal1
- Parameter AdcSamplingDurationNormal2
- Container AdcAlternateConvTimings
- Parameter AdcHardwareAverageEnableAlternate
- Parameter AdcHardwareAverageSelectAlternate
- Parameter AdcSamplingDurationAlt0
- Parameter AdcSamplingDurationAlt1
- Parameter AdcSamplingDurationAlt2
- Container AdcChannel
- Parameter AdcLogicalChannelId
- Parameter AdcChannelName
- Parameter AdcChannelId
- Parameter AdcChannelConvTime
- Parameter AdcChannelLimitCheck
- Parameter AdcChannelHighLimit
- Parameter AdcChannelLowLimit
- Parameter AdcChannelRangeSelect
- Parameter AdcChannelRefVoltsrcHigh
- Parameter AdcChannelRefVoltsrcLow
- Parameter AdcChannelResolution
- Parameter AdcChannelSampTime
- Parameter AdcEnablePresampling
- Parameter AdcEnableThresholds
- Parameter AdcWdogNotification
- Reference AdcThresholdRegister
- Container AdcGroup
- Parameter AdcGroupAccessMode
- Parameter AdcGroupConversionMode
- Parameter AdcGroupConversionType
- Parameter AdcGroupId
- Parameter AdcGroupPriority
- Parameter AdcGroupReplacement
- Parameter AdcGroupTriggSrc
- Parameter AdcHwTrigSignal
- Parameter AdcHwTrigTimer
- Parameter AdcNotification
- Parameter AdcExtraNotification
- Parameter AdcStreamingBufferMode
- Parameter AdcEnableOptimizeDmaStreamingGroups
- Parameter AdcEnableHalfInterrupt
- Parameter AdcStreamingNumSamples
- Parameter AdcStreamResultGroup
- Parameter AdcEnableChDisableChGroup

- Parameter AdcWithoutInterrupts
- Parameter AdcWithoutDma
- Parameter AdcExtDMAChanEnable
- Reference AdcGroupHwTriggerSource
- Reference AdcGroupDefinition
- Reference AdcGroupEcucPartitionRef
- Container AdcGroupConversionConfiguration
- Parameter AdcGroupHardwareAverageEnable
- Parameter AdcGroupHardwareAverageSelect
- Parameter AdcSamplingDuration0
- Parameter AdcSamplingDuration1
- Parameter AdcSamplingDuration2
- Container AdcAlternateGroupConvTimings
- Parameter AdcGroupAltHardwareAverageEnable
- Parameter AdcGroupAltHardwareAverageSelect
- Parameter AdcAltGroupSamplingDuration0
- Parameter AdcAltGroupSamplingDuration1
- Parameter AdcAltGroupSamplingDuration2
- Container AdcThresholdControl
- Parameter AdcThresholdControlRegister
- Parameter AdcHighThreshold
- Parameter AdcLowThreshold

* Container AdcHwTrigger
  - Parameter AdcHwTrigSrc
* Container BctuHwUnit
  - Parameter BctuLowPowerMode
  - Parameter BctuGlobalHwTriggers
  - Parameter BctuNewDataDMAEnableMask
  - Parameter BctuFifoDmaRawData
  - Parameter BctuTriggerNotification
  - Container BctuInternalTrigger
  - Parameter BctuTriggerLoop
  - Parameter BctuDataDestination
  - Parameter BctuHwTriggerEnable
  - Parameter BctuTriggerConversionMode
  - Parameter BctuAdcTargetMask
  - Parameter BctuConversionListStartIndex
  - Reference BctuTriggerSource
  - Reference BctuAdcChannelSingle
  - Container BctuAdcNotifications
  - Parameter BctuAdcNewDataNotification
  - Parameter BctuDataOverrunNotification
  - Parameter BctuListLastConversionNotification
  - Reference BctuAdcNotificationsAdcIndex
  - Container BctuListItems
  - Parameter BctuAdcChannelList
  - Parameter BctuNextChannelWaitOnTrig

        - Parameter BctuLastChannel
        - Container BctuResultFifos
        - Parameter BctuResultFifoIndex
        - Parameter BctuWatermarkValue
        - Parameter BctuFifoNotificationsEnable
        - Parameter BctuWatermarkNotification
        - Parameter BctuUnderrunNotification
        - Parameter BctuOverrunNotification
        - Parameter BctuFifoDmaEnable
        - Parameter BctuFifoDmaBuffer
        - Reference BctuFifoDmaChannelId
- Container AdcGeneral
    * Parameter AdcDeInitApi
    * Parameter AdcDevErrorDetect
    * Parameter AdcEnableLimitCheck
    * Parameter AdcEnableQueuing
    * Parameter AdcPriorityQueueMaxDepth
    * Parameter AdcEnableStartStopGroupApi
    * Parameter AdcGrpNotifCapability
    * Parameter AdcHwTriggerApi
    * Parameter AdcPriorityImplementation
    * Parameter AdcReadGroupApi
    * Parameter AdcResultAlignment
    * Parameter AdcVersionInfoApi
    * Parameter AdcLowPowerStatesSupport
    * Parameter AdcPowerStateAsynchTransitionMode
    * Reference AdcEcucPartitionRef
    * Reference AdcKernelEcucPartitionRef
    * Container AdcPowerStateConfig
        - Parameter AdcPowerState
        - Parameter AdcPowerStateReadyCbkRef
- Container AdcInterrupt
    * Parameter AdcInterruptSource
    * Parameter AdcInterruptEnable
- Container AdcPublishedInformation
    * Parameter AdcChannelValueSigned
    * Parameter AdcGroupFirstChannelFixed
    * Parameter AdcMaxChannelResolution
- Container CommonPublishedInformation
    * Parameter ArReleaseMajorVersion
    * Parameter ArReleaseMinorVersion
    * Parameter ArReleaseRevisionVersion

    * Parameter ModuleId
    * Parameter SwMajorVersion
    * Parameter SwMinorVersion
    * Parameter SwPatchVersion
    * Parameter VendorApiInfix
    * Parameter VendorId
  - Container AutosarExt
    * Parameter AdcTimeoutMethod
    * Parameter AdcTimeoutVal
    * Parameter AdcSarIpDevErrorDetect
    * Parameter BctuIpDevErrorDetect
    * Parameter AdcMulticoreSupport
    * Parameter AdcEnableGroupDependentChannelNames
    * Parameter AdcBypassAbortChainCheck
    * Parameter AdcConvTimeOnce
    * Parameter AdcOptimizeOneShotHwTriggerConversions
    * Parameter AdcOptimizeDmaStreamingGroups
    * Parameter AdcPreSamplingOnce
    * Parameter AdcEnableInitialNotification
    * Parameter AdcEnableDmaTransferMode
    * Parameter AdcUseSoftwareInjectedGroups
    * Parameter AdcUseHardwareNormalGroups
    * Parameter AdcEnableUserModeSupport
    * Parameter AdcSetHwUnitPowerModeApi
    * Parameter AdcEnableChDisableChApi
    * Parameter AdcGetInjectedConvStatusApi
    * Parameter AdcEnableThresholdConfigurationApi
    * Parameter AdcEnableCtuTrigAutosarExtApi
    * Parameter AdcCtuHardwareTriggerOptimization
    * Parameter AdcEnableCtuControlModeApi
    * Parameter CtuEnableDmaTransferMode
    * Parameter AdcEnableWatchdogApi
    * Parameter AdcEnableSetChannel
    * Parameter AdcEnableDualClockMode
    * Parameter AdcEnableCalibration
    * Parameter AdcEnableSelfTest
    * Parameter AdcEnableReadRawDataApi
    * Parameter AdcEnableGroupStreamingResultReorder
    * Parameter AdcSarEnableTempsenseApi
    * Parameter AdcSarPowerOnTempSense
    * Parameter AdcSarTempSenseVsupply

4.1 Module Adc

Configuration of the Adc (Analog Digital Conversion) module.

Included containers:

- AdcConfigSet
- AdcGeneral
- AdcInterrupt
- AdcPublishedInformation
- CommonPublishedInformation
- AutosarExt

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.2 Container AdcConfigSet

This container contains the configuration parameters and sub containers of the AUTOSAR Adc module.

Included subcontainers:

- AdcHwUnit
- AdcHwTrigger
- BctuHwUnit

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.3 Container AdcHwUnit

This container contains the Driver configuration (parameters) depending on grouping of channels. This container could contain HW specific parameters which are not defined in the Standardized Module Definition. They must be added in the Vendor Specific Module Definition.

Included subcontainers:

- AdcNormalConvTimings
- AdcAlternateConvTimings
- AdcChannel
- AdcGroup
- AdcThresholdControl

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.4 Parameter AdcHwUnitId

Numeric ID of the HW Unit. This symbolic name allows accessing Hw Unit data. Enumeration literals are defined vendor specific.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC0
literals	['ADC0', 'ADC1', 'ADC2']

4.5 Parameter AdcLogicalUnitId

Specifies the Logical id of the Hardware Unit.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	2
min	0

4.6 Parameter AdcTransferType

Select the Interrupt or Dma transfer Type.

If DMA is required for AdcTransferType, it is recommended to keep AdcWithoutInterrupts as false, otherwise, DMA will not be configured and it will be user responsibility to read the results from registers directly by calling Adc_ReadGroup.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_INTERRUPT
literals	['ADC_INTERRUPT', 'ADC_DMA']

4.7 Parameter AdcClockSource

The ADC module specific clock input for the conversion unit can statically be configured to select different clock sources if provided by hardware. Enumeration literals are defined vendor specific.

This parameter is not used by the current implementation.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	CLK_SRC_0
literals	['CLK_SRC_0']

4.8 Parameter AdcPrescale

Optional ADC module specific clock prescale factor, if supported by hardware.

ImplementationType: Adc_PrescaleType.

The Prescaler value for NORMAL mode. Only the following are allowed:

1: ADC module clock frequency.

2: ADC module clock frequency / 2.

4: ADC module clock frequency / 4.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	4
min	1

4.9 Parameter AdcAltPrescale

The Prescaler value for ALTERNATE mode. This feature can be configured if Adc Set Clock Mode API from General/AutosarExt is enabled.

1: ADC module clock frequency.

2: ADC module clock frequency / 2.

4: ADC module clock frequency / 4.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2
max	4
min	1

4.10 Parameter AdcCalibrationPrescale

Specifies the used clock input for calibration. This feature can be configured if Adc Enable Calibration API from General/AutosarExt is enabled.

1: ADC module clock frequency.

2: ADC module clock frequency / 2.

4: ADC module clock frequency / 4.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2
max	4
min	1

4.11 Parameter AdcHighSpeedEnable

Enables/disables high speed conversion or calibration.

The frequency of the conversion clock has to be within the limits defined in the data sheet.

If the module clock frequency is higher than the maximum frequency of the conversion clock allowed during the functional conversion or during the calibration (see the chip data sheet),

then you must configure the ADC conversion clock divider so that the frequency of the conversion clock is within allowed limits.

For some conversion clock frequencies this feature must be enabled. Refer to Reference Manual or data sheet for more details.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.12 Parameter AdcAltHighSpeedEnable

Alt configuration: Enables/disables high speed conversion or calibration.

The frequency of the conversion clock has to be within the limits defined in the data sheet.

If the module clock frequency is higher than the maximum frequency of the conversion clock allowed during the functional conversion or during the calibration (see the chip data sheet),

then you must configure the ADC conversion clock divider so that the frequency of the conversion clock is within allowed limits.

For some conversion clock frequencies this feature must be enabled. Refer to Reference Manual or data sheet for more details.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.13 Parameter AdcPowerDownDelay

The delay between the power down bit reset and the starting of conversion.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	15
max	255
min	0

4.14 Parameter AdcAltPowerDownDelay

The delay between the power down bit reset and the starting of conversion when ADC runs on low power system frequency.

This node is EDITABLE only if "Adc Set Clock Mode API" is enabled

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	15
max	255
min	0

4.15 Parameter AdcMuxDelay

The delay between the external decode signals and the start of the sampling phase.

It is used to take into account the settling time of the external mux when ADC runs on normal system frequency.

The decode signal delay is calculated as (DSD x 1/Frequency of Adc_Clock).

The DSDR register is 12-bit.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	15
max	65535
min	0

4.16 Parameter AdcAutoClockOff

Enables/disables the auto-clock-off features.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.17 Parameter AdcBypassSampling

Select whether the presampling is followed by the comparison.

Disabled: The presampling will be followed by sampling the selected input.

Enabled: The presampling will be followed by the successive approximation algorithm and the conversion data will be written to the conversion data register of the selected input.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.18 Parameter AdcHwUnitOverwriteEn

Specifies whether a conversion data register accepts new data before the current conversion data has been read.

When enabled, the new conversion result overwrites the older data, regardless of the validity of the older conversion data. When CDRn[VALID] = 1, the conversion data has not been read.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.19 Parameter AdcPresamplingInternalSignal0

Select the Adc presampling internal voltage for channels 0-31, AdcPresamplingInternalSignal0 shall take the following values:

VREFL - Pre-sample voltage - 0.

VREFH - Pre-sample voltage - 1.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	VREFL
literals	['VREFL', 'VREFH']

4.20 Parameter AdcPresamplingInternalSignal1

Select the Adc presampling internal voltage for channels 32-63, AdcPresamplingInternalSignal0 shall take the

following values:

VREFL - Pre-sample voltage - 0.

VREFH - Pre-sample voltage - 1.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	VREFL
literals	['VREFL', 'VREFH']

4.21 Parameter AdcPresamplingInternalSignal2

Select the Adc presampling internal voltage for the external channels on current hardware unit:

AdcPresamplingInternalSignal2 shall take the following values:

VREFL - Pre-sample voltage - 0.

VREFH - Pre-sample voltage - 1.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	VREFL
literals	['VREFL', 'VREFH']

4.22 Parameter AdcHwUnitUsrOffset

The ADC can compensate a maximum of +/-15LSB at 12b offset by programming OFFSET_USER field in USROFSGN register.

The field is in 2's complement format and stored with an 8x multiplication. Thus, if a value of +/-X is stored in this field, it leads to

a final result adjusted with +/-(X/8) from its unprogrammed version. It is required to keep the 3-LSBs zero to avoid any fractional

subtraction-related errors.

ADC_OUTPUT_NEW = ADC_OUTPUT_OLD - (OFFSET_USER/8).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.23 Parameter AdcHwUnitUsrGain

The ADC can compensate a maximum of +/-63 LSB at 12b gain by programming GAIN_USER field in USROFSGN register.

The field is in 2's complement format and stored with an 8x multiplication. Thus, if a value of +/- Y is stored in this field, it leads to a

final result adjusted with +/-(Y/8) LSB from its unprogrammed version. It is required to keep the 3-LSBs zero to avoid any fractional

subtraction/multiplication related errors.

ADC_OUTPUT_NEW = ADC_OUTPUT_OLD * (1 + (GAIN_USER / (8 * 4096))).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	65535
min	0

4.24 Parameter AdcHwUnitResolution

Select the number of significant bits per conversion data.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	RESOLUTION_14
literals	['RESOLUTION_14', 'RESOLUTION_12', 'RESOLUTION_10', 'RESOLUT↩ ION_8']

4.25 Parameter AdcHwUnitBypassResolution

Disable resolution processing for results, allowing the API to return the raw result data from CDR register.

Enabling this will allow results read by the API to be on the same bit width as the ones read using DMA, since DMA will always get the data from the CDR register unprocessed.

The resolution of the result will be the same as the one configured above but its number of bits will always be 15.

The threshold values will also need to be given on 15 bits.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.26 Parameter AdcHwUnitDmaClearSource

Select the DMA request clearing source.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	DMA_REQ_CLEAR_ON_ACK
literals	['DMA_REQ_CLEAR_ON_ACK', 'DMA_REQ_CLEAR_ON_READ']

4.27 Parameter AdcSarHwUnitVref

Adc Voltage Reference, expressed in fixed point format with 1 bit for the sign, 11 bits for the integer part and 4 bits for the decimal part.

The value must be filled in by the user according to actual board setup. It and will be used for calculating the value measured by TempSense.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	53
max	88
min	0

4.28 Reference AdcDmaChannelId

ID of the DMA channel used to transfer the data.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Mcl/MclConfig/dmaLogicChannel_Type

4.29 Reference AdcCountingDmaChannelId

Configurable only when Transfer Type is DMA and at least one group of hw unit has:

AdcEnableOptimizeDmaStreamingGroups enabled with more than one ADC channel

OR has selected Without Interrupts, ACCESS_MODE_STREAMING and Group Streaming Results Reorder and number of channels > 1

OR has selected Without Interrupts, ACCESS_MODE_STREAMING and a single channel - for this case, AutosarExt Adc Enable Group Streaming Results Reorder feature must be enabled, but can be disabled at group level.

Linked to DMA channel by ADC driver, to count the number of samples converted in streaming mode

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Mcl/MclConfig/dmaLogicChannel_Type

4.30 Reference AdcHwUnitEcucPartitionRef

Maps a ADC hardware unit to zero or one ECUC partition to limit the access to this hardware unit. The ECUC partitions referenced are a subset of the ECUC partitions where the ADC driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.31 Container AdcNormalConvTimings

Selects Normal values used for programming CTR Conversion Timing Registers in Adc_SetClockMode API and also when AdcConvTimeOnce option is enabled.

This node is EDITABLE only if "Adc Conversion Time Once" is enabled

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.32 Parameter AdcHardwareAverageEnable

Enables the hardware average function of the ADC.

This feature is enabled if "Adc Conversion Time Once" from General/AutosarExt is enabled

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.33 Parameter AdcHardwareAverageSelect

Determines how many ADC conversions will be averaged to create the ADC average result. This functionality is activated when ADCx_MCR[AVGEN] = 1.

SAMPLES_4 - 4 samples averaged.

SAMPLES_8 - 8 samples averaged.

SAMPLES_16 - 16 samples averaged.

SAMPLES_32 - 32 samples averaged.

.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	SAMPLES_4
literals	['SAMPLES_4', 'SAMPLES_8', 'SAMPLES_16', 'SAMPLES_32']

4.34 Parameter AdcSamplingDurationNormal0

Select the Normal Sampling Duration for channels 0-31 when calling Adc_SetClockMode(ADC_NORMAL) for CTR0 register.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	22
max	255
min	8

4.35 Parameter AdcSamplingDurationNormal1

Select the Normal Sampling Duration for channels 32-63 when calling Adc_SetClockMode(ADC_NORMAL) for CTR1 register.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	22
max	255
min	8

4.36 Parameter AdcSamplingDurationNormal2

Select the Normal Sampling Duration for channels 64-95 when calling Adc_SetClockMode(ADC_NORMAL) for CTR2 register.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	22
max	255
min	8

4.37 Container AdcAlternateConvTimings

Selects Alternate values used in Adc_SetClockMode API for programming CTR Conversion Timing Registers.

This container is EDITABLE only if "Adc Conversion Time Once" and "Adc Set Clock Mode API" are enabled. Hardware averaging functionality is also available for configuration if supported.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.38 Parameter AdcHardwareAverageEnableAlternate

Alternate configuration: Enables the hardware average function of the ADC.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.39 Parameter AdcHardwareAverageSelectAlternate

Alternate configuration: Determines how many ADC conversions will be averaged to create the ADC average result. This functionality is activated when ADCx_MCR[AVGEN] = 1.

SAMPLES_4 - 4 samples averaged.

SAMPLES_8 - 8 samples averaged.

SAMPLES_16 - 16 samples averaged.

SAMPLES_32 - 32 samples averaged.

.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	SAMPLES_4
literals	['SAMPLES_4', 'SAMPLES_8', 'SAMPLES_16', 'SAMPLES_32']

4.40 Parameter AdcSamplingDurationAlt0

Alternate configuration: Select the Normal Sampling Duration for channels 0-31 when calling Adc_SetClockMode(ADC_NORM
for CTR0 register.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	22
max	255
min	8

4.41 Parameter AdcSamplingDurationAlt1

Alternate configuration: Select the Normal Sampling Duration for channels 32-63 when calling Adc_SetClockMode(ADC_NOR
for CTR1 register.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	22
max	255
min	8

4.42 Parameter AdcSamplingDurationAlt2

Alternate configuration: Select the Normal Sampling Duration for channels 64-95 when calling Adc_SetClockMode(ADC_NOR
for CTR2 register.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	22
max	255
min	8

4.43 Container AdcChannel

This container contains the channel configuration (parameters) depending on the hardware capability.

The organization of this data structure could contain dependencies to the microcontroller so this is left up to the implementer and its location is left up to the configuration.

Note: Since a AdcChannel can be part of several AdcGroups, this container is not realized as a subcontainer of AdcGroup but instead as a subcontainer of AdcHwUnit.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.44 Parameter AdcLogicalChannelId

This is the logical Id of the ADC channel.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	1024
min	0

4.45 Parameter AdcChannelName

This parameter defines the assignment of the channel to the physical ADC hardware channel. Note: - Range of the ADC Channels depends on the selected package.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	P0_ChanNum0
literals	['P0_ChanNum0', 'P1_ChanNum1', 'P2_ChanNum2', 'P3_ChanNum3', 'P4_ChanNum4', 'P5_ChanNum5', 'P6_ChanNum6', 'P7_ChanNum7', 'S8_ChanNum32', 'S9_ChanNum33', 'S10_ChanNum34', 'S11_ChanNum35', 'S12_ChanNum36', 'S13_ChanNum37', 'S14_ChanNum38', 'S15_ChanNum39', 'S16_ChanNum40', 'S17_ChanNum41', 'S18_ChanNum42', 'S19_ChanNum43', 'S20_ChanNum44', 'S21_ChanNum45', 'S22_ChanNum46', 'S23_ChanNum47', 'BANDGAP_ChanNum48', 'TEMPSENSOR_OUTPUT_ChanNum49', 'ANAMUX_OUTPUT_ChanNum50', 'VREFL_ChanNum54', 'VREFH_ChanNum55', 'X0_ChanNum64', 'X0_ChanNum65', 'X0_ChanNum66', 'X0_ChanNum67', 'X0_ChanNum68', 'X0_ChanNum69', 'X0_ChanNum70', 'X0_ChanNum71', 'X1_ChanNum72', 'X1_ChanNum73', 'X1_ChanNum74', 'X1_ChanNum75', 'X1_ChanNum76', 'X1_ChanNum77', 'X1_ChanNum78', 'X1_ChanNum79', 'X2_ChanNum80', 'X2_ChanNum81', 'X2_ChanNum82', 'X2_ChanNum83', 'X2_ChanNum84', 'X2_ChanNum85', 'X2_ChanNum86', 'X2_ChanNum87', 'X3_ChanNum88', 'X3_ChanNum89', 'X3_ChanNum90', 'X3_ChanNum91', 'X3_ChanNum92', 'X3_ChanNum93', 'X3_ChanNum94', 'X3_ChanNum95']

4.46 Parameter AdcChannelId

This parameter defines the assignment of the channel to the physical ADC hardware channel. Note: - Range of the ADC Channels depends on the selected package.

IMPORTANT NOTE: This node must be in sync with 'Adc Physical Channel Name': must be equal with number specified after 'ChanNum' in the selected 'Adc Physical Channel Name'. E.g. ADC_CH_0_ChanN => Channel ID must be 38. The node is required by Autosar standard. 'Adc Physical Channel Name' node is added because refers to names from ReferenceManual and is more user friendly.

In Tresos configurator, after selecting a new Channel Name, the new Channel Id value can be filled in automatically by using 'Calculate Value' button.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	1024
min	0

4.47 Parameter AdcChannelConvTime

Configuration of conversion time, i.e. the time during which the analogue value is converted into digital representation, (in clock cycles) for each channel, if supported by hardware.

ImplementationType: Adc_ConversionTimeType.

This parameter is not used by the current implementation.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	9223372036854775807
min	0

4.48 Parameter AdcChannelLimitCheck

Enables or disables limit checking for an ADC channel.

This node is EDITABLE only if "Adc Enable Limit Check" is enabled

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.49 Parameter AdcChannelHighLimit

High limit - used for limit checking. This value depends on the Adc Hw Unit resolution. If AdcHwUnitBypassResolution is enabled, then this value will always be on 15 bits.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	16383
max	9223372036854775807
min	0

4.50 Parameter AdcChannelLowLimit

Low limit - used for limit checking.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	9223372036854775807
min	0

4.51 Parameter AdcChannelRangeSelect

In case of active limit checking: defines which conversion values are taken into account related to the boarders defined with AdcChannelLowLimit and AdcChannelHighLimit.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_RANGE_ALWAYS
literals	['ADC_RANGE_ALWAYS', 'ADC_RANGE_BETWEEN', 'ADC_RANGE↩ _NOT_BETWEEN', 'ADC_RANGE_NOT_OVER_HIGH', 'ADC_RANG↩ E_NOT_UNDER_LOW', 'ADC_RANGE_OVER_HIGH', 'ADC_RANGE↩ _UNDER_LOW']

4.52 Parameter AdcChannelRefVoltsrcHigh

Upper reference voltage source for each channel. Enumeration literals are defined vendor specific.

This parameter is not used by the current implementation.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	UPPER_REF_VOLT_0
literals	['UPPER_REF_VOLT_0']

4.53 Parameter AdcChannelRefVoltsrcLow

Lower reference voltage source for each channel. Enumeration literals are defined vendor specific.

This parameter is not used by the current implementation.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	LOWER_REF_VOLT_0
literals	['LOWER_REF_VOLT_0']

4.54 Parameter AdcChannelResolution

Channel Resolution in bits of converted value. This value is not used. The resolution can be modified at hardware unit level.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	14
max	63
min	1

4.55 Parameter AdcChannelSampTime

Sampling time, i.e. the time during which the value is sampled, (in clock cycles) for each channel. This parameter is not used by the current implementation.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	22
max	9223372036854775807
min	0

4.56 Parameter AdcEnablePresampling

When true, this parameter enables the presampling phase for the selected channel.

The normal operation sequence on the channel: Presampling -> Sampling -> Conversion.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.57 Parameter AdcEnableThresholds

When true, this parameter enables the threshold detection feature for the selected channel.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.58 Parameter AdcWdogNotification

This function pointer is called whenever the conversion of the channel issued a watchdog interrupt.

The notification takes one uint8 parameter which represents the flags(low/high threshold) that triggered the interrupt.

In order to interpret them please use ADC_WDG_... defines.

Property	Value
type	ECUC-FUNCTION-NAME-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.59 Reference AdcThresholdRegister

Select the threshold register which provides the values to be used for upper and lower thresholds.

ADCHwUnits support threshold registers from ADC_THRESHOLD_REG_0 to ADC_THRESHOLD_REG_3.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D34M20I0R0/Adc/AdcConfigSet/AdcHwUnit/AdcThresholdControl

4.60 Container AdcGroup

This container contains the Group configuration (parameters).

Included subcontainers:

- AdcGroupConversionConfiguration
- AdcAlternateGroupConvTimings

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.61 Parameter AdcGroupAccessMode

Type of access mode to group conversion results.

ImplementationType: Adc_GroupAccessModeType.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_ACCESS_MODE_SINGLE
literals	['ADC_ACCESS_MODE_SINGLE', 'ADC_ACCESS_MODE_STREAMING']

4.62 Parameter AdcGroupConversionMode

Type of conversion mode supported by the driver.

ImplementationType: Adc_GroupConvModeType.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_CONV_MODE_ONESHOT
literals	['ADC_CONV_MODE_ONESHOT', 'ADC_CONV_MODE_CONTINUOUS']

4.63 Parameter AdcGroupConversionType

Normal or Injected conversion type.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_CONV_TYPE_NORMAL
literals	['ADC_CONV_TYPE_NORMAL', 'ADC_CONV_TYPE_INJECTED']

4.64 Parameter AdcGroupId

Numeric ID of the group. This parameter is the symbolic name to be used on the API. This symbolic name allows accessing Channel Group data. This value will be assigned to the symbolic name derived of the AdcGroup container shortName.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	1023
min	0

4.65 Parameter AdcGroupPriority

Priority level of the AdcGroup. This item is ignored if Adc/AdcGeneral/AdcPriorityImplementation is defined to ADC_PRIORITY_NONE.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.66 Parameter AdcGroupReplacement

Replacement mechanism, which is used on ADC group level, if a group conversion is interrupted by a group which has a higher priority.

ImplementationType: Adc_GroupReplacementType

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_GROUP_REPL_ABORT_RESTART
literals	['ADC_GROUP_REPL_ABORT_RESTART', 'ADC_GROUP_REPL_SU↩ SPEND_RESUME']

4.67 Parameter AdcGroupTriggSrc

Type of source event that starts a group conversion. It's possible select Hw or Sw trigger.

In case of Hw trigger the trigger source can be from the CTU or External hardware pins of the controller.

In this controller only Bctu trigger source is supported which is selected by the "Adc Group Hardware Trigger Source" parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_TRIGG_SRC_SW
literals	['ADC_TRIGG_SRC_HW', 'ADC_TRIGG_SRC_SW']

4.68 Parameter AdcHwTrigSignal

Configures the edge of the hardware trigger signal, i.e. to start the conversion.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF

Property	Value
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_HW_TRIG_RISING_EDGE
literals	['ADC_HW_TRIG_BOTH_EDGES', 'ADC_HW_TRIG_RISING_EDGE', 'ADC_HW_TRIG_FALLING_EDGE']

4.69 Parameter AdcHwTrigTimer

Reload value of the ADC module embedded timer. This parameter is not used by the current implementation.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	9223372036854775807
min	0

4.70 Parameter AdcNotification

Callback function for each group. This function pointer is called whenever the conversion of this group is completed.

This node is EDITABLE only if "Adc Notification Capability" is enabled

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.71 Parameter AdcExtraNotification

Extra callback function for each group. This function pointer will be called at the beginning of the interrupt routine, before updating any HW registers or Group status.

This feature is enabled if "Adc Notification Capability" from General/Adc General Configuration is enabled.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.72 Parameter AdcStreamingBufferMode

Select the streaming buffer as linear buffer (i.e. the ADC Driver stops the conversion as soon as the stream buffer is full) or as circular buffer (wraps around if the end of the stream buffer is reached).

This feature is enabled only if "Adc Group Access Mode" is set to ADC_ACCESS_MODE_STREAMI

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_STREAM_BUFFER_LINEAR
literals	['ADC_STREAM_BUFFER_LINEAR', 'ADC_STREAM_BUFFER_CIRC↩ ULAR']

4.73 Parameter AdcEnableOptimizeDmaStreamingGroups

Enable/Disable The Adc driver enable Optimize DMA streaming groups for reducing the number of interrupts required for processing the conversions of Adc Groups that consist of one or more channels (depending on HW capabilities) and which are configured as ADC_ACCESS_MODE_STREAMING.

When this feature is enabled, only one interrupt will be raised after the completion of all stream conversions (as configured by AdcStreamingNumSamples parameter). An additional interrupt to be raised after half of the stream is converted shall also be configurable.

This feature is enabled if "Adc Global Enable DMA Transfer" from General/AutosarExt is also enabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.74 Parameter AdcEnableHalfInterrupt

Enable/ Disable the interrupt when half sample complete for optimize DMA streaming groups feature.

The "Adc Optimize DMA Streaming Groups" must be enabled for configuring this feature.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.75 Parameter AdcStreamingNumSamples

Number of ADC values to be acquired per channel in streaming access mode.

Note: in single access mode this parameter assumes value 1, since only one sample per channel is processed.

ImplementationType: Adc_StreamNumSampleType.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	255
min	1

4.76 Parameter AdcStreamResultGroup

Arrange the ADC results as multiple sets of group result buffer.

E.g: for a group with channels {CH1 CH5 CH7} the resulting stream buffer shall be:

{ CH1, CH5, CH7, CH1, CH5, CH7, CH1, CH5, CH7}

instead of

{ CH1, CH1, CH1, CH5, CH5, CH5, CH7, CH7, CH7} like supported by AUTOSAR standard.

This Parameter can be configured only for groups configured with ADC_ACCESS_MODE_STREAMING Access Mode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.77 Parameter AdcEnableChDisableChGroup

If this parameter is enabled, it allows the feature of enabling or disabling a particular channel in the group.

Max.no of Groups with this feature enabled, should be configured are 254 if the configuration parameter AdcEnableChDisableC is enabled in Autosar Extension container.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.78 Parameter AdcWithoutInterrupts

Enable/ Disable the occurring of ADC Interrupts and Reading of the group conversion results periodically without interrupts (ADC interrupt or DMA notification).

1. When this parameter is enabled, interrupts are disabled. The conversion will run without software intervention (no interrupt generated anymore) and the application can read the results by calling Adc_ReadGroup().

2. If Transfer Type is ADC_INTERRUPT (or ADC_DMA) and this one is enabled, the result buffer registered via Adc_SetupResultBuffer will no longer be used populated with results. Adc_ReadGroup will read the results directly from ADC HW registers (or Internal Dma Buffer).

When this parameter is Disabled, normal functionality shall be executed.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.79 Parameter AdcWithoutDma

When true, disables completely DMA configuration done by ADC driver for the group, to not affect other groups for which the ADC driver has already configured the DMA. It is intended to be used when DMA Transfer Mode is selected for the ADC unit, for groups required to work without interrupt and without DMA.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.80 Parameter AdcExtDMAChanEnable

Enable/ Disable DMA functionality of individual group.

In this mode the adc will only enable the dma request for the last channel in the group and, so that the dma transfer will start after the group conversion has finished.

In this mode it is the users responsibility to configure the dma to transfer the data according to his needs.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.81 Reference AdcGroupHwTriggerSource

Select the HW trigger signal for triggering the conversion group. Configurable only when HW trigger source is selected for the group.

Property	Value
type	ECUC-REFERENCE-DEF

Property	Value
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D34M20I0R0/Adc/AdcConfigSet/AdcHwTrigger

4.82 Reference AdcGroupDefinition

Assignment of channels to a AdcGroups. For each AdcChannel that should belong to the group, a reference needs to be defined.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Adc/AdcConfigSet/AdcHwUnit/AdcChannel

4.83 Reference AdcGroupEcucPartitionRef

Maps an ADC channel group to zero or multiple ECUC partitions to limit the access to this channel group. The ECUC partitions referenced are a subset of the ECUC partitions where the ADC driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite

Property	Value
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.84 Container AdcGroupConversionConfiguration

Configure the Sampling and Conversion TimeGroup.

Hardware averaging functionality is also available for configuration.

Note: This is an Implementation Specific Parameter.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.85 Parameter AdcGroupHardwareAverageEnable

Enables the hardware average function of the ADC.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.86 Parameter AdcGroupHardwareAverageSelect

Determines how many ADC conversions will be averaged to create the ADC average result. This functionality is activated when ADCx_MCR[AVGEN] = 1.

SAMPLES_4 - 4 samples averaged.

SAMPLES_8 - 8 samples averaged.

SAMPLES_16 - 16 samples averaged.

SAMPLES_32 - 32 samples averaged.

.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	SAMPLES_4
literals	['SAMPLES_4', 'SAMPLES_8', 'SAMPLES_16', 'SAMPLES_32']

4.87 Parameter AdcSamplingDuration0

Select the Sampling Duration for channels 0-31 from CTR0 register

Property	Value
type	ECUC-INTEGER-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	22
max	255
min	8

4.88 Parameter AdcSamplingDuration1

Select the Sampling Duration for channels 32-63 from CTR1 register

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	22
max	255
min	8

4.89 Parameter AdcSamplingDuration2

Select the Sampling Duration for channels 64-95 from CTR2 register

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	22
max	255
min	8

4.90 Container AdcAlternateGroupConvTimings

Selects Alternate values used in Adc_SetClockMode API for programming CTR Conversion Timing Registers.

This container is EDITABLE only if "Adc Conversion Time Once" is disabled and "Adc Set Clock Mode API" is enabled. Hardware averaging functionality is also available for configuration if supported.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.91 Parameter AdcGroupAltHardwareAverageEnable

Alternate configuration: Enables the hardware average function of the ADC.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.92 Parameter AdcGroupAltHardwareAverageSelect

Selects the Group Alternate Hardware Average to determine how many ADC conversions will be averaged to create the ADC average result. This functionality is activated when ADCx_MCR[AVGEN] = 1.

SAMPLES_4 - 4 samples averaged.

SAMPLES_8 - 8 samples averaged.

SAMPLES_16 - 16 samples averaged.

SAMPLES_32 - 32 samples averaged.

.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	SAMPLES_4
literals	['SAMPLES_4', 'SAMPLES_8', 'SAMPLES_16', 'SAMPLES_32']

4.93 Parameter AdcAltGroupSamplingDuration0

Select the Alternate Sampling Duration for channels 0-31 from CTR0 register

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	22
max	255
min	8

4.94 Parameter AdcAltGroupSamplingDuration1

Select the Alternate Sampling Duration for channels 32-63 from CTR1 register

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	22
max	255
min	8

4.95 Parameter AdcAltGroupSamplingDuration2

Select the Alternate Sampling Duration for channels 64-95 from CTR2 register

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	22
max	255
min	8

4.96 Container AdcThresholdControl

Configure threshold detection feature for the selected channel.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	4
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.97 Parameter AdcThresholdControlRegister

Select the threshold register which provides the values to be used for upper and lower thresholds.

ADCHwUnits support threshold registers from ADC_THRESHOLD_REG_0 to ADC_THRESHOLD_REG_3.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_THRESHOLD_REG_0
literals	['ADC_THRESHOLD_REG_0', 'ADC_THRESHOLD_REG_1', 'ADC_TH↩ RESHOLD_REG_2', 'ADC_THRESHOLD_REG_3']

4.98 Parameter AdcHighThreshold

Set the value for High Threshold. This value depends on the Adc Hw Unit resolution. If AdcHwUnitBypassResolution is enabled, then this value will always be on 15 bits.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	16383
max	16383
min	0

4.99 Parameter AdcLowThreshold

Set the value for Low Threshold.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	16383
min	0

4.100 Container AdcHwTrigger

This container contains the Hardware trigger source configured for the group. Editable only if at least one group has HW trigger source selected.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.101 Parameter AdcHwTrigSrc

On this implementation the HW triggers available are from Bctu.

Note: This is an implementation specific parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	EXT_TRIG
literals	['BCTU_EMIOS_0_0', 'BCTU_EMIOS_0_1', 'BCTU_EMIOS_0_2', 'BCTU_EMIOS_0_3', 'BCTU_EMIOS_0_4', 'BCTU_EMIOS_0_5', 'BCTU_EMIOS_0_6', 'BCTU_EMIOS_0_7', 'BCTU_EMIOS_0_8', 'BCTU_EMIOS_0_9', 'BCTU_EMIOS_0_10', 'BCTU_EMIOS_0_11', 'BCTU_EMIOS_0_12', 'BCTU_EMIOS_0_13', 'BCTU_EMIOS_0_14', 'BCTU_EMIOS_0_15', 'BCTU_EMIOS_0_16', 'BCTU_EMIOS_0_17', 'BCTU_EMIOS_0_18', 'BCTU_EMIOS_0_19', 'BCTU_EMIOS_0_20', 'BCTU_EMIOS_0_21', 'BCTU_EMIOS_0_22', 'BCTU_Trg23', 'BCTU_EMIOS_1_0', 'BCTU_EMIOS_1_1', 'BCTU_EMIOS_1_2', 'BCTU_EMIOS_1_3', 'BCTU_EMIOS_1_4', 'BCTU_EMIOS_1_5', 'BCTU_EMIOS_1_6', 'BCTU_EMIOS_1_7', 'BCTU_EMIOS_1_8', 'BCTU_EMIOS_1_9', 'BCTU_EMIOS_1_10', 'BCTU_EMIOS_1_11', 'BCTU_EMIOS_1_12', 'BCTU_EMIOS_1_13', 'BCTU_EMIOS_1_14', 'BCTU_EMIOS_1_15', 'BCTU_EMIOS_1_16', 'BCTU_EMIOS_1_17', 'BCTU_EMIOS_1_18', 'BCTU_EMIOS_1_19', 'BCTU_EMIOS_1_20', 'BCTU_EMIOS_1_21', 'BCTU_EMIOS_1_22', 'BCTU_Trg47', 'BCTU_EMIOS_2_0', 'BCTU_EMIOS_2_1', 'BCTU_EMIOS_2_2', 'BCTU_EMIOS_2_3', 'BCTU_EMIOS_2_4', 'BCTU_EMIOS_2_5', 'BCTU_EMIOS_2_6', 'BCTU_EMIOS_2_7', 'BCTU_EMIOS_2_8', 'BCTU_EMIOS_2_9', 'BCTU_EMIOS_2_10', 'BCTU_EMIOS_2_11', 'BCTU_EMIOS_2_12', 'BCTU_EMIOS_2_13', 'BCTU_EMIOS_2_14', 'BCTU_EMIOS_2_15', 'BCTU_EMIOS_2_16', 'BCTU_EMIOS_2_17', 'BCTU_EMIOS_2_18', 'BCTU_EMIOS_2_19', 'BCTU_EMIOS_2_20', 'BCTU_EMIOS_2_21', 'BCTU_EMIOS_2_22', 'BCTU_Trg71', 'EXT_TRIG', 'AUX_EXT_TRIG', 'EXT_AND_AUX_EXT_TRIG']

4.102 Container BctuHwUnit

This container contains configuration of the Bctu unit. This node is EDITABLE only if "Adc CTU Control Mode API" is enabled.

Included subcontainers:

- BctuInternalTrigger
- BctuAdcNotifications
- BctuListItems
- BctuResultFifos

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.103 Parameter BctuLowPowerMode

Enable Bctu low power mode. If enabled, during runtime it must be disabled before allowing triggers to start ADC conversions.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.104 Parameter BctuGlobalHwTriggers

Enable hardware triggers to start ADC conversions. If disabled only BCTU software triggers can start conversions.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.105 Parameter BctuNewDataDMAEnableMask

Bitmask for DMA transfer enable per ADC instance: (1 shifted left n) corresponds to n-th ADC instance connected to the BCTU. The user is responsable for setting up DMA transfer buffer and configuration.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	7
min	0

4.106 Parameter BctuFifoDmaRawData

Enable Dma Transfer Raw Data of Fifo Result Register (includes Trigger Source, ADC Channel, ADC Number and ADC Data Conversion).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.107 Parameter BctuTriggerNotification

This function pointer is called whenever a BCTU trigger is asserted. NULL value disables the notification.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.108 Container BctuInternalTrigger

This container contains the BCTU input trigger configuration parameters

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.109 Parameter BctuTriggerLoop

Enable/Disable loop trigger conversion. This functionality is disabled by default.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.110 Parameter BctuDataDestination

Select the destination of the conversion data.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	BCTU_ADC_DATA_REG
literals	['BCTU_ADC_DATA_REG', 'BCTU_FIFO1', 'BCTU_FIFO2']

4.111 Parameter BctuHwTriggerEnable

Enable/disable initiation of ADC conversions by hw input trigger: true - enabled; false - disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.112 Parameter BctuTriggerConversionMode

Select list mode or single mode for conversion. The multiple ADC selection is only valid for multiple parallel conversions LIST functionality.

If more than one ADC is selected for a single conversion, instead of for a LIST, no conversion will be triggered by this register.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	SINGLE
literals	['SINGLE', 'LIST']

4.113 Parameter BctuAdcTargetMask

Bitmask to select target ADC instance(s): (1 shifted left n) corresponds to n-th ADC instance.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	7
min	0

4.114 Parameter BctuConversionListStartIndex

Start index in the BCTU list, to be enabled when the trigger is asserted. Only enabled if Trigger mode is LIST.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	31
min	0

4.115 Reference BctuTriggerSource

On this implementation the HW triggers available are from BCTU.

(Note: This is an Implementation Specific Parameter.)

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D34M20I0R0/Adc/AdcConfigSet/AdcHwTrigger

4.116 Reference BctuAdcChannelSingle

ADC channel to be converted when the trigger is asserted in SINGLE Trigger Conversion Mode

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Adc/AdcConfigSet/AdcHwUnit/AdcChannel

4.117 Container BctuAdcNotifications

This container allows configuration of the BCTU notifications corresponding to each ADC unit.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.118 Parameter BctuAdcNewDataNotification

This function is called whenever a BCTU new data event is asserted, for the selected ADC.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.119 Parameter BctuDataOverrunNotification

This function is called whenever a BCTU data overrun event is asserted, for the selected ADC.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.120 Parameter BctuListLastConversionNotification

This function pointer is called when the last conversion in the list is started, for the selected ADC.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.121 Reference BctuAdcNotificationsAdcIndex

ADC instance for which the BCTU notifications are configured.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Adc/AdcConfigSet/AdcHwUnit

4.122 Container BctuListItems

This container allows configuration of BCTU conversion list elements.

IMPORTANT NOTE: Command List is generated in the order given by this array list.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.123 Parameter BctuAdcChannelList

Selects the physical Hardware Adc Channel for the Conversion List. Note: Range of the ADC Channels depends on the selected package.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	P0_ChanNum0
literals	['P0_ChanNum0', 'P1_ChanNum1', 'P2_ChanNum2', 'P3_ChanNum3', 'P4_ChanNum4', 'P5_ChanNum5', 'P6_ChanNum6', 'P7_ChanNum7', 'S8_ChanNum32', 'S9_ChanNum33', 'S10_ChanNum34', 'S11_ChanNum35', 'S12_ChanNum36', 'S13_ChanNum37', 'S14_ChanNum38', 'S15_ChanNum39', 'S16_ChanNum40', 'S17_ChanNum41', 'S18_ChanNum42', 'S19_ChanNum43', 'S20_ChanNum44', 'S21_ChanNum45', 'S22_ChanNum46', 'S23_ChanNum47', 'BANDGAP_ChanNum48', 'TEMPSENSOR_OUTPUT_ChanNum49', 'ANAMUX_OUTPUT_ChanNum50', 'VREFL_ChanNum54', 'VREFH_ChanNum55', 'X0_ChanNum64', 'X0_ChanNum65', 'X0_ChanNum66', 'X0_ChanNum67', 'X0_ChanNum68', 'X0_ChanNum69', 'X0_ChanNum70', 'X0_ChanNum71', 'X1_ChanNum72', 'X1_ChanNum73', 'X1_ChanNum74', 'X1_ChanNum75', 'X1_ChanNum76', 'X1_ChanNum77', 'X1_ChanNum78', 'X1_ChanNum79', 'X2_ChanNum80', 'X2_ChanNum81', 'X2_ChanNum82', 'X2_ChanNum83', 'X2_ChanNum84', 'X2_ChanNum85', 'X2_ChanNum86', 'X2_ChanNum87', 'X3_ChanNum88', 'X3_ChanNum89', 'X3_ChanNum90', 'X3_ChanNum91', 'X3_ChanNum92', 'X3_ChanNum93', 'X3_ChanNum94', 'X3_ChanNum95']

4.124 Parameter BctuNextChannelWaitOnTrig

Disable or enable, if the list execution is continuous (for FALSE value) or if the list execution is halted after executing this channel, until the same trigger responsible for start of list is re-applied/re-asserted.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.125 Parameter BctuLastChannel

If enabled, this is the last channel converted the current list. Another list can be started from the next position in the BCTU list.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.126 Container BctuResultFifos

This container allows configuration of BCTU result FIFOs.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	2
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.127 Parameter BctuResultFifoIndex

Bctu result FIFO index.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	FIFO_1
literals	['FIFO_1', 'FIFO_2']

4.128 Parameter BctuWatermarkValue

If the number of active FIFO entries exceeds the watermark level, a DMA or interrupt request is raised.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	15
min	0

4.129 Parameter BctuFifoNotificationsEnable

Enable FIFO interrupt notifications.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.130 Parameter BctuWatermarkNotification

This function is called if the number of valid FIFO entries is greater than the watermark level. Used for both cases Interrupt and Dma.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.131 Parameter BctuUnderrunNotification

This function is called for FIFO underrun event (FIFO is empty and read attempt is made).

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.132 Parameter BctuOverrunNotification

This function is called for FIFO overrun event (FIFO is full and write attempt is made).

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.133 Parameter BctuFifoDmaEnable

Controls the DMA request that occurs if the number of valid FIFO entries is greater than the watermark level for that FIFO. The Fifo Interrupt should be disabled when actives Dma to prevent conflict Data Transfer.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.134 Parameter BctuFifoDmaBuffer

Pointer to the Fifo Data Buffer (destination for conversion results) when BctuFifoDmaEnable is enabled.

Property	Value
type	ECUC-LINKER-SYMBOL-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	BctuDmaFifo

4.135 Reference BctuFifoDmaChannelId

Select Logical Dma Channel that will be used to transfer the Fifo Buffer from FIFO Result Data Register to the user fifo buffer.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Mcl/MclConfig/dmaLogicChannel_Type

4.136 Container AdcGeneral

General configuration (parameters) of the ADC Driver software module.

Included subcontainers:

- AdcPowerStateConfig

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.137 Parameter AdcDeInitApi

Adds/removes the service Adc_DeInit() from the code.

true: Adc_DeInit() can be used.

false: Adc_DeInit() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.138 Parameter AdcDevErrorDetect

Switches the development error detection and notification on or off.

true: detection and notification is enabled.

false: detection and notification is disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.139 Parameter AdcEnableLimitCheck

Enables or disables limit checking feature in the ADC driver.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.140 Parameter AdcEnableQueuing

Determines, if the queuing mechanism is active in case of priority mechanism disabled.

Note: If priority mechanism is enabled, queuing mechanism is always active and the parameter ADC_ENABLE_QUEUING is not evaluated.

true: Enabled.

false: Disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.141 Parameter AdcPriorityQueueMaxDepth

Maximum depth of queue used for queuing of incoming conversion requests when hardware unit is busy.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	1024
min	1

4.142 Parameter AdcEnableStartStopGroupApi

Adds / removes the services Adc_StartGroupConversion() and Adc_StopGroupConversion() from the code.

true: Adc_StartGroupConversion() and Adc_StopGroupConversion() can be used.

false: Adc_StartGroupConversion() and Adc_StopGroupConversion() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.143 Parameter AdcGrpNotifCapability

Determines, if the group notification mechanism (the functions to enable and disable the notifications) is available at runtime.

true: Enabled.

false: Disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.144 Parameter AdcHwTriggerApi

Adds / removes the services Adc_EnableHardwareTrigger() and Adc_DisableHardwareTrigger() from the code.

true: Adc_EnableHardwareTrigger() and Adc_DisableHardwareTrigger() can be used.

false: Adc_EnableHardwareTrigger() and Adc_DisableHardwareTrigger() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.145 Parameter AdcPriorityImplementation

Determines whether a priority mechanism is available for prioritization of the conversion requests and if available, the type of prioritization mechanism. The selection applies for groups with trigger source software and trigger source hardware. Two types of prioritization mechanism can be selected.

The hardware prioritization mechanism (AdcPriorityHw) uses the ADC hardware features for prioritization of the software conversion requests and hardware trigger signals for groups with trigger source hardware.

The mixed hardware and software prioritization mechanism (AdcPriorityHwSw) uses the ADC hardware features for prioritization of ADC hardware trigger for groups with trigger source hardware and a software implemented prioritization mechanism for groups with trigger source software.

The group priorities for software triggered groups are typically configured with lower priority levels than the group priorities for hardware triggered groups.

ImplementationType: Adc_PriorityImplementationType.

Note: In this version the ADC_PRIORITY_HW isn't used.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_PRIORITY_NONE
literals	['ADC_PRIORITY_HW', 'ADC_PRIORITY_HW_SW', 'ADC_PRIORITY↩ _NONE']

4.146 Parameter AdcReadGroupApi

Adds / removes the service Adc_ReadGroup() from the code.

true: Adc_ReadGroup() can be used.

false: Adc_ReadGroup() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.147 Parameter AdcResultAlignment

Alignment of ADC raw results in ADC result buffer (left/right alignment).

Implementation Type: Adc_ResultAlignmentType.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	ADC_ALIGN_RIGHT
literals	['ADC_ALIGN_RIGHT', 'ADC_ALIGN_LEFT']

4.148 Parameter AdcVersionInfoApi

Adds / removes the service Adc_GetVersionInfo() from the code.

true: Adc_GetVersionInfo() can be used.

false: Adc_GetVersionInfo() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.149 Parameter AdcLowPowerStatesSupport

Adds / removes all power state management related APIs (ADC_SetPowerState, ADC_GetCurrentPowerState, ADC_GetTargetPowerState, ADC_PreparePowerState, ADC_Main_PowerTransitionManager), indicating if the HW offers low power state management.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.150 Parameter AdcPowerStateAsynchTransitionMode

Enables / disables support of the ADC Driver to the asynchronous power state transition. This feature is not implemented on this platform.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.151 Reference AdcEcucPartitionRef

Maps the ADC driver to zero or multiple ECUC partitions to make the driver API available in the according partition.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.152 Reference AdcKernelEcucPartitionRef

Maps the ADC kernel to zero or one ECUC partition to assign the driver kernel to a certain core. The ECUC partition referenced is a subset of the ECUC partitions where the ADC driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.153 Container AdcPowerStateConfig

Each instance of this parameter defines a power state and the callback to be called when this power state is reached.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.154 Parameter AdcPowerState

Each instance of this parameter describes a different power state

supported by the ADC HW. It should be defined by the HW supplier and used by the ADCDriver to reference specific HW configurations which set the ADC HW module in the referenced power state. At least the power mode corresponding to full power state shall be always configured.

This parameter shall only be configured if the parameter AdcLowPowerStatesSupport is set to true.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	9223372036854775807
min	0

4.155 Parameter AdcPowerStateReadyCbkRef

Each instance of this parameter contains a reference to a power mode callback defined in a CDD or IoHwAbs component.

This parameter shall only be configured if the parameter AdcLowPowerStatesSupport is set to true

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.156 Container AdcInterrupt

Selects whether the interrupt for each ADC Unit will be enabled. For each Adc HW unit, there are 2 interrupts that can be enabled: the End of Conversion and the Watchdog interrupts. These settings are used for optimizing the code size by removing the interrupt handling code for interrupts that are not needed.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	2
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.157 Parameter AdcInterruptSource

The name of the interrupt. Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	ADC0_EOC
literals	['ADC0_EOC', 'ADC0_WD', 'ADC1_EOC', 'ADC1_WD', 'ADC2_EOC', 'A↩ DC2_WD']

4.158 Parameter AdcInterruptEnable

Adds / removes the interrupt handling routine from the ADC driver code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.159 Container AdcPublishedInformation

Additional published parameters not covered by CommonPublishedInformation container.

Note that these parameters do not have any configuration class setting, since they are published information.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.160 Parameter AdcChannelValueSigned

Information whether the result value of the ADC driver has sign information (true) or not (false). If the result shall be interpreted as signed value it shall apply to C-language rules.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	false

4.161 Parameter AdcGroupFirstChannelFixed

Information whether the first channel of an ADC Channel group can be configured (false) or is fixed (true) to a value determined by the ADC HW Unit.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	false

4.162 Parameter AdcMaxChannelResolution

Maximum Channel resolution in bits (does not specify accuracy).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	14
max	63
min	1

4.163 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.164 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.165 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.166 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.167 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION

Property	Value
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	123
max	123
min	123

4.168 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	2
max	2
min	2

4.169 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0

Property	Value
max	0
min	0

4.170 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.171 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>_>VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.172 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

4.173 Container AutosarExt

Autosar Extension API settings.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.174 Parameter AdcTimeoutMethod

Configures the timeout method for Adc.

Based on this selection a certain timeout method from OsIf will be used in the driver.

Note: If OSIF_COUNTER_SYSTEM or OSIF_COUNTER_CUSTOM are selected make sure the corresponding timer is enabled in OsIf General configuration.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COU↩ NTER_CUSTOM']

4.175 Parameter AdcTimeoutVal

The timeout is used for preventing endless loops as resulted from driver FMEA analysis.

The hardware failure mode which is preventing, is potential cases with frozen peripheral status used for driver synchronization.

The timeout is used as escape for avoidance of endless loops. For more details please refer to driver FMEA.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	100000
max	4294967295
min	0

4.176 Parameter AdcSarIpDevErrorDetect

This parameter Enables / Disables development error detection for Adc Sar.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.177 Parameter BctuIpDevErrorDetect

This parameter Enables / Disables development error detection for Bctu.

This feature is enabled if Adc CTU Control Mode API from General/AutosarExt is enabled or if Adc Hw Trigger API from General is enabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

Property	Value
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.178 Parameter AdcMulticoreSupport

This parameter globally enables the possibility to support multicore. If this parameter is enabled, at least one EcucPartition needs to be defined (in all variants).

NoteThis is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.179 Parameter AdcEnableGroupDependentChannelNames

This is used to generate ADC symbolic names, that depend also on the ADC group

to which each ADC channel is mapped. The generated symbolic name will be something like:

#define "ADC_GroupName"_"ADC_ChannelName" "Channel index value",

where "Channel index value" is the channel index in the current group.

Channel indexes in each group are generated to allow result buffer access by symbolic names.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.180 Parameter AdcBypassAbortChainCheck

Bypass the delay introduced to check if an aborted conversion chain has stopped.

This increases ADC driver performance at the cost of HW-SW coherency no longer being guaranteed.

The user must make sure he does not call an ADC service before the hardware reaches the correct state.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.181 Parameter AdcConvTimeOnce

Implementation Specific Parameter.

Enable/Disable one time setting of the registers.

If Enabled, the setting of the conversion time registers will be done only once in Adc_Init() function

for the configured hardware unit.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.182 Parameter AdcOptimizeOneShotHwTriggerConversions

Implementation Specific Parameter.

Enable/Disable The Adc driver optimization for HW Triggered groups, OneShot, Single access.

If Enabled, other types of groups cannot be configured in ADC driver and the code for interrupt routine / Dma notification will be optimized for speed.

Also, all groups must have at most 8 channels configured.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.183 Parameter AdcOptimizeDmaStreamingGroups

Implementation Specific Parameter.

Enable/Disable The Adc driver enable Optimize DMA streaming groups for reducing the number of interrupts required for processing the conversions of Adc Groups that consist of one or more channels (depending on HW capabilities) and which are configured as ADC_ACCESS_MODE_STREAMING.

When this feature is enabled, only one interrupt will be raised after the completion of all stream conversions (as configured by AdcStreamingNumSamples parameter). An additional interrupt to be raised after half of the stream is converted shall also be configurable.

This feature is enabled if Adc Global Enable DMA Transfer from General/AutosarExt is also enabled.

Note:

- SetChannel(), Enable/DisableChannel() and Optimize one-shot hardware trigger cannot be use concurrently with this feature.

This node is EDITABLE only if "Adc Enable DMA support" is enabled

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.184 Parameter AdcPreSamplingOnce

Implementation Specific Parameter.

Enable/Disable one time setting of the registers.

If Enabled, the setting of the presampling time registers will be done only once in Adc_Init() function

for the configured hardware unit.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.185 Parameter AdcEnableInitialNotification

Enable/disable an extra notification to be called for each Adc Group conversion.

This feature is intended to be used together with Adc_SetChannel service. The initial notification can be used by the user application to call Adc_SetChannel API before ADC driver updates the hardware configuration for the next conversion.

This node is EDITABLE only if "Adc Set Channel API" is enabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.186 Parameter AdcEnableDmaTransferMode

This parameter enables the possibility to configure DMA transfer for ADC converted data. If this parameter is disabled then DMA handling code will be removed at pre-compile time and DMA transfer cannot be configure for any Adc unit in any variant. If this parameter is enabled then the DMA configuration code will not be removed.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.187 Parameter AdcUseSoftwareInjectedGroups

This parameter defines if Software Injected Groups are used in any Hardware Unit, any variant.

It needs to be enabled if Software Injected Groups are needed. If Software Injected Groups are not needed, this parameter should be disabled for code optimizations.

NoteThis is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.188 Parameter AdcUseHardwareNormalGroups

This parameter defines if Hardware Normal Groups are used in any Hardware Unit, any variant. It needs to be enabled if Hardware Normal Groups are needed. If Hardware Normal Groups are not needed, this parameter should be disabled for code optimizations. Normal Hardware conversions are not supported on this platform.

NoteThis is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.189 Parameter AdcEnableUserModeSupport

When this parameter is enabled, the Adc module will adapt to run from User Mode, by configuring REG_PROT for ADC IPs

Note: The Adc driver code can be executed at any time from both supervisor and user mode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.190 Parameter AdcSetHwUnitPowerModeApi

Adds/removes the Autosar Extension implementation API Adc_SetHwUnitPowerMode() and Adc_CtuSetPowerMode() from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.191 Parameter AdcEnableChDisableChApi

Enable/disable the Autosar Extension implementation api(s) Adc_EnableChannel() and Adc_DisableChannel() in ADC driver.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.192 Parameter AdcGetInjectedConvStatusApi

Enable/disable the Autosar Extension API Adc_GetInjectedConversionStatus() in ADC driver.

NoteThis is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.193 Parameter AdcEnableThresholdConfigurationApi

Enable/disable the Autosar Extension APIs Adc_ConfigureThreshold() in ADC driver.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.194 Parameter AdcEnableCtuTrigAutosarExtApi

This is used to enable the Autosar Extension API for the hardware triggered group.

If this parameter is enabled then Adc_EnableCTUTrigger(), Adc_DisableCTUTrigger() and Adc_HwResultReadGroup() will be available in the driver code.

This is an Implementation Specific Parameter.

When this parameter is enabled, the result buffer is no longer to be used to read the results as the result will be directly read from HW registers.

When this parameter is disabled, normal functionality shall be executed.

This node is EDITABLE only if "Adc Hw Trigger API" is enabled

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.195 Parameter AdcCtuHardwareTriggerOptimization

When this parameter is enabled, the Bctu channel lists and triggers are configured only once at initialization in order to reduce cpu load when calling the Adc_EnableHardwareTrigger and Adc_EnableCtuTrigger APIs.

The Adc_SetChannel service cannot be used, and the maximum size of groups cannot exceed the size of the entire command list divided by the number of adc hardware units triggered by the CTU/BCTU.

This node is EDITABLE only if "Adc Hw Trigger API" is enabled

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.196 Parameter AdcEnableCtuControlModeApi

This is used to enable the Autosar Extension API for the enabling and disabling CTU control mode for an ADC unit.

If this parameter is enabled than Adc_EnableCtuControlMode(), Adc_DisableCtuControlMode() will be available in the driver code.

When a unit works in CTU control mode, no other conversions shall run in parallel(Adc). The only conversions occurring shall be the ones defined in the CTU configuration.

If AdcEnableCtuControlModeApi is enabled, BCTU must be configured.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.197 Parameter CtuEnableDmaTransferMode

This parameter enables the possibility to configure DMA transfer for BCTU control mode.

If this parameter is disabled then DMA handling code will be removed at pre-compile time and DMA transfer cannot be configured for any BCTU unit in any variant.

If this parameter is enabled then the DMA configuration code will not be removed.

Note: This is an Implementation Specific Parameter.

This node is EDITABLE only if "Adc CTU Control Mode API" is enabled

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.198 Parameter AdcEnableWatchdogApi

This parameter globally enables the possibility to use the Adc Watchdog feature. If this parameter is disabled, the Watchdog handling code will be removed at pre-compile time and nothing related to this functionality can be configured in any unit, for anu variant. If this parameter is enabled, Analog Watchdog functionality can be configured.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.199 Parameter AdcEnableSetChannel

If this parameter has been configured to "TRUE", the Autosar Extension function "Adc_SetChannel()" shall be accessible, otherwise this function shall be removed from the code.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.200 Parameter AdcEnableDualClockMode

Adds/removes the Dual Clock mode service Adc_SetClockMode from the code.

Also it enables the Programming of Conversion Timing registers in Adc_SetClockMode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.201 Parameter AdcEnableCalibration

If this parameter has been configured to "TRUE", the Autosar Extension function "Adc_Calibrate()" shall be accessible, otherwise this function shall be removed from the code.

Note This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.202 Parameter AdcEnableSelfTest

If this parameter has been configured to "TRUE", the Autosar Extension function "Adc_SelfTest()" shall be

accessible, otherwise this function shall be removed from the code.

NoteThis is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.203 Parameter AdcEnableReadRawDataApi

When this parameter is enabled, the Api for reading the raw result data from an ADC unit is available to use at runtime.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.204 Parameter AdcEnableGroupStreamingResultReorder

When this parameter is enabled, the adc results can be arranged as multiple sets of group result buffer if AdcStreamResultGrou is enabled for that selected group.

E.g: for a group with channels {CH1 CH5 CH7} the resulting stream buffer shall be:

{ CH1, CH5, CH7, CH1, CH5, CH7, CH1, CH5, CH7}

instead of

{ CH1, CH1, CH1, CH5, CH5, CH5, CH7, CH7, CH7} like supported by AUTOSAR standard.

Apply only for ADC_ACCESS_MODE_STREAMING Access Mode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.205 Parameter AdcSarEnableTempsenseApi

When this parameter is enabled, the Apis for the tempsense are available to be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.206 Parameter AdcSarPowerOnTempSense

When this parameter is enabled, the TempSense module is powered on.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.207 Parameter AdcSarTempSenseVsupply

Voltage Supply of TempSense module, expressed in fixed point format with 1 bit for the sign, 11 bits for the integer part and 4 bits for the decimal part. The value must be filled in by the user according to actual board setup. It and will be used for calculating the value measured by TempSense.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	53
max	88
min	0

Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Adc driver 147
Adc Sar IPL 195
Bctu IPL 232

Chapter 6

Module Documentation

6.1 Adc driver

6.1.1 Detailed Description Adc HLD.

Adc Autosar High Level Driver.

Data Structures

- struct Adc_ValidationResultType
  Structure for validation results. More...
- struct Adc_GroupStatusType
  Structure for group status. More...
- struct Adc_UnitStatusType
  Structure for hardware unit status. More...
- struct Adc_ConfigType
  Structure for ADC configuration. More...

Macros

- #define ADC_E_UNINIT
  API service used without Adc module initialization.
- #define ADC_E_BUSY
  Adc module is busy with a running operation.
- #define ADC_E_IDLE
  Adc module is in idle state.
- #define ADC_E_ALREADY_INITIALIZED
  The ADC module is already initialized.
- #define ADC_E_PARAM_CONFIG
  The ADC module is not properly configured.
- #define ADC_E_PARAM_POINTER
  API service is called using an invalid pointer (e.g. the pointer should not be NULL).

- #define ADC_E_PARAM_GROUP

  API service used with an invalid ADC group.
- #define ADC_E_WRONG_CONV_MODE

  API service used with an invalid ADC Conversion Mode.
- #define ADC_E_WRONG_TRIGG_SRC

  API service used with an invalid ADC Trigger Source.
- #define ADC_E_NOTIF_CAPABILITY

  Check the notification capability of a group.
- #define ADC_E_BUFFER_UNINIT

  API service used without initializing the buffer.
- #define ADC_E_NOT_DISENGAGED

  One or more ADC group/channel not in IDLE state.
- #define ADC_E_POWER_STATE_NOT_SUPPORTED

  Unsupported power state request.
- #define ADC_E_TRANSITION_NOT_POSSIBLE

  Requested power state can not be reached directly.
- #define ADC_E_PERIPHERAL_NOT_PREPARED

  ADC not prepared for target power state.
- #define ADC_E_QUEUE_FULL

  The Adc_StartGroupConversion and Adc_EnableHardwareTrigger services can not queue another conversion (queue is full)
- #define ADC_E_SET_HW_UNIT_POWER_MODE

  An error occurred when the Adc_SetHwUnitPowerMode services is used.
- #define ADC_E_PARAM_TRIGGER

  Wrong trigger source to be used for the group.
- #define ADC_E_WRONG_CONF_THRHLD_VALUE

  Adc_ConfigureThreshold service is called using wrong values.
- #define ADC_E_PARAM_UNIT

  API service called using a wrong ADC unit.
- #define ADC_E_INVALID_CLOCK_MODE

  Adc_SetClockMode service called using an invalid clock mode.
- #define ADC_E_TIMEOUT

  An error occurred if the timeout counter variable has expired when checking status flags.
- #define ADC_E_CONTROL_MODE_DISABLED

  Error when a function which works only in CTU Control Mode is called when CTU control mode is disabled.
- #define ADC_INIT_ID

  API service ID for Adc_Init function.
- #define ADC_DEINIT_ID

  API service ID for Adc_DeInit function.
- #define ADC_STARTGROUPCONVERSION_ID

  API service ID for Adc_StartGroupConversion function.
- #define ADC_STOPGROUPCONVERSION_ID

  API service ID for Adc_StopGroupConversion function.
- #define ADC_VALUEREADGROUP_ID

  API service ID for Adc_ReadGroup function.
- #define ADC_ENABLEHARDWARETRIGGER_ID

  API service ID for Adc_EnableHardwareTrigger function.

- #define ADC_DISABLEHARDWARETRIGGER_ID

  API service ID for Adc_DisableHardwareTrigger function.
- #define ADC_ENABLEGROUPNOTIFICATION_ID

  API service ID for Adc_EnableGroupNotification function.
- #define ADC_DISABLEGROUPNOTIFICATION_ID

  API service ID for Adc_DisableGroupNotification function.
- #define ADC_GETGROUPSTATUS_ID

  API service ID for Adc_GetGroupStatus function.
- #define ADC_GETVERSIONINFO_ID

  API service ID for Adc_GetVersionInfo function.
- #define ADC_GETSTREAMLASTPOINTER_ID

  API service ID for Adc_GetStreamLastPointer function.
- #define ADC_SETUPRESULTBUFFER_ID

  API service ID for Adc_SetupResultBuffer function.
- #define ADC_SETPOWERSTATE_ID

  API service ID for Adc_SetPowerState function.
- #define ADC_GETCURRENTPOWERSTATE_ID

  API service ID for Adc_GetCurrentPowerState function.
- #define ADC_GETTARGETPOWERSTATE_ID

  API service ID for Adc_GetTargetPowerState function.
- #define ADC_PREPAREPOWERSTATE_ID

  API service ID for Adc_PreparePowerState function.
- #define ADC_HWRESULTREADGROUP_ID

  API service ID for Adc_HwResultReadGroup function.
- #define ADC_ENABLECTUTRIGGER_ID

  API service ID for Adc_EnableCTUTrigge function.
- #define ADC_DISABLECTUTRIGGER_ID

  API service ID for Adc_DisableCTUTrigger function.
- #define ADC_SET_HW_UNIT_POWER_MODE_ID

  API service ID for Adc_SetHwUnitPowerMode function.
- #define ADC_CTU_SET_POWER_MODE_ID

  API service ID for Adc_CtuSetPowerMode function.
- #define ADC_SETCLOCKMODE_ID

  API service ID for Adc_SetClockMode function.
- #define ADC_GETINJECTEDCONVERSIONSTATUS_ID

  API service ID for Adc_GetInjectedConversionStatus function.
- #define ADC_CALIBRATE_ID

  API service ID for Adc_Calibrate function.
- #define ADC_CONFIGURE_THRESHOLD_ID

  API service ID for Adc_ConfigureThreshold function.
- #define ADC_ENABLE_CTU_CONTROL_MODE_ID

  API service ID for Adc_EnableCtuControlMode function.
- #define ADC_DISABLE_CTU_CONTROL_MODE_ID

  API service ID for Adc_DisableCtuControlMode function.

Types Reference

- typedef void(∗ Adc_NotifyType) (void)

  Notification function pointer definition.
- typedef uint8 Adc_ResolutionType

  channel resolution in number of bits
- typedef Adc_ChannelType Adc_GroupDefType

  definition of channels in a group
- typedef uint8 Adc_PrescaleType

  clock prescaler factor
- typedef uint8 Adc_SamplingTimeType

  sampling time
- typedef uint16 Adc_StreamNumSampleType

  Number of samples of a streaming conversion buffer.

Enum Reference

- enum Adc_GlobalStateType

  ADC driver status.
- enum Adc_GroupConversionStateType

  ADC group already converted type.
- enum Adc_GroupAccessModeType

  Adc group access Mode.
- enum Adc_GroupReplacementType

  Adc group replacement.
- enum Adc_StreamBufferModeType

  Adc group streaming buffer mode.
- enum Adc_StatusType

  ADC group status.
- enum Adc_NotificationType

  ADC group notification.
- enum Adc_HwTriggerSignalType

  Adc hardware trigger edge.
- enum Adc_TriggerSourceType

  Adc hardware trigger source.
- enum Adc_HwTriggeringType

  Adc Hardware trigger.

Function Reference

- void Adc_Init (const Adc_ConfigType *ConfigPtr)

  Initializes the ADC hardware unit and the driver.
- Std_ReturnType Adc_SetupResultBuffer (Adc_GroupType Group, Adc_ValueGroupType *const Data↩ BufferPtr)

  Initializes the group specific ADC result buffer pointer as configured to point to the pDataBufferPtr address which is passed as parameter.
- void Adc_DeInit (void)

  Returns all ADC HW Units to a state comparable to their power on reset state.
- void Adc_StartGroupConversion (Adc_GroupType Group)

  Starts the conversion of all channels of the requested ADC Channel group.
- void Adc_StopGroupConversion (Adc_GroupType Group)

  Stops the conversion of all channels of the requested ADC Channel group.
- Std_ReturnType Adc_ReadGroup (Adc_GroupType Group, Adc_ValueGroupType *DataBufferPtr)

  Reads the group conversion results.
- void Adc_ReadRawData (Adc_HwUnitType Unit, const Adc_ChannelType *const ChansArray, uint8 Num↩ Items, Adc_ValueGroupType *const DataBufferPtr)

  Read the raw result data from an ADC unit.
- void Adc_EnableHardwareTrigger (Adc_GroupType Group)

  Enables the hardware trigger for the requested ADC Channel group.
- void Adc_DisableHardwareTrigger (Adc_GroupType Group)

  Disables the hardware trigger for the requested ADC Channel group.
- void Adc_EnableGroupNotification (Adc_GroupType Group)

  Enables the notification mechanism for the requested ADC channel group.
- void Adc_DisableGroupNotification (Adc_GroupType Group)

  Disables the notification mechanism for the requested ADC channel group.
- Adc_StatusType Adc_GetGroupStatus (Adc_GroupType Group)

  Returns the conversion status of the requested ADC Channel group.
- Adc_StreamNumSampleType Adc_GetStreamLastPointer (Adc_GroupType Group, Adc_ValueGroupType **PtrToSamplePtr)

  Returns the number of valid samples per channel.
- void Adc_GetVersionInfo (Std_VersionInfoType *versioninfo)

  Returns the version information of this module.
- Std_ReturnType Adc_SetHwUnitPowerMode (Adc_HwUnitType Unit, Adc_SetPowerModeType SetPower↩ Mode)

  Set the ADC mode either to powerdown or normal.
- Std_ReturnType Adc_CtuSetPowerMode (Adc_HwUnitType CtuUnit, Adc_PowerStateType State)

  Function to set BCTU/CTU power mode.
- void Adc_EnableCTUTrigger (Adc_GroupType Group, Adc_HwTriggerTimerType TriggerSource)

  Enable the TriggerSource for group selected by Group parameter.
- void Adc_DisableCTUTrigger (Adc_GroupType Group, Adc_HwTriggerTimerType TriggerSource)

  Disable the TriggerSource for group selected by Group parameter.
- Std_ReturnType Adc_HwResultReadGroup (Adc_GroupType Group, Adc_ValueGroupType *DataPtr)

  Read the result of the hardware triggered groups conversion result.
- Adc_StatusType Adc_GetInjectedConversionStatus (Adc_HwUnitType Unit)

  Get the injected conversions status.

- void Adc_Calibrate (Adc_HwUnitType Unit, Adc_CalibrationStatusType *pStatus)

  Executes high accuracy calibration of a ADC HW unit.
- Std_ReturnType Adc_SelfTest (Adc_HwUnitType Unit)

  Executes hardware Self Test of a ADC HW unit.
- void Adc_ConfigureThreshold (Adc_ThresholdControlIndexType ThresholdControlIndex, Adc_Wdg↩ ThresholdValueType LowValue, Adc_WdgThresholdValueType HighValue)

  Function to reconfigure High and Low thresholds for a given threshold control index.
- void Adc_EnableWdgNotification (Adc_ChannelType ChannelId)

  Enable notification of a channel that has watchdog functionality configured at initialization.
- void Adc_DisableWdgNotification (Adc_ChannelType ChannelId)

  Disable notification of a channel that has watchdog functionality configured at initialization.
- Std_ReturnType Adc_SetClockMode (Adc_SelectPrescalerType Prescaler)

  Set the ADC clock prescaler if available and modify the conversion timings.
- Std_ReturnType Adc_SetPowerState (Adc_PowerStateRequestResultType *Result)

  Enters the already prepared power state.
- Std_ReturnType Adc_GetCurrentPowerState (Adc_PowerStateType *CurrentPowerState, Adc_Power↩ StateRequestResultType *Result)

  Get the current power state of the ADC HW unit.
- Std_ReturnType Adc_GetTargetPowerState (Adc_PowerStateType *TargetPowerState, Adc_PowerState↩ RequestResultType *Result)

  Get the target power state of the ADC HW unit.
- Std_ReturnType Adc_PreparePowerState (Adc_PowerStateType PowerState, Adc_PowerStateRequest↩ ResultType *Result)

  Starts the needed process to allow the ADC HW module to enter the requested power state.
- void Adc_EnableCtuControlMode (Adc_HwUnitType Unit)

  Function to enable CTU control mode for an ADC unit.
- void Adc_DisableCtuControlMode (Adc_HwUnitType Unit)

  Function to disable CTU control mode for an ADC unit.
- void Adc_CtuEnableHwTrigger (Adc_CtuTrigSrcType TrigSource)

  Function to enable CTU hardware trigger.
- void Adc_CtuDisableHwTrigger (Adc_CtuTrigSrcType TrigSource)

  Function to disable CTU hardware trigger.
- void Adc_CtuStartConversion (Adc_CtuTrigSrcType TrigSource)

  Function to start CTU conversion.
- Adc_ValueGroupType Adc_CtuReadConvData (Adc_HwUnitType AdcUnit)

  Function to read BCTU conversion data.
- void Adc_CtuReadConvResult (Adc_HwUnitType AdcUnit, Adc_CtuResultType *pResult)

  Function to read BCTU conversion result.
- void Adc_CtuStopLoopConversions (Adc_CtuTrigSrcType TrigSource)

  Function to stop CTU loop conversion.
- void Adc_CtuReadFifoData (Adc_CtuFifoIdxType FifoIdx, uint16 *Data, uint8 DataLength)

  Function to read CTU conversion data from FIFO.
- void Adc_CtuReadFifoResult (Adc_CtuFifoIdxType FifoIdx, Adc_CtuFifoResultType *Result, uint8 ResultLength)

  Function to read CTU conversion results from FIFO.
- void Adc_CtuSetFifoWatermark (Adc_CtuFifoIdxType FifoIdx, uint8 Watermark)

  Function to set CTU FIFO watermark.

- void Adc_CtuEnableNotification (Adc_CtuNotificationType Notification)

  Function to enable CTU notification.
- void Adc_CtuDisableNotification (Adc_CtuNotificationType Notification)

  Function to disable CTU notification.
- void Adc_CtuSetList (const Adc_CtuListItemType *const ListItemsArray, const uint8 NumItems, const uint8 ListStartPosition)

  Function to set CTU list.
- void Adc_CtuSetListPointer (Adc_CtuTrigSrcType TrigSource, Adc_CtuListPtrType ListPtr)

  Reconfigure the list pointer of a triggered CTU source.
- uint16 Adc_TempSenseCalculateTemp (Adc_HwUnitType Unit, const uint16 TempSenseAdcConvData)

  Function to calculate temperature on chip from provided data.
- Std_ReturnType Adc_TempSenseGetTemp (Adc_HwUnitType Unit, uint16 *const TempSenseVal)

  Function to get the temperature on chip directly.
- Std_ReturnType Adc_TempSenseSetPowerMode (Adc_SetPowerModeType SetPowerMode)

  Set the Tempsense mode either to powerdown or normal.

Variables

- const Adc_ConfigType * Adc_apxCfgPtr [(1U)]

  Used to point the configuration structure.

6.1.2 Data Structure Documentation

6.1.2.1 struct Adc_ValidationResultType

Structure for validation results.

This structure contains the validation information

Definition at line 286 of file Adc_Types.h.

Data Fields

Type	Name	Description
boolean	EndValidations	Signal if validation ended.
Std_ReturnType	ValidParams	Return status.

6.1.2.2 struct Adc_GroupStatusType

Structure for group status.

This structure contains the group status information.

Definition at line 297 of file Adc_Types.h.

Data Fields

Type	Name	Description
volatile Adc_StatusType	Conversion	Group status.
volatile Adc_GroupConversionStateType	AlreadyConverted	Group was previously converted or not.
Adc_HwTriggeringType	HwTriggering	hw trigger enabled/disabled
Adc_NotificationType	Notification	notification enabled/disabled
volatile Adc_StreamNumSampleType	ResultIndex	index into streaming buffer that is currently being filled
Adc_ValueGroupType *	ResultsBufferPtr	Pointer to user result buffer array.
volatile boolean	LimitCheckFailed	check limit check fail

6.1.2.3 struct Adc_UnitStatusType

Structure for hardware unit status.

This structure contains the HW unit status information.

Definition at line 323 of file Adc_Types.h.

Data Fields

Type	Name	Description
volatile Adc_QueueIndexType	SwNormalQueueIndex	Filled slots in the queue.
volatile Adc_GroupType	SwNormalQueue[(1U)]	Queued groups indexes, always executing Queue[0].
volatile Adc_GroupType	OngoingHwGroup	Ongoing hardware group ID.
volatile Adc_GroupType	SwInjectedQueue[1U]	The depth of the software injected queue.
volatile Adc_QueueIndexType	SwInjectedQueueIndex	Filled slots in the Sw injected queue.
boolean	CtuControlOngoing	Indicates Ctu control mode is ongoing.

6.1.2.4 struct Adc_ConfigType

Structure for ADC configuration.

Data structure containing the set of configuration parameters required for initializing the ADC Driver.

SWS_Adc_00505

Definition at line 442 of file Adc_Types.h.

Data Fields

- const Adc_GroupConfigurationType * GroupsPtr
  Group configurations.
- Adc_GroupType GroupCount
  Total number of groups.
- const uint16 * GroupIdToIndexMapPtr
  Miscellaneous configuration parameters.
- uint32 CoreId
  Configuration CoreID.
- const uint8 AssignedPartitionCount
  Number of Partition.

6.1.2.4.1 Field Documentation

6.1.2.4.1.1 GroupsPtr `const Adc_GroupConfigurationType* GroupsPtr`

Group configurations.

Definition at line 447 of file Adc_Types.h.

6.1.2.4.1.2 GroupCount `Adc_GroupType GroupCount`

Total number of groups.

Definition at line 449 of file Adc_Types.h.

6.1.2.4.1.3 GroupIdToIndexMapPtr `const uint16* GroupIdToIndexMapPtr`

Miscellaneous configuration parameters.

Definition at line 451 of file Adc_Types.h.

6.1.2.4.1.4 CoreId `uint32 CoreId`

Configuration CoreID.

Assigned Partition

Definition at line 453 of file Adc_Types.h.

6.1.2.4.1.5 AssignedPartitionCount `const uint8 AssignedPartitionCount`

Number of Partition.

<

Definition at line 457 of file Adc_Types.h.

6.1.3 Macro Definition Documentation

6.1.3.1 ADC_E_UNINIT

```
#define ADC_E_UNINIT
```

API service used without Adc module initialization.

Development errors. The following errors shall be detectable by the ADC module depending on its configuration (development / production mode).

All error codes

Definition at line 133 of file Adc.h.

6.1.3.2 ADC_E_BUSY

```
#define ADC_E_BUSY
```

Adc module is busy with a running operation.

Definition at line 138 of file Adc.h.

6.1.3.3 ADC_E_IDLE

```
#define ADC_E_IDLE
```

Adc module is in idle state.

Definition at line 143 of file Adc.h.

6.1.3.4 ADC_E_ALREADY_INITIALIZED

```
#define ADC_E_ALREADY_INITIALIZED
```

The ADC module is already initialized.

Definition at line 148 of file Adc.h.

6.1.3.5 ADC_E_PARAM_CONFIG

```
#define ADC_E_PARAM_CONFIG
```

The ADC module is not properly configured.

Definition at line 153 of file Adc.h.

6.1.3.6 ADC_E_PARAM_POINTER

```
#define ADC_E_PARAM_POINTER
```

API service is called using an invalid pointer (e.g. the pointer should not be NULL).

Definition at line 158 of file Adc.h.

6.1.3.7 ADC_E_PARAM_GROUP

```
#define ADC_E_PARAM_GROUP
```

API service used with an invalid ADC group.

Definition at line 163 of file Adc.h.

6.1.3.8 ADC_E_WRONG_CONV_MODE

```
#define ADC_E_WRONG_CONV_MODE
```

API service used with an invalid ADC Conversion Mode.

Definition at line 168 of file Adc.h.

6.1.3.9 ADC_E_WRONG_TRIGG_SRC

```
#define ADC_E_WRONG_TRIGG_SRC
```

API service used with an invalid ADC Trigger Source.

Definition at line 173 of file Adc.h.

6.1.3.10 ADC_E_NOTIF_CAPABILITY

```
#define ADC_E_NOTIF_CAPABILITY
```

Check the notification capability of a group.

Definition at line 178 of file Adc.h.

6.1.3.11 ADC_E_BUFFER_UNINIT

```
#define ADC_E_BUFFER_UNINIT
```

API service used without initializing the buffer.

Definition at line 183 of file Adc.h.

6.1.3.12 ADC_E_NOT_DISENGAGED

```
#define ADC_E_NOT_DISENGAGED
```

One or more ADC group/channel not in IDLE state.

Definition at line 189 of file Adc.h.

6.1.3.13 ADC_E_POWER_STATE_NOT_SUPPORTED

```
#define ADC_E_POWER_STATE_NOT_SUPPORTED
```

Unsupported power state request.

Definition at line 194 of file Adc.h.

6.1.3.14 ADC_E_TRANSITION_NOT_POSSIBLE

```
#define ADC_E_TRANSITION_NOT_POSSIBLE
```

Requested power state can not be reached directly.

Definition at line 199 of file Adc.h.

6.1.3.15 ADC_E_PERIPHERAL_NOT_PREPARED

```
#define ADC_E_PERIPHERAL_NOT_PREPARED
```

ADC not prepared for target power state.

Definition at line 204 of file Adc.h.

6.1.3.16 ADC_E_QUEUE_FULL

```
#define ADC_E_QUEUE_FULL
```

The Adc_StartGroupConversion and Adc_EnableHardwareTrigger services can not queue another conversion (queue is full)

Definition at line 211 of file Adc.h.

6.1.3.17 ADC_E_SET_HW_UNIT_POWER_MODE

```
#define ADC_E_SET_HW_UNIT_POWER_MODE
```

An error occurred when the Adc_SetHwUnitPowerMode services is used.

Definition at line 217 of file Adc.h.

6.1.3.18 ADC_E_PARAM_TRIGGER

```
#define ADC_E_PARAM_TRIGGER
```

Wrong trigger source to be used for the group.

Definition at line 224 of file Adc.h.

6.1.3.19 ADC_E_WRONG_CONF_THRHLD_VALUE

```
#define ADC_E_WRONG_CONF_THRHLD_VALUE
```

Adc_ConfigureThreshold service is called using wrong values.

Definition at line 242 of file Adc.h.

6.1.3.20 ADC_E_PARAM_UNIT

```
#define ADC_E_PARAM_UNIT
```

API service called using a wrong ADC unit.

Definition at line 248 of file Adc.h.

6.1.3.21 ADC_E_INVALID_CLOCK_MODE

```
#define ADC_E_INVALID_CLOCK_MODE
```

Adc_SetClockMode service called using an invalid clock mode.

Definition at line 265 of file Adc.h.

6.1.3.22 ADC_E_TIMEOUT

```
#define ADC_E_TIMEOUT
```

An error occurred if the timeout counter variable has expired when checking status flags.

Definition at line 278 of file Adc.h.

6.1.3.23 ADC_E_CONTROL_MODE_DISABLED

```
#define ADC_E_CONTROL_MODE_DISABLED
```

Error when a function which works only in CTU Control Mode is called when CTU control mode is disabled.

Definition at line 284 of file Adc.h.

6.1.3.24 ADC_INIT_ID

```
#define ADC_INIT_ID
```

API service ID for Adc_Init function.

All AUTOSAR API's service IDs

Definition at line 303 of file Adc.h.

6.1.3.25 ADC_DEINIT_ID

```
#define ADC_DEINIT_ID
```

API service ID for Adc_DeInit function.

Definition at line 308 of file Adc.h.

6.1.3.26 ADC_STARTGROUPCONVERSION_ID

```
#define ADC_STARTGROUPCONVERSION_ID
```

API service ID for Adc_StartGroupConversion function.

Definition at line 313 of file Adc.h.

6.1.3.27 ADC_STOPGROUPCONVERSION_ID

```
#define ADC_STOPGROUPCONVERSION_ID
```

API service ID for Adc_StopGroupConversion function.

Definition at line 318 of file Adc.h.

6.1.3.28 ADC_VALUEREADGROUP_ID

```
#define ADC_VALUEREADGROUP_ID
```

API service ID for Adc_ReadGroup function.

Definition at line 323 of file Adc.h.

6.1.3.29 ADC_ENABLEHARDWARETRIGGER_ID

```
#define ADC_ENABLEHARDWARETRIGGER_ID
```

API service ID for Adc_EnableHardwareTrigger function.

Definition at line 328 of file Adc.h.

6.1.3.30 ADC_DISABLEHARDWARETRIGGER_ID

```
#define ADC_DISABLEHARDWARETRIGGER_ID
```

API service ID for Adc_DisableHardwareTrigger function.

Definition at line 333 of file Adc.h.

6.1.3.31 ADC_ENABLEGROUPNOTIFICATION_ID

```
#define ADC_ENABLEGROUPNOTIFICATION_ID
```

API service ID for Adc_EnableGroupNotification function.

Definition at line 338 of file Adc.h.

6.1.3.32 ADC_DISABLEGROUPNOTIFICATION_ID

```
#define ADC_DISABLEGROUPNOTIFICATION_ID
```

API service ID for Adc_DisableGroupNotification function.

Definition at line 343 of file Adc.h.

6.1.3.33 ADC_GETGROUPSTATUS_ID

```
#define ADC_GETGROUPSTATUS_ID
```

API service ID for Adc_GetGroupStatus function.

Definition at line 348 of file Adc.h.

6.1.3.34 ADC_GETVERSIONINFO_ID

```
#define ADC_GETVERSIONINFO_ID
```

API service ID for Adc_GetVersionInfo function.

Definition at line 353 of file Adc.h.

6.1.3.35 ADC_GETSTREAMLASTPOINTER_ID

```
#define ADC_GETSTREAMLASTPOINTER_ID
```

API service ID for Adc_GetStreamLastPointer function.

Definition at line 358 of file Adc.h.

6.1.3.36 ADC_SETUPRESULTBUFFER_ID

```
#define ADC_SETUPRESULTBUFFER_ID
```

API service ID for Adc_SetupResultBuffer function.

Definition at line 363 of file Adc.h.

6.1.3.37 ADC_SETPOWERSTATE_ID

```
#define ADC_SETPOWERSTATE_ID
```

API service ID for Adc_SetPowerState function.

Definition at line 369 of file Adc.h.

6.1.3.38 ADC_GETCURRENTPOWERSTATE_ID

```
#define ADC_GETCURRENTPOWERSTATE_ID
```

API service ID for Adc_GetCurrentPowerState function.

Definition at line 374 of file Adc.h.

6.1.3.39 ADC_GETTARGETPOWERSTATE_ID

```
#define ADC_GETTARGETPOWERSTATE_ID
```

API service ID for Adc_GetTargetPowerState function.

Definition at line 379 of file Adc.h.

6.1.3.40 ADC_PREPAREPOWERSTATE_ID

```
#define ADC_PREPAREPOWERSTATE_ID
```

API service ID for Adc_PreparePowerState function.

Definition at line 384 of file Adc.h.

6.1.3.41 ADC_HWRESULTREADGROUP_ID

```
#define ADC_HWRESULTREADGROUP_ID
```

API service ID for Adc_HwResultReadGroup function.

All Autosar Extension API's service IDs NOTE: Parameters used when raising an error/exception

Definition at line 395 of file Adc.h.

6.1.3.42 ADC_ENABLECTUTRIGGER_ID

```
#define ADC_ENABLECTUTRIGGER_ID
```

API service ID for Adc_EnableCTUTrigge function.

Definition at line 399 of file Adc.h.

6.1.3.43 ADC_DISABLECTUTRIGGER_ID

```
#define ADC_DISABLECTUTRIGGER_ID
```

API service ID for Adc_DisableCTUTrigger function.

Definition at line 403 of file Adc.h.

6.1.3.44 ADC_SET_HW_UNIT_POWER_MODE_ID

```
#define ADC_SET_HW_UNIT_POWER_MODE_ID
```

API service ID for Adc_SetHwUnitPowerMode function.

Definition at line 410 of file Adc.h.

6.1.3.45 ADC_CTU_SET_POWER_MODE_ID

```
#define ADC_CTU_SET_POWER_MODE_ID
```

API service ID for Adc_CtuSetPowerMode function.

Definition at line 415 of file Adc.h.

6.1.3.46 ADC_SETCLOCKMODE_ID

```
#define ADC_SETCLOCKMODE_ID
```

API service ID for Adc_SetClockMode function.

Definition at line 423 of file Adc.h.

6.1.3.47 ADC_GETINJECTEDCONVERSIONSTATUS_ID

```
#define ADC_GETINJECTEDCONVERSIONSTATUS_ID
```

API service ID for Adc_GetInjectedConversionStatus function.

Definition at line 441 of file Adc.h.

6.1.3.48 ADC_CALIBRATE_ID

```
#define ADC_CALIBRATE_ID
```

API service ID for Adc_Calibrate function.

Definition at line 448 of file Adc.h.

6.1.3.49 ADC_CONFIGURE_THRESHOLD_ID

```
#define ADC_CONFIGURE_THRESHOLD_ID
```

API service ID for Adc_ConfigureThreshold function.

Definition at line 459 of file Adc.h.

6.1.3.50 ADC_ENABLE_CTU_CONTROL_MODE_ID

```
#define ADC_ENABLE_CTU_CONTROL_MODE_ID
```

API service ID for Adc_EnableCtuControlMode function.

Definition at line 492 of file Adc.h.

6.1.3.51 ADC_DISABLE_CTU_CONTROL_MODE_ID

```
#define ADC_DISABLE_CTU_CONTROL_MODE_ID
```

API service ID for Adc_DisableCtuControlMode function.

Definition at line 496 of file Adc.h.

6.1.4 Types Reference

6.1.4.1 Adc_NotifyType

```
typedef void(* Adc_NotifyType) (void)
```

Notification function pointer definition.

Definition at line 253 of file Adc_Types.h.

6.1.4.2 Adc_ResolutionType

```
typedef uint8 Adc_ResolutionType
```

channel resolution in number of bits

Definition at line 257 of file Adc_Types.h.

6.1.4.3 Adc_GroupDefType

```
typedef Adc_ChannelType Adc_GroupDefType
```

definition of channels in a group

Definition at line 267 of file Adc_Types.h.

6.1.4.4 Adc_PrescaleType

```
typedef uint8 Adc_PrescaleType
```

clock prescaler factor

Definition at line 271 of file Adc_Types.h.

6.1.4.5 Adc_SamplingTimeType

```
typedef uint8 Adc_SamplingTimeType
```

sampling time

Definition at line 275 of file Adc_Types.h.

6.1.4.6 Adc_StreamNumSampleType

```
typedef uint16 Adc_StreamNumSampleType
```

Number of samples of a streaming conversion buffer.

Definition at line 279 of file Adc_Types.h.

6.1.5 Enum Reference

6.1.5.1 Adc_GlobalStateType

```
enum Adc_GlobalStateType
```

ADC driver status.

Used to differentiate if ADC driver is already uninit, during init or already initialized or not.

Enumerator

ADC_STATE_UNINIT	Adc driver uninitialized.
ADC_STATE_BUSY	Adc driver busy.
ADC_STATE_IDLE	Adc driver idle.

Definition at line 120 of file Adc_Types.h.

6.1.5.2 Adc_GroupConversionStateType

enum Adc_GroupConversionStateType

ADC group already converted type.

Used to differentiate if group is already converted or not.

Enumerator

ADC_NOT_YET_CONVERTED	Group not yet converted.
ADC_ALREADY_CONVERTED	Group is already converted.

Definition at line 133 of file Adc_Types.h.

6.1.5.3 Adc_GroupAccessModeType

enum Adc_GroupAccessModeType

Adc group access Mode.

Used for value received by Tressos interface configuration.

SWS_Adc_00528

Enumerator

ADC_ACCESS_MODE_SINGLE	Single access mode.
ADC_ACCESS_MODE_STREAMING	Streaming access mode.

Definition at line 146 of file Adc_Types.h.

6.1.5.4 Adc_GroupReplacementType

enum Adc_GroupReplacementType

Adc group replacement.

Used for value received by Tressos interface configuration.

SWS_Adc_00523

Enumerator

ADC_GROUP_REPL_ABORT_RESTART	Abort and restart of group.
ADC_GROUP_REPL_SUSPEND_RESUME	Suspend and resuming of group.

Definition at line 159 of file Adc_Types.h.

6.1.5.5 Adc_StreamBufferModeType

enum Adc_StreamBufferModeType

Adc group streaming buffer mode.

Used for value received by Tressos interface configuration.

SWS_Adc_00519

Enumerator

ADC_STREAM_BUFFER_LINEAR	Linear streaming.
ADC_STREAM_BUFFER_CIRCULAR	Circular streaming.

Definition at line 172 of file Adc_Types.h.

6.1.5.6 Adc_StatusType

enum Adc_StatusType

ADC group status.

ADC group enumeration type.

SWS_Adc_00513

Enumerator

ADC_IDLE	Group is in IDLE state.
ADC_BUSY	Group is in BUSY state.
ADC_COMPLETED	Group is in COMPLETED state.
ADC_STREAM_COMPLETED	Group is in STREAM_COMPLETED state.

Definition at line 185 of file Adc_Types.h.

6.1.5.7 Adc_NotificationType

enum Adc_NotificationType

ADC group notification.

Indicates if notification is enabled for the group.

Enumerator

ADC_NOTIFICATION_DISABLED	Notification is disabled.
ADC_NOTIFICATION_ENABLED	Notification is enabled.

Definition at line 198 of file Adc_Types.h.

6.1.5.8 Adc_HwTriggerSignalType

enum Adc_HwTriggerSignalType

Adc hardware trigger edge.

Used for value received by Tressos interface configuration.

SWS_Adc_00520

Enumerator

ADC_HW_TRIG_RISING_EDGE	Rising edge.
ADC_HW_TRIG_FALLING_EDGE	Falling edge.
ADC_HW_TRIG_BOTH_EDGES	falling and rising edge

Definition at line 212 of file Adc_Types.h.

6.1.5.9 Adc_TriggerSourceType

```
enum Adc_TriggerSourceType
```

Adc hardware trigger source.

Used for value received by Tressos interface configuration.

SWS_Adc_00514

Enumerator

ADC_TRIGG_SRC_SW	Software triggered.
ADC_TRIGG_SRC_HW	Hardware triggered.

Definition at line 226 of file Adc_Types.h.

6.1.5.10 Adc_HwTriggeringType

```
enum Adc_HwTriggeringType
```

Adc Hardware trigger.

Indicates if hardware trigger is enabled for group.

Enumerator

ADC_HWTRIGGER_DISABLED	Hardware trigger is disabled.
ADC_HWTRIGGER_ENABLED	Hardware trigger is enabled.

Definition at line 241 of file Adc_Types.h.

6.1.6 Function Reference

6.1.6.1 Adc_Init()

```
void Adc_Init (
            const Adc_ConfigType * ConfigPtr )
```

Initializes the ADC hardware unit and the driver.

This function will initialize both the ADC HW unit and the driver structures.

Parameters

in	*ConfigPtr*	Pointer to configuration set in Variant PB (Variant PC requires a NULL_PTR).

Returns

void

6.1.6.2 Adc_SetupResultBuffer()

```
Std_ReturnType Adc_SetupResultBuffer (
            Adc_GroupType Group,
            Adc_ValueGroupType *const DataBufferPtr )
```

Initializes the group specific ADC result buffer pointer as configured to point to the pDataBufferPtr address which is passed as parameter.

Initializes ADC driver with the group specific result buffer start address where the conversion results will be stored. The application has to ensure that the application buffer, where pDataBufferPtr points to, can hold all the conversion results of the specified group. The initialization with Adc_SetupResultBuffer is required after reset, before a group conversion can be started.

Parameters

in	*Group*	Numeric ID of requested ADC channel group.
in	*pDataBufferPtr*	Pointer to result data buffer

Returns

Std_ReturnType Standard return type. E_OK: Result buffer pointer initialized correctly. E_NOT_OK: Operation failed or development error occurred.

6.1.6.3 Adc_DeInit()

```
void Adc_DeInit (
            void  )
```

Returns all ADC HW Units to a state comparable to their power on reset state.

Returns all ADC HW Units to a state comparable to their power on reset state, and de-initialize the ADC driver.

Returns

void

6.1.6.4 Adc_StartGroupConversion()

```
void Adc_StartGroupConversion (
            Adc_GroupType Group )
```

Starts the conversion of all channels of the requested ADC Channel group.

This function will start the SW conversion of all channels of the requested ADC channel group.

Parameters

in	*Group*	Numeric ID of requested ADC channel group.

Returns

void

6.1.6.5 Adc_StopGroupConversion()

```
void Adc_StopGroupConversion (
            Adc_GroupType Group )
```

Stops the conversion of all channels of the requested ADC Channel group.

This function will stop the SW conversion of all channels of the requested ADC channel group.

Parameters

in	*Group*	Numeric ID of requested ADC channel group.

Returns

void

6.1.6.6 Adc_ReadGroup()

```
Std_ReturnType Adc_ReadGroup (
            Adc_GroupType Group,
            Adc_ValueGroupType * DataBufferPtr )
```

Reads the group conversion results.

Reads the group conversion results of the last completed conversion round of the requested group and stores the channel values starting at the pDataBufferPtr address. The group channel values are stored in ascending channel number order (in contrast to the storage layout of the result buffer if streaming access is configured).

Parameters

in	*Group*	Numeric ID of requested ADC Channel group.
in	*pDataBufferPtr*	ADC results of all channels of the selected group are stored in the data buffer addressed with the pointer.

Returns

Std_ReturnType Standard return type. E_OK: results are available and written to the data buffer. E_NO↩ T_OK: no results are available or development error occurred.

6.1.6.7 Adc_ReadRawData()

```
void Adc_ReadRawData (
            Adc_HwUnitType Unit,
            const Adc_ChannelType *const ChansArray,
            uint8 NumItems,
            Adc_ValueGroupType *const DataBufferPtr )
```

Read the raw result data from an ADC unit.

Read the raw result data from an ADC unit. Intended for reading ADC results directly from ADC registers and can eliminate surplus interrupts if there are more triggered measurements than FIFO length. Measured values remain in ADC result registers(user must ensure that they are not overwritten).

Parameters

in	*Unit*	Adc unit used. Recommended to use generated define for Adc Logical Unit Id.
in	*ChansArray*	List of channels for which results to be read
in	*NumItems*	Number of results to read
out	*DataBufferPtr*	Destination pointer in which the results will be written

Returns

void

6.1.6.8 Adc_EnableHardwareTrigger()

```
void Adc_EnableHardwareTrigger (
            Adc_GroupType Group )
```

Enables the hardware trigger for the requested ADC Channel group.

This function will enable the HW trigger source for the requested ADC channel group. This function does set the CTU register for all platform that have the CTU Hw Unit.

Parameters

in	*Group*	Numeric ID of requested ADC channel group.

Returns

void

6.1.6.9 Adc_DisableHardwareTrigger()

```
void Adc_DisableHardwareTrigger (
            Adc_GroupType Group )
```

Disables the hardware trigger for the requested ADC Channel group.

This function will disable the HW trigger source for the requested ADC channel group.

Parameters

in	*Group*	Numeric ID of requested ADC channel group.

Returns

void

6.1.6.10 Adc_EnableGroupNotification()

```
void Adc_EnableGroupNotification (
            Adc_GroupType Group )
```

Enables the notification mechanism for the requested ADC channel group.

This function will enable the notification mechanism only for the requested ADC channel group.

Parameters

in	*Group*	Numeric ID of requested ADC channel group.

Returns

void

6.1.6.11 Adc_DisableGroupNotification()

```
void Adc_DisableGroupNotification (
            Adc_GroupType Group )
```

Disables the notification mechanism for the requested ADC channel group.

This function will disable the notification mechanism only for the requested ADC channel group.

Parameters

in	*Group*	Numeric ID of requested ADC channel group.

Returns

void

6.1.6.12 Adc_GetGroupStatus()

```
Adc_StatusType Adc_GetGroupStatus (
            Adc_GroupType Group )
```

Returns the conversion status of the requested ADC Channel group.

This function will return the conversion status of the requested ADC channel group.

Parameters

in	*Group*	Numeric ID of requested ADC channel group.

Returns

Adc_StatusType Conversion status for the requested group. ADC_IDLE in case of errors. conversion status in case of no errors.

6.1.6.13 Adc_GetStreamLastPointer()

```
Adc_StreamNumSampleType Adc_GetStreamLastPointer (
            Adc_GroupType Group,
            Adc_ValueGroupType ** PtrToSamplePtr )
```

Returns the number of valid samples per channel.

Returns the number of valid samples per channel, stored in the result buffer. Reads a pointer, pointing to a position in the group result buffer. With the pointer position, the results of all group channels of the last completed conversion round can be accessed. With the pointer and the return value, all valid group conversion results can be accessed (the user has to take the layout of the result buffer into account).

Parameters

in	*Group*	Numeric ID of requested ADC channel group.
out	*PtrToSamplePtr*	Pointer to result buffer pointer.

Returns

Adc_StreamNumSampleType Number of valid samples per channel. 0 in case of errors. >0 Number of valid samples per channel.

6.1.6.14 Adc_GetVersionInfo()

```
void Adc_GetVersionInfo (
            Std_VersionInfoType * versioninfo )
```

Returns the version information of this module.

Returns the version information of this module.

Parameters

out	*pVersionInfo*	Pointer to where to store the version information of this module. structure in case of no errors.

6.1.6.15 Adc_SetHwUnitPowerMode()

```
Std_ReturnType Adc_SetHwUnitPowerMode (
            Adc_HwUnitType Unit,
            Adc_SetPowerModeType SetPowerMode )
```

Set the ADC mode either to powerdown or normal.

Set the ADC either to powerdown or normal mode.

Parameters

in	*Unit*	Adc unit used. Recommended to use generated define for Adc Logical Unit Id
in	*SetPowerMode*	Power mode to set: normal or powerdown.

Returns

Std_ReturnType Standard return type. E_OK: Transition successful. E_NOT_OK: Transition unsuccessful.

6.1.6.16 Adc_CtuSetPowerMode()

```
Std_ReturnType Adc_CtuSetPowerMode (
            Adc_HwUnitType CtuUnit,
            Adc_PowerStateType State )
```

Function to set BCTU/CTU power mode.

Set BCTU/CTU power mode.

Parameters

in	*CtuUnit*	Bctu/Ctu hardware unit. Recommended to use generated define for Bctu/Ctu Logical Unit Id.
in	*State*	Power state to be set

Returns

Std_ReturnType Standard return type. E_OK: Transition successful. E_NOT_OK: Transition unsuccessful.

6.1.6.17 Adc_EnableCTUTrigger()

```
void Adc_EnableCTUTrigger (
            Adc_GroupType Group,
            Adc_HwTriggerTimerType TriggerSource )
```

Enable the TriggerSource for group selected by Group parameter.

This Autosar Extension API is used to enable any one of the configured TriggerSource of the Group. When this Autosar Extension API is used to enable the trigger source the CTU interrupt will be disabled by the driver. So user has to call the Autosar Extension API Adc_HwResultReadGroup to read the converted result from the ADC hardware register.

Parameters

in	*Group*	Index of group.
in	*TriggerSource*	Trigger source to be used for the group. (Configuration file should contain it for that group).

Returns

void

6.1.6.18 Adc_DisableCTUTrigger()

```
void Adc_DisableCTUTrigger (
            Adc_GroupType Group,
            Adc_HwTriggerTimerType TriggerSource )
```

Disable the TriggerSource for group selected by Group parameter.

This Autosar Extension API is used to disable the already enabled TriggerSource of the Group.

Parameters

in	*Group*	Index of group.
in	*TriggerSource*	Trigger source to be disabled for the group. (Configuration file should contain it for that group).

Returns

void

6.1.6.19 Adc_HwResultReadGroup()

```
Std_ReturnType Adc_HwResultReadGroup (
            Adc_GroupType Group,
            Adc_ValueGroupType * DataPtr )
```

Read the result of the hardware triggered groups conversion result.

This Autosar Extension API is used to read the result of the hardware triggered groups conversion result from the ADC hardware register in this case the CTU interrupt will be disabled for the group. The VALID bit C↩ DR register will be cleared automatically when we read the results from the channel data register. If the user calls Autosar Extension function Adc_HwResultReadGroup() once again before the next conversion takes place, the Adc_HwResultReadGroup() returns E_NOT_OK.

Parameters

in	*Group*	Index of group.
in	*DataPtr*	Pointer to a buffer which will be filled by the conversion results.

Returns

Std_ReturnType Standard return type. E_OK: results are available and written to the data buffer. E_NO↩ T_OK: no results are available or development error occurred.

6.1.6.20 Adc_GetInjectedConversionStatus()

```
Adc_StatusType Adc_GetInjectedConversionStatus (
            Adc_HwUnitType Unit )
```

Get the injected conversions status.

This function checks if an injected conversion (HW,SW) is ongoing

Parameters

in	*Unit*	Adc unit used. Recommended to use generated define for Adc Logical Unit Id

Returns

Adc_StatusType Status of the ADC HW unit. ADC_IDLE: SW,HW Injection or Hardware Trigger group are idle. ADC_BUSY: SW,HW Injection or Hardware Trigger is in progress.

6.1.6.21 Adc_Calibrate()

```
void Adc_Calibrate (
            Adc_HwUnitType Unit,
            Adc_CalibrationStatusType * pStatus )
```

Executes high accuracy calibration of a ADC HW unit.

This function calibrates the ADC HW unit and updates calibration related registers

Parameters

in	*Unit*	Adc unit used. Recommended to use generated define for Adc Logical Unit Id
in	*pStatus*	Status of the ADC HW unit calibration and list of failed and passed tests.

Returns

void

6.1.6.22 Adc_SelfTest()

```
Std_ReturnType Adc_SelfTest (
            Adc_HwUnitType Unit )
```

Executes hardware Self Test of a ADC HW unit.

This function checks if the ADC HW unit is functioning correctly

Parameters

in	*Unit*	Adc unit used. Recommended to use generated define for Adc Logical Unit Id

Returns

Std_ReturnType. Status of the ADC HW unit Self Test.

6.1.6.23 Adc_ConfigureThreshold()

```
void Adc_ConfigureThreshold (
            Adc_ThresholdControlIndexType ThresholdControlIndex,
            Adc_WdgThresholdValueType LowValue,
            Adc_WdgThresholdValueType HighValue )
```

Function to reconfigure High and Low thresholds for a given threshold control index.

This function is used to reconfigure High and Low thresholds for a given threshold control index. Use generated symbolic name defines of threshold register (e.g. AdcThresholdControl_0_0), because function assumes Threshold↩ ControlIndex to be in following format: Logical threshold id on bits until position defined by ADC_THRESHOL↩ D_SYMBOLIC_NAME_SHIFT_HW_UNIT_ID_U16, and for the rest the Logical Unit Id

Parameters

in	*ThresholdControlIndex*	Symbolic name of threshold control
in	*LowValue*	Low threshold value of the threshold control
in	*HighValue*	High threshold value of the threshold control

Returns

void

6.1.6.24 Adc_EnableWdgNotification()

```
void Adc_EnableWdgNotification (
            Adc_ChannelType ChannelId )
```

Enable notification of a channel that has watchdog functionality configured at initialization.

Enable notification of a channel that has watchdog functionality configured at initialization Use generated symbolic channel name defines (e.g. AdcChannel_0_0), because function assumes ChannelId to be in following format: Logical channel id on bits until position defined by ADC_CHANNEL_SYMBOLIC_NAME_SHIFT_HW_UNIT_ID_↩ U16, and for the rest the Logical Unit Id

Parameters

in	*Adc_ChannelType*	Symbolic name of channel

Returns

void.

6.1.6.25 Adc_DisableWdgNotification()

```
void Adc_DisableWdgNotification (
            Adc_ChannelType ChannelId )
```

Disable notification of a channel that has watchdog functionality configured at initialization.

Disable notification of a channel that has watchdog functionality configured at initialization Use generated symbolic channel name defines (e.g. AdcChannel_0_0), because function assumes ChannelId to be in following format: Logical channel id on bits until position defined by ADC_CHANNEL_SYMBOLIC_NAME_SHIFT_HW_UNIT_ID_↩ U16, and for the rest the Logical Unit Id

Parameters

in	*Adc_ChannelType*	Symbolic name of channel

Returns

void.

6.1.6.26 Adc_SetClockMode()

```
Std_ReturnType Adc_SetClockMode (
            Adc_SelectPrescalerType Prescaler )
```

Set the ADC clock prescaler if available and modify the conversion timings.

This function sets the ADC clock prescaler (Analog clock frequency selector)

Parameters

in	*Prescaler*	Normal or Alternate mode.

Returns

Std_ReturnType Standard return type. E_OK: In case of successful settings. E_NOT_OK: In case of unsuccessful settings.

6.1.6.27 Adc_SetPowerState()

```
Std_ReturnType Adc_SetPowerState (
            Adc_PowerStateRequestResultType * Result )
```

Enters the already prepared power state.

This API configures the Adc module so that it enters the already prepared power state, chosen between a predefined set of configured ones.

Parameters

out	*Result*	Pointer to a variable to store the result of this function

Returns

Std_ReturnType Standard return type. E_OK: Power Mode changed. E_NOT_OK: Request rejected.

6.1.6.28 Adc_GetCurrentPowerState()

```
Std_ReturnType Adc_GetCurrentPowerState (
            Adc_PowerStateType * CurrentPowerState,
            Adc_PowerStateRequestResultType * Result )
```

Get the current power state of the ADC HW unit.

This API returns the current power state of the ADC HW unit.

Parameters

out	*CurrentPowerState*	The current power mode of the ADC HW Unit is returned in this parameter
out	*Result*	Pointer to a variable to store the result of this function

Returns

Std_ReturnType Standard return type. E_OK: Mode could be read. E_NOT_OK: Service is rejected.

6.1.6.29 Adc_GetTargetPowerState()

```
Std_ReturnType Adc_GetTargetPowerState (
            Adc_PowerStateType * TargetPowerState,
            Adc_PowerStateRequestResultType * Result )
```

Get the target power state of the ADC HW unit.

This API returns the target power state of the ADC HW unit.

Parameters

out	*TargetPowerState*	The Target power mode of the ADC HW Unit is returned in this parameter.
out	*Result*	Pointer to a variable to store the result of this function.

Returns

Std_ReturnType Standard return type. E_OK: Mode could be read. E_NOT_OK: Service is rejected.

6.1.6.30 Adc_PreparePowerState()

```
Std_ReturnType Adc_PreparePowerState (
            Adc_PowerStateType PowerState,
            Adc_PowerStateRequestResultType * Result )
```

Starts the needed process to allow the ADC HW module to enter the requested power state.

This API starts the needed process to allow the ADC HW module to enter the requested power state.

Parameters

in	*PowerState*	The target power state intended to be attained.
out	*Result*	Pointer to a variable to store the result of this function.

Returns

Std_ReturnType Standard return type. E_OK: Mode could be read. E_NOT_OK: Service is rejected.

6.1.6.31 Adc_EnableCtuControlMode()

```
void Adc_EnableCtuControlMode (
            Adc_HwUnitType Unit )
```

Function to enable CTU control mode for an ADC unit.

Enable CTU control mode for an ADC unit. This function to enable CTU control mode for Adc. When a unit works in CTU control mode, no other conversions shall run in parallel(Adc). The only conversions occurring shall be the ones defined in the CTU configuration.

Parameters

in	*Unit*	Adc unit used. Recommended to use generated define for Adc Logical Unit Id

Returns

void

6.1.6.32 Adc_DisableCtuControlMode()

```
void Adc_DisableCtuControlMode (
            Adc_HwUnitType Unit )
```

Function to disable CTU control mode for an ADC unit.

Disable CTU control mode for an ADC unit. This function to disable CTU control mode for Adc. The other Adc conversions can run in software trigger normal mode, software trigger injected mode or hardware trigger mode.

Parameters

in	*Unit*	Adc unit used. Recommended to use generated define for Adc Logical Unit Id

Returns

void

6.1.6.33 Adc_CtuEnableHwTrigger()

```
void Adc_CtuEnableHwTrigger (
            Adc_CtuTrigSrcType TrigSource )
```

Function to enable CTU hardware trigger.

Enable CTU hardware trigger.

Parameters

in	*TrigSource*	Trigger source.

Returns

void

6.1.6.34 Adc_CtuDisableHwTrigger()

```
void Adc_CtuDisableHwTrigger (
            Adc_CtuTrigSrcType TrigSource )
```

Function to disable CTU hardware trigger.

Disable CTU hardware trigger.

Parameters

in	*TrigSource*	Trigger source.

Returns

void

6.1.6.35 Adc_CtuStartConversion()

```
void Adc_CtuStartConversion (
            Adc_CtuTrigSrcType TrigSource )
```

Function to start CTU conversion.

Start CTU conversion.

Parameters

in	*TrigSource*	Trigger source.

Returns

void

6.1.6.36 Adc_CtuReadConvData()

```
Adc_ValueGroupType Adc_CtuReadConvData (
            Adc_HwUnitType AdcUnit )
```

Function to read BCTU conversion data.

Read CTU conversion data.

Parameters

in	*AdcUnit*	Adc HW unit. Recommended to use generated define for Adc Logical Unit Id.

Returns

Adc_ValueGroupType Conversion result.

6.1.6.37 Adc_CtuReadConvResult()

```
void Adc_CtuReadConvResult (
            Adc_HwUnitType AdcUnit,
            Adc_CtuResultType * pResult )
```

Function to read BCTU conversion result.

Read BCTU conversion result.

Parameters

in	*AdcUnit*	Adc HW unit. Recommended to use generated define for Adc Logical Unit Id.
out	*pResult*	Adc result structure.

Returns

void

6.1.6.38 Adc_CtuStopLoopConversions()

```
void Adc_CtuStopLoopConversions (
            Adc_CtuTrigSrcType TrigSource )
```

Function to stop CTU loop conversion.

Stop CTU loop conversions.

Parameters

in	*TrigSource*	Trigger source of conversions to be stopped.

Returns

void

6.1.6.39 Adc_CtuReadFifoData()

```
void Adc_CtuReadFifoData (
            Adc_CtuFifoIdxType FifoIdx,
```

```
uint16 * Data,
uint8 DataLength )
```

Function to read CTU conversion data from FIFO.

Read CTU conversion data from FIFO.

Parameters

in	*FifoIdx*	CTU FIFO index.
out	*Data*	Pointer to pre-allocated result array.
in	*DataLength*	Max amount of results to be read.

Returns

void

6.1.6.40 Adc_CtuReadFifoResult()

```
void Adc_CtuReadFifoResult (
            Adc_CtuFifoIdxType FifoIdx,
            Adc_CtuFifoResultType * Result,
            uint8 ResultLength )
```

Function to read CTU conversion results from FIFO.

Read CTU conversion results from FIFO.

Parameters

in	*FifoIdx*	CTU FIFO index.
out	*Result*	Pointer to pre-allocated result array.
in	*ResultLength*	Max amount of results to be read.

Returns

void

6.1.6.41 Adc_CtuSetFifoWatermark()

```
void Adc_CtuSetFifoWatermark (
            Adc_CtuFifoIdxType FifoIdx,
            uint8 Watermark )
```

Function to set CTU FIFO watermark.

Set CTU FIFO watermark.

Parameters

in	*FifoIdx*	FIFO index.
in	*Watermark*	Watermark value.

Returns

void

6.1.6.42 Adc_CtuEnableNotification()

```
void Adc_CtuEnableNotification (
            Adc_CtuNotificationType Notification )
```

Function to enable CTU notification.

Enable CTU notification.

Parameters

in	*Notification*	Notification to be enabled.

Returns

void

6.1.6.43 Adc_CtuDisableNotification()

```
void Adc_CtuDisableNotification (
            Adc_CtuNotificationType Notification )
```

Function to disable CTU notification.

Disable CTU notification.

Parameters

in	*Notification*	Notification to be disabled.

Returns

void

6.1.6.44 Adc_CtuSetList()

```
void Adc_CtuSetList (
            const Adc_CtuListItemType *const ListItemsArray,
            const uint8 NumItems,
            const uint8 ListStartPosition )
```

Function to set CTU list.

Reconfigure the CTU list of conversions in CTU Control Mode.

Parameters

in	*ListItemsArray*	Pointer to list items array to be set.
in	*NumItems*	Number of items in the array.
in	*ListStartPosition*	Start position of the list.

Returns

void

6.1.6.45 Adc_CtuSetListPointer()

```
void Adc_CtuSetListPointer (
            Adc_CtuTrigSrcType TrigSource,
            Adc_CtuListPtrType ListPtr )
```

Reconfigure the list pointer of a triggered CTU source.

Reconfigure the list pointer of a trigger source in CTU Control Mode.

Parameters

in	*TrigSource*	Source trigger index.
in	*ListPtr*	Position of the first channel of the command list.

Returns

void

6.1.6.46 Adc_TempSenseCalculateTemp()

```
uint16 Adc_TempSenseCalculateTemp (
            Adc_HwUnitType Unit,
            const uint16 TempSenseAdcConvData )
```

Function to calculate temperature on chip from provided data.

Calculates temperature on chip from provided data.

Parameters

in	*Unit*	Adc unit used. Recommended to use generated define for Adc Logical Unit Id.
in	*TempSenseConvData*	Data measured on the ADC internal channel for TempSense. (1 bit for the sign, 11 bits for the integer part and 4 bits for the decimal part)

Returns

uint16 Temperature value on chip.in degrees C, expressed in fixed point format.

6.1.6.47 Adc_TempSenseGetTemp()

```
Std_ReturnType Adc_TempSenseGetTemp (
            Adc_HwUnitType Unit,
            uint16 *const TempSenseVal )
```

Function to get the temperature on chip directly.

This function starts a normal software conversion with one-shot mode on tempsense channel and calculates the temperature on chip from the data conversion The function is synchronous: waits until the ADC conversion completes or timeout occurs

Parameters

in	*Unit*	Adc unit used. Recommended to use generated define for Adc Logical Unit Id.
out	*TempSenseVal*	Temperature value on chip in degrees C, expressed in fixed point format. (1 bit for the sign, 11 bits for the integer part and 4 bits for the decimal part)

Returns

Std_ReturnType Standard return type. E_OK: temperature read successful E_NOT_OK: operation failed

6.1.6.48 Adc_TempSenseSetPowerMode()

```
Std_ReturnType Adc_TempSenseSetPowerMode (
            Adc_SetPowerModeType SetPowerMode )
```

Set the Tempsense mode either to powerdown or normal.

Set the Tempsense mode either to powerdown or normal.

Parameters

in	*SetPowerMode*	Power mode to set: normal or powerdown.

Returns

Std_ReturnType Successful/Unsuccessful transition.

6.1.7 Variable Documentation

6.1.7.1 Adc_apxCfgPtr

```
const Adc_ConfigType* Adc_apxCfgPtr[(1U)]  [extern]
```

Used to point the configuration structure.

6.2 Adc Sar IPL

6.2.1 Detailed Description Adc Sar HW module.

Adc Sar IP layer hardware module.

Data Structures

- struct Adc_Sar_Ip_ChanConfigType

  Defines the channel configuration. More...
- struct Adc_Sar_Ip_ClockConfigType

  Defines the ADC clock configuration. More...
- struct Adc_Sar_Ip_WdgThresholdType

  Defines the upper and lower thresholds for analog watchdog. More...
- struct Adc_Sar_Ip_ChanResultType

  Defines the data regarding a conversion, beyond the conversion data. More...
- struct Adc_Sar_Ip_ChansIdxMaskType

  Defines configuration of channels in a chain. More...
- struct Adc_Sar_Ip_ConfigType

  Defines the module configuration. More...
- struct Adc_Sar_Ip_StateStructType

  Structure used to store runtime info. More...

Macros

- #define ADC_SAR_IP_NOTIF_FLAG_NORMAL_ENDCHAIN

  Macros for status and notification flags.
- #define ADC_SAR_IP_CHAN_NOTIF_EOC

  Macros for channel notifications.
- #define ADC_SAR_IP_WDG_HIGH_FLAG

  Macros for watchdog registers.

Types Reference

- typedef void Adc_Sar_Ip_WdgNotificationType(const uint16 ChanIdx, const uint8 Flags)

  Defines the watchdog notification header.

Enum Reference

- enum Adc_Sar_Ip_StatusType

  ADC_SAR status return type.
- enum Adc_Sar_Ip_ConvModeType

  Conversion mode selection (One-shot or Scan)
- enum Adc_Sar_Ip_ClockSelType

  Converter input clock.
- enum Adc_Sar_Ip_ExtTriggerEdgeType

  External Trigger selection.
- enum Adc_Sar_Ip_ExtTriggerSourceType

  External Trigger Source Enable.
- enum Adc_Sar_Ip_ConvChainType

  Conversion chain selection.
- enum Adc_Sar_Ip_DataAlignedType

  Data alignment selection.
- enum Adc_Sar_Ip_ClearSourceType

  Clear DMA source.
- enum Adc_Sar_Ip_PresamplingSourceType

  Presampling Voltage selection.
- enum Adc_Sar_Ip_ChanGroupType

  Channel group selection.
- enum Adc_Sar_Ip_Resolution

  Adc group conversion resolution.

Function Reference

- uint32 Adc_Sar_Ip_GetDataAddress (uint32 u32Instance, uint32 u32ChannelIndex)

  Return the address of the specified data register.
- Adc_Sar_Ip_StatusType Adc_Sar_Ip_Init (const uint32 u32Instance, const Adc_Sar_Ip_ConfigType *const pConfig)

  Initialize ADC_SAR module.
- Adc_Sar_Ip_StatusType Adc_Sar_Ip_Deinit (const uint32 u32Instance)

  Deinitialize ADC_SAR module.
- void Adc_Sar_Ip_ChainConfig (const uint32 u32Instance, const Adc_Sar_Ip_ChansIdxMaskType *const pChansIdxMask, const Adc_Sar_Ip_ConvChainType pChainType)

  Configures the converter chains the given configuration structure.
- void Adc_Sar_Ip_EnableChannel (const uint32 u32Instance, const Adc_Sar_Ip_ConvChainType pChain↩ Type, const uint32 u32ChnIdx)

  Enable a channel.
- void Adc_Sar_Ip_DisableChannel (const uint32 u32Instance, const Adc_Sar_Ip_ConvChainType pChain↩ Type, const uint32 u32ChnIdx)

  Disable a channel.
- void Adc_Sar_Ip_SetResolution (const uint32 u32Instance, const Adc_Sar_Ip_Resolution eResolution)

  Set conversion resolution.

- void Adc_Sar_Ip_StartConversion (const uint32 u32Instance, const Adc_Sar_Ip_ConvChainType pChain↩ Type)

  Start conversion.
- uint32 Adc_Sar_Ip_GetStatusFlags (const uint32 u32Instance)

  Get the status flags.
- void Adc_Sar_Ip_ClearStatusFlags (const uint32 u32Instance, const uint32 u32Mask)

  Clear the status flags.
- Adc_Sar_Ip_StatusType Adc_Sar_Ip_SelfTest (const uint32 u32Instance)

  Self test.
- uint32 Adc_Sar_Ip_GetConvDataToArray (const uint32 u32Instance, const Adc_Sar_Ip_ConvChainType pChainType, const uint32 u32Length, uint16 *const pResults)

  Get conversion results for a conversion chain.
- uint32 Adc_Sar_Ip_GetConvResultsToArray (const uint32 u32Instance, const Adc_Sar_Ip_ConvChainType pChainType, const uint32 u32Length, Adc_Sar_Ip_ChanResultType *const pResults)

  Get conversion results for a conversion chain with extended information.
- uint16 Adc_Sar_Ip_GetConvData (const uint32 u32Instance, const uint32 u32ChnIdx)

  Return the result of the conversion.
- void Adc_Sar_Ip_GetConvResult (const uint32 u32Instance, const uint32 u32ChnIdx, const Adc_Sar_Ip_ConvChainType pChainType, Adc_Sar_Ip_ChanResultType *const pResult)

  Return the result and the status of the conversion.
- Adc_Sar_Ip_StatusType Adc_Sar_Ip_DoCalibration (const uint32 u32Instance)

  Perform Calibration of the ADC.
- Adc_Sar_Ip_StatusType Adc_Sar_Ip_Powerup (const uint32 u32Instance)

  Power up the ADC.
- Adc_Sar_Ip_StatusType Adc_Sar_Ip_Powerdown (const uint32 u32Instance)

  Power down the ADC.
- void Adc_Sar_Ip_EnableNotifications (const uint32 u32Instance, const uint32 u32NotificationMask)

  Enable ADC interrupts.
- void Adc_Sar_Ip_DisableNotifications (const uint32 u32Instance, const uint32 u32NotificationMask)

  Disable ADC interrupts.
- void Adc_Sar_Ip_EnableChannelNotifications (const uint32 u32Instance, const uint32 u32ChnIdx, const uint32 u32Mask)

  Enable ADC interrupt for a channel.
- void Adc_Sar_Ip_DisableChannelNotifications (const uint32 u32Instance, const uint32 u32ChnIdx, const uint32 u32Mask)

  Disable ADC interrupt for a channel.
- Adc_Sar_Ip_StatusType Adc_Sar_Ip_SetClockMode (const uint32 u32Instance, const Adc_Sar_Ip_ClockConfigType *const pConfig)

  Set the ADC clocks.
- void Adc_Sar_Ip_SetSampleTimes (const uint32 u32Instance, const uint8 *const aSampleTimes)

  Set the sample times.
- void Adc_Sar_Ip_AbortConversion (const uint32 u32Instance)

  Abort ongoing conversion.
- Adc_Sar_Ip_StatusType Adc_Sar_Ip_AbortChain (const uint32 u32Instance, const boolean bBlocking, const boolean bAllowRestart)

  Abort ongoing chain conversion.
- void Adc_Sar_Ip_SetPresamplingSource (const uint32 u32Instance, const Adc_Sar_Ip_ChanGroupType p↩ ChanGroup, const Adc_Sar_Ip_PresamplingSourceType pPresampleSource)

Set the Presampling Source for the channel group.

- void Adc_Sar_Ip_EnableChannelPresampling (const uint32 u32Instance, const uint32 u32ChnIdx)
  Enable Presampling on one channel.
- void Adc_Sar_Ip_DisableChannelPresampling (const uint32 u32Instance, const uint32 u32ChnIdx)
  Disable Presampling on one channel.
- void Adc_Sar_Ip_EnablePresampleConversion (const uint32 u32Instance)
  Enable Conversion Presampled Data.
- void Adc_Sar_Ip_DisablePresampleConversion (const uint32 u32Instance)
  Disable Conversion of Presampled Data.
- void Adc_Sar_Ip_EnableDma (const uint32 u32Instance)
  Enable DMA Requests.
- void Adc_Sar_Ip_DisableDma (const uint32 u32Instance)
  Disable DMA Requests.
- void Adc_Sar_Ip_EnableChannelDma (const uint32 u32Instance, const uint32 u32ChnIdx)
  Enable DMA on one channel.
- void Adc_Sar_Ip_DisableChannelDma (const uint32 u32Instance, const uint32 u32ChnIdx)
  Disable DMA on one channel.
- void Adc_Sar_Ip_DisableChannelDmaAll (const uint32 u32Instance)
  Disable DMA on all channels.
- void Adc_Sar_Ip_SetDmaClearSource (const uint32 u32Instance, const Adc_Sar_Ip_ClearSourceType pDmaClear)
  Set DMA Request Clear Source.
- void Adc_Sar_Ip_SetWdgThreshold (const uint32 u32Instance, const uint8 u8RegisterIdx, const Adc_Sar_Ip_WdgThresholdType *const pThresholdValues)
  Configure watchdog threshold register.
- void Adc_Sar_Ip_SetConversionMode (const uint32 u32Instance, const Adc_Sar_Ip_ConvModeType eConvMode)
  Configure conversion mode.
- void Adc_Sar_Ip_SetExternalTrigger (const uint32 u32Instance, const Adc_Sar_Ip_ExtTriggerEdgeType eTriggerEdge, const Adc_Sar_Ip_ExtTriggerSourceType eTrggerSrc)
  Configure external trigger.

6.2.2 Data Structure Documentation

6.2.2.1 struct Adc_Sar_Ip_ChanConfigType

Defines the channel configuration.

This structure is used to configure channels

Implements : Adc_Sar_Ip_ChanConfigType_Class

Definition at line 325 of file Adc_Sar_Ip_Types.h.

6.2.2.2 struct Adc_Sar_Ip_ClockConfigType

Defines the ADC clock configuration.

This structure is used to the ADC_SAR clock

Implements : Adc_Sar_Ip_ClockConfigType_Class

Definition at line 346 of file Adc_Sar_Ip_Types.h.

Data Fields

Type	Name	Description
Adc_Sar_Ip_ClockSelType	ClkSelect	Selected clock
boolean	HighSpeedConvEn	
uint8	SampleTimeArr[ADC_SAR_IP_NUM_GROUP_CHAN]	Sample time for each channel group
uint8	PowerDownDelay	Delay before entering Power Down

6.2.2.3 struct Adc_Sar_Ip_WdgThresholdType

Defines the upper and lower thresholds for analog watchdog.

This structure is used to configure the analog watchdog threshold registers.

Implements : Adc_Sar_Ip_WdgThresholdType_Class

Definition at line 373 of file Adc_Sar_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	WdgIndex	Watchdog threshold register index
uint16	LowThreshold	Lower threshold
uint16	HighThreshold	Upper threshold
boolean	LowThresholdIntEn	Enable interrupt when lower threshold exceeded
boolean	HighThresholdIntEn	Enable interrupt when upper threshold exceeded

6.2.2.4 struct Adc_Sar_Ip_ChanResultType

Defines the data regarding a conversion, beyond the conversion data.

This structure is used to return information about conversions beyond just conversion data

Implements : Adc_Sar_Ip_ChanResultType_Class

Definition at line 390 of file Adc_Sar_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	AdcChnIdx	ADC Channel Index
boolean	ValidFlag	Data Valid Flag
boolean	OverWrittenFlag	Data Overwritten Flag
uint16	ConvData	Conversion Data

6.2.2.5 struct Adc_Sar_Ip_ChansIdxMaskType

Defines configuration of channels in a chain.

This structure is used to configure channels in chain.

Implements : Adc_Sar_Ip_ChansIdxMaskType_Class

Definition at line 404 of file Adc_Sar_Ip_Types.h.

Data Fields

Type	Name	Description
uint32	ChanMaskArr[ADC_SAR_IP_NUM_GROUP_CHAN]	Bit-mask used to configure channels in chain

6.2.2.6 struct Adc_Sar_Ip_ConfigType

Defines the module configuration.

This structure is used to configure the ADC_SAR module

Implements : Adc_Sar_Ip_ConfigType_Class

Definition at line 416 of file Adc_Sar_Ip_Types.h.

Data Fields

- Adc_Sar_Ip_ConvModeType ConvMode
- Adc_Sar_Ip_Resolution AdcResolution
- boolean BypassResolution
- Adc_Sar_Ip_ClockSelType ClkSelect
- Adc_Sar_Ip_ClockSelType CalibrationClkSelect
- Adc_Sar_Ip_ExtTriggerEdgeType InjectedEdge
- Adc_Sar_Ip_ExtTriggerEdgeType ExtTrigger
- boolean NormalExtTrgEn
- boolean NormalAuxExtTrgEn
- uint8 SampleTimeArr [ADC_SAR_IP_NUM_GROUP_CHAN]
- Adc_Sar_Ip_PresamplingSourceType PresamplingSourceArr [ADC_SAR_IP_NUM_GROUP_CHAN]
- boolean AutoClockOff
- boolean OverwriteEnable
- Adc_Sar_Ip_DataAlignedType DataAlign
- uint8 PowerDownDelay
- uint32 ChanMaskNormal [ADC_SAR_IP_NUM_GROUP_CHAN]
- uint32 ChanMaskInjected [ADC_SAR_IP_NUM_GROUP_CHAN]

6.2.2.6.1 Field Documentation

6.2.2.6.1.1 ConvMode Adc_Sar_Ip_ConvModeType ConvMode

Conversion Mode (One-shot or Scan)

Definition at line 418 of file Adc_Sar_Ip_Types.h.

6.2.2.6.1.2 AdcResolution Adc_Sar_Ip_Resolution AdcResolution

Adc resolution

Definition at line 420 of file Adc_Sar_Ip_Types.h.

6.2.2.6.1.3 BypassResolution boolean BypassResolution

Bypass Adc resolution processing for the result

Definition at line 421 of file Adc_Sar_Ip_Types.h.

6.2.2.6.1.4 ClkSelect Adc_Sar_Ip_ClockSelType ClkSelect

Clock input

Definition at line 423 of file Adc_Sar_Ip_Types.h.

6.2.2.6.1.5 CalibrationClkSelect Adc_Sar_Ip_ClockSelType CalibrationClkSelect

Clock input for calibration

Definition at line 424 of file Adc_Sar_Ip_Types.h.

6.2.2.6.1.6 InjectedEdge Adc_Sar_Ip_ExtTriggerEdgeType InjectedEdge

Injected Trigger selection

Definition at line 432 of file Adc_Sar_Ip_Types.h.

6.2.2.6.1.7 ExtTrigger Adc_Sar_Ip_ExtTriggerEdgeType ExtTrigger

External Trigger selection

Definition at line 435 of file Adc_Sar_Ip_Types.h.

6.2.2.6.1.8 NormalExtTrgEn boolean NormalExtTrgEn

Enables normal trigger source

Definition at line 436 of file Adc_Sar_Ip_Types.h.

6.2.2.6.1.9 NormalAuxExtTrgEn boolean NormalAuxExtTrgEn

Enables auxiliary normal trigger source

Definition at line 438 of file Adc_Sar_Ip_Types.h.

6.2.2.6.1.10 SampleTimeArr uint8 SampleTimeArr[ADC_SAR_IP_NUM_GROUP_CHAN]

Sample time for each channel group

Definition at line 442 of file Adc_Sar_Ip_Types.h.

6.2.2.6.1.11 PresamplingSourceArr Adc_Sar_Ip_PresamplingSourceType PresamplingSourceArr[ADC_SAR_I↩ P_NUM_GROUP_CHAN]

Presampling sources for each channel group

Definition at line 446 of file Adc_Sar_Ip_Types.h.

6.2.2.6.1.12 AutoClockOff boolean AutoClockOff

Enable Auto Clock Off

Definition at line 448 of file Adc_Sar_Ip_Types.h.

6.2.2.6.1.13 OverwriteEnable `boolean OverwriteEnable`

Overwrite new conversion data over old data

Definition at line 449 of file Adc_Sar_Ip_Types.h.

6.2.2.6.1.14 DataAlign `Adc_Sar_Ip_DataAlignedType DataAlign`

Data alignment in conversion result register

Definition at line 450 of file Adc_Sar_Ip_Types.h.

6.2.2.6.1.15 PowerDownDelay `uint8 PowerDownDelay`

Delay before entering Power Down

Definition at line 454 of file Adc_Sar_Ip_Types.h.

6.2.2.6.1.16 ChanMaskNormal `uint32 ChanMaskNormal[ADC_SAR_IP_NUM_GROUP_CHAN]`

Bit-mask used to configure Normal Chain

Definition at line 467 of file Adc_Sar_Ip_Types.h.

6.2.2.6.1.17 ChanMaskInjected `uint32 ChanMaskInjected[ADC_SAR_IP_NUM_GROUP_CHAN]`

Bit-mask used to configure Injected Chain

Definition at line 468 of file Adc_Sar_Ip_Types.h.

6.2.2.7 struct Adc_Sar_Ip_StateStructType

Structure used to store runtime info.

This structure is used to store ADC SAR runtime info

Implements : Adc_Sar_Ip_StateStructType_Class

Definition at line 506 of file Adc_Sar_Ip_Types.h.

Data Fields

- boolean InitStatus
- Adc_Sar_Ip_DataAlignedType DataAlign
- Adc_Sar_Ip_ClockSelType CalibrationClkSelect
- boolean BypassResolution
- uint32 ChanWdgThresholdIndex [ADC_SAR_IP_CDR_COUNT]

6.2.2.7.1 Field Documentation

6.2.2.7.1.1 InitStatus `boolean InitStatus`

Check if the driver was initialized.

Definition at line 508 of file Adc_Sar_Ip_Types.h.

6.2.2.7.1.2 DataAlign `Adc_Sar_Ip_DataAlignedType DataAlign`

Data alignment in conversion result register

Definition at line 509 of file Adc_Sar_Ip_Types.h.

6.2.2.7.1.3 CalibrationClkSelect `Adc_Sar_Ip_ClockSelType CalibrationClkSelect`

Clock input for calibration

Definition at line 510 of file Adc_Sar_Ip_Types.h.

6.2.2.7.1.4 BypassResolution `boolean BypassResolution`

Adc raw data result

Definition at line 512 of file Adc_Sar_Ip_Types.h.

6.2.2.7.1.5 ChanWdgThresholdIndex `uint32 ChanWdgThresholdIndex[ADC_SAR_IP_CDR_COUNT]`

Channel index to threshold index mapping

Definition at line 515 of file Adc_Sar_Ip_Types.h.

6.2.3 Macro Definition Documentation

6.2.3.1 ADC_SAR_IP_NOTIF_FLAG_NORMAL_ENDCHAIN

```
#define ADC_SAR_IP_NOTIF_FLAG_NORMAL_ENDCHAIN
```

Macros for status and notification flags.

These flags map to internal hardware flags in different registers, but are grouped together for convenience.

Definition at line 114 of file Adc_Sar_Ip.h.

6.2.3.2 ADC_SAR_IP_CHAN_NOTIF_EOC

```
#define ADC_SAR_IP_CHAN_NOTIF_EOC
```

Macros for channel notifications.

These notification flags map to internal hardware flags in different registers, but are grouped together for convenience.

Definition at line 158 of file Adc_Sar_Ip.h.

6.2.3.3 ADC_SAR_IP_WDG_HIGH_FLAG

```
#define ADC_SAR_IP_WDG_HIGH_FLAG
```

Macros for watchdog registers.

These macros help decode and compose bit mask for watchdog functionality

Definition at line 172 of file Adc_Sar_Ip.h.

6.2.4 Types Reference

6.2.4.1 Adc_Sar_Ip_WdgNotificationType

```
typedef void Adc_Sar_Ip_WdgNotificationType(const uint16 ChanIdx, const uint8 Flags)
```

Defines the watchdog notification header.

This header is used for out of range watchdog notification callbacks u8Flags will contain which WDG flag was triggered(low and/or high). These are defined by ADC_SAR_IP_WDG_LOW_FLAG and ADC_SAR_IP_W↩ DG_HIGH_FLAG

Implements : Adc_Sar_Ip_WdgChanNotificationType_Class

Definition at line 315 of file Adc_Sar_Ip_Types.h.

6.2.5 Enum Reference

6.2.5.1 Adc_Sar_Ip_StatusType

```
enum Adc_Sar_Ip_StatusType
```

ADC_SAR status return type.

This enum is used as return type

Implements : Adc_Sar_Ip_StatusType_Class

Enumerator

ADC_SAR_IP_STATUS_SUCCESS	Function completed successfully
ADC_SAR_IP_STATUS_ERROR	Function didn't complete successfully
ADC_SAR_IP_STATUS_TIMEOUT	Function timed out

Definition at line 102 of file Adc_Sar_Ip_Types.h.

6.2.5.2 Adc_Sar_Ip_ConvModeType

`enum Adc_Sar_Ip_ConvModeType`

Conversion mode selection (One-shot or Scan)

This enum is used to configure the conversion mode

Implements : Adc_Sar_Ip_ConvModeType_Class

Enumerator

ADC_SAR_IP_CONV_MODE_ONESHOT	One-shot conversion mode
ADC_SAR_IP_CONV_MODE_SCAN	Scan conversion mode

Definition at line 116 of file Adc_Sar_Ip_Types.h.

6.2.5.3 Adc_Sar_Ip_ClockSelType

`enum Adc_Sar_Ip_ClockSelType`

Converter input clock.

This enum is used to configure the converter input clock

Implements : Adc_Sar_Ip_ClockSelType_Class

Enumerator

ADC_SAR_IP_CLK_FULL_BUS	Adc module clock
ADC_SAR_IP_CLK_HALF_BUS	Adc module clock/2

Definition at line 129 of file Adc_Sar_Ip_Types.h.

6.2.5.4 Adc_Sar_Ip_ExtTriggerEdgeType

enum Adc_Sar_Ip_ExtTriggerEdgeType

External Trigger selection.

This enum is used to configure the external trigger

Implements : Adc_Sar_Ip_ExtTriggerEdgeType_Class

Enumerator

ADC_SAR_IP_EXT_TRIG_EDGE_DISABLED	Injected trigger disabled
ADC_SAR_IP_EXT_TRIG_EDGE_FALLING	Injected trigger on Falling Edge
ADC_SAR_IP_EXT_TRIG_EDGE_RISING	Injected trigger on Rising Edge

Definition at line 162 of file Adc_Sar_Ip_Types.h.

6.2.5.5 Adc_Sar_Ip_ExtTriggerSourceType

enum Adc_Sar_Ip_ExtTriggerSourceType

External Trigger Source Enable.

This enum is used to enable the external trigger source to start the conversion.

Implements : Adc_Sar_Ip_ExtTriggerSourceType_Class

Enumerator

ADC_SAR_IP_NORMAL_EXT_TRIG	Enables normal trigger input
ADC_SAR_IP_AUX_NORMAL_EXT_TRIG	Enables auxiliary normal trigger input
ADC_SAR_IP_ALL_NORMAL_EXT_TRIG	Enables normal and auxiliary trigger inputs
ADC_SAR_IP_INJECTED_EXT_TRIG	Enables injection trigger input

Definition at line 176 of file Adc_Sar_Ip_Types.h.

6.2.5.6 Adc_Sar_Ip_ConvChainType

enum Adc_Sar_Ip_ConvChainType

Conversion chain selection.

This enum is used to configure type of the conversion

Implements : Adc_Sar_Ip_ConvChainType_Class

Enumerator

ADC_SAR_IP_CONV_CHAIN_NORMAL	Selects the "Normal" Conversion Chain
ADC_SAR_IP_CONV_CHAIN_INJECTED	Selects the "Injected" Conversion Chain
ADC_SAR_IP_CONV_CHAIN_CTU	Selects the "CTU" Conversion Chain

Definition at line 197 of file Adc_Sar_Ip_Types.h.

6.2.5.7 Adc_Sar_Ip_DataAlignedType

enum Adc_Sar_Ip_DataAlignedType

Data alignment selection.

This enum is used to configure data alignment

Implements : Adc_Sar_Ip_DataAlignedType_Class

Enumerator

ADC_SAR_IP_DATA_ALIGNED_RIGHT	Measured data is right-aligned
ADC_SAR_IP_DATA_ALIGNED_LEFT	Measured data is left-aligned

Definition at line 211 of file Adc_Sar_Ip_Types.h.

6.2.5.8 Adc_Sar_Ip_ClearSourceType

enum Adc_Sar_Ip_ClearSourceType

Clear DMA source.

This enum is used to configure source used to clear a DMA request

Implements : Adc_Sar_Ip_ClearSourceType_Class

Enumerator

ADC_SAR_IP_DMA_REQ_CLEAR_ON_ACK	Clear DMA Request on Ack from DMA Controller
ADC_SAR_IP_DMA_REQ_CLEAR_ON_READ	Clear DMA Request on read of Data Registers

Definition at line 224 of file Adc_Sar_Ip_Types.h.

6.2.5.9 Adc_Sar_Ip_PresamplingSourceType

`enum Adc_Sar_Ip_PresamplingSourceType`

Presampling Voltage selection.

This enum is used to configure the presampling voltage

Implements : Adc_Sar_Ip_PresamplingSourceType_Class

Enumerator

ADC_SAR_IP_PRESAMPLE_VREFL	Presampling from VREFL
ADC_SAR_IP_PRESAMPLE_VREFH	Presampling from VREFH

Definition at line 237 of file Adc_Sar_Ip_Types.h.

6.2.5.10 Adc_Sar_Ip_ChanGroupType

`enum Adc_Sar_Ip_ChanGroupType`

Channel group selection.

This enum is used to select the group of ADC channels

Implements : Adc_Sar_Ip_ChanGroupType_Class

Enumerator

ADC_SAR_IP_CHAN_GROUP↩ _0	Channels Group (0-31)
ADC_SAR_IP_CHAN_GROUP↩ _1	Channels Group (32-63)

Definition at line 254 of file Adc_Sar_Ip_Types.h.

6.2.5.11 Adc_Sar_Ip_Resolution

```
enum Adc_Sar_Ip_Resolution
```

Adc group conversion resolution.

Used for value received by Tressos interface configuration.

Enumerator

ADC_SAR_IP_RESOLUTION_14	14-bit per conversion data
ADC_SAR_IP_RESOLUTION_12	12-bit per conversion data
ADC_SAR_IP_RESOLUTION_10	10-bit per conversion data
ADC_SAR_IP_RESOLUTION_8	8-bit per conversion data

Definition at line 285 of file Adc_Sar_Ip_Types.h.

6.2.6 Function Reference

6.2.6.1 Adc_Sar_Ip_GetDataAddress()

```
uint32 Adc_Sar_Ip_GetDataAddress (
            uint32 u32Instance,
            uint32 u32ChannelIndex )
```

Return the address of the specified data register.

This function returns the address of the specified data register

Parameters

in	*u32Instance*	- ADC instance number
in	*u32ChannelIndex*	- adc channel of the Hw unit

Returns

status:

- value of the address of the data for the specified channel

6.2.6.2 Adc_Sar_Ip_Init()

```
Adc_Sar_Ip_StatusType Adc_Sar_Ip_Init (
            const uint32 u32Instance,
            const Adc_Sar_Ip_ConfigType *const pConfig )
```

Initialize ADC_SAR module.

This function initializes the ADC_SAR module by configuring all available features.

Parameters

in	*u32Instance*	- ADC instance number
in	*pConfig*	- configuration struct pointer

Returns

status:

- ADC_SAR_IP_STATUS_SUCCESS: init successful
- ADC_SAR_IP_STATUS_TIMEOUT: init step timed out

6.2.6.3 Adc_Sar_Ip_Deinit()

```
Adc_Sar_Ip_StatusType Adc_Sar_Ip_Deinit (
            const uint32 u32Instance )
```

Deinitialize ADC_SAR module.

This function resets the ADC internal registers to default values.

Parameters

in	*u32Instance*	- ADC instance number

Returns

status:

- ADC_SAR_IP_STATUS_SUCCESS: deinit successful
- ADC_SAR_IP_STATUS_TIMEOUT: deinit step timed out

6.2.6.4 Adc_Sar_Ip_ChainConfig()

```
void Adc_Sar_Ip_ChainConfig (
            const uint32 u32Instance,
            const Adc_Sar_Ip_ChansIdxMaskType *const pChansIdxMask,
            const Adc_Sar_Ip_ConvChainType pChainType )
```

Configures the converter chains the given configuration structure.

This function configures the ADC Normal and Injected Chains with the options provided in the structure.

Parameters

in	*u32Instance*	- ADC instance number
in	*pChansIdxMask*	- channel configuration structure
in	*pChainType*	- conversion chain (Normal or Injected)

Returns

void

6.2.6.5 Adc_Sar_Ip_EnableChannel()

```
void Adc_Sar_Ip_EnableChannel (
            const uint32 u32Instance,
            const Adc_Sar_Ip_ConvChainType pChainType,
            const uint32 u32ChnIdx )
```

Enable a channel.

This function enables a channel in a specified conversion chain

Parameters

in	*u32Instance*	- ADC instance number
in	*pChainType*	- conversion chain (Normal or Injected)
in	*u32ChnIdx*	- channel 0

Returns

void

6.2.6.6 Adc_Sar_Ip_DisableChannel()

```
void Adc_Sar_Ip_DisableChannel (
            const uint32 u32Instance,
            const Adc_Sar_Ip_ConvChainType pChainType,
            const uint32 u32ChnIdx )
```

Disable a channel.

This function disables a channel in a specified conversion chain

Parameters

in	*u32Instance*	- ADC instance number
in	*pChainType*	- conversion chain (Normal or Injected)
in	*u32ChnIdx*	- channel 0

Returns

void

6.2.6.7 Adc_Sar_Ip_SetResolution()

```
void Adc_Sar_Ip_SetResolution (
            const uint32 u32Instance,
            const Adc_Sar_Ip_Resolution eResolution )
```

Set conversion resolution.

This function sets the conversion resolution (number of bits per conversion data)

Parameters

in	*u32Instance*	- ADC instance number
in	*eResolution*	- conversion resolution

Returns

void

6.2.6.8 Adc_Sar_Ip_StartConversion()

```
void Adc_Sar_Ip_StartConversion (
            const uint32 u32Instance,
            const Adc_Sar_Ip_ConvChainType pChainType )
```

Start conversion.

This function starts a conversion channel (Normal or Injected)

Parameters

in	*u32Instance*	- ADC instance number
in	*pChainType*	- conversion chain (Normal or Injected)

Returns

void

6.2.6.9 Adc_Sar_Ip_GetStatusFlags()

```
uint32 Adc_Sar_Ip_GetStatusFlags (
            const uint32 u32Instance )
```

Get the status flags.

This function returns the status flags of the ADC.

Parameters

in	*u32Instance*	- ADC instance number

Returns

the status flag bit-mask

6.2.6.10 Adc_Sar_Ip_ClearStatusFlags()

```
void Adc_Sar_Ip_ClearStatusFlags (
            const uint32 u32Instance,
            const uint32 u32Mask )
```

Clear the status flags.

This function clears the status flags of the ADC.

Parameters

in	*u32Instance*	- ADC instance number
in	*u32Mask*	- bit-mask of flags to clear

Returns

void

6.2.6.11 Adc_Sar_Ip_SelfTest()

```
Adc_Sar_Ip_StatusType Adc_Sar_Ip_SelfTest (
            const uint32 u32Instance )
```

Self test.

This function executes a self test on the ADC instance.

Parameters

in	*u32Instance*	- ADC instance number

Returns

status:

- ADC_SAR_IP_STATUS_SUCCESS: self testing successful
- ADC_SAR_IP_STATUS_TIMEOUT: self testing step timed out

6.2.6.12 Adc_Sar_Ip_GetConvDataToArray()

```
uint32 Adc_Sar_Ip_GetConvDataToArray (
            const uint32 u32Instance,
            const Adc_Sar_Ip_ConvChainType pChainType,
            const uint32 u32Length,
            uint16 *const pResults )
```

Get conversion results for a conversion chain.

This function gets the conversion results for the selected Conversion Chain.

Parameters

in	*u32Instance*	- ADC instance number
in	*pChainType*	- conversion chain (Normal, Injected or CTU)
in	*u32Length*	- the length of the buffer
out	*pResults*	- the output buffer

Returns

the number of values written in the buffer (max length)

6.2.6.13 Adc_Sar_Ip_GetConvResultsToArray()

```
uint32 Adc_Sar_Ip_GetConvResultsToArray (
            const uint32 u32Instance,
            const Adc_Sar_Ip_ConvChainType pChainType,
            const uint32 u32Length,
            Adc_Sar_Ip_ChanResultType *const pResults )
```

Get conversion results for a conversion chain with extended information.

This function gets the conversion results for the selected Conversion Chain, with extended information about each conversion result (channel index, valid an overwritten properties and conversion data). This function should be used in case of configurations with overlapping channel lists in different chains, resulting in overwrite of conversion data when a higher priority chain is executed before all data was read.

Parameters

in	*u32Instance*	- ADC instance number
in	*pChainType*	- conversion chain (Normal, Injected or CTU)
in	*u32Length*	- the length of the buffer
out	*pResults*	- the output buffer

Returns

the number of values written in the buffer (max length)

6.2.6.14 Adc_Sar_Ip_GetConvData()

```
uint16 Adc_Sar_Ip_GetConvData (
            const uint32 u32Instance,
            const uint32 u32ChnIdx )
```

Return the result of the conversion.

This function returns the result of the conversion for a single channel

Parameters

in	*u32Instance*	- ADC instance number
in	*u32ChnIdx*	- channel 0

Returns

conversion data

6.2.6.15 Adc_Sar_Ip_GetConvResult()

```
void Adc_Sar_Ip_GetConvResult (
            const uint32 u32Instance,
            const uint32 u32ChnIdx,
            const Adc_Sar_Ip_ConvChainType pChainType,
            Adc_Sar_Ip_ChanResultType *const pResult )
```

Return the result and the status of the conversion.

This function returns the result and the status of the conversion for a single channel

Parameters

in	*u32Instance*	- ADC instance number
in	*u32ChnIdx*	- channel 0
in	*pChainType*	- conversion chain (Normal, Injected or CTU)
out	*pResult*	- pointer to the buffer where the result will be written

Returns

void

6.2.6.16 Adc_Sar_Ip_DoCalibration()

```
Adc_Sar_Ip_StatusType Adc_Sar_Ip_DoCalibration (
            const uint32 u32Instance )
```

Perform Calibration of the ADC.

This function performs a calibration of the ADC. The maximum input clock frequency for the ADC is 80 MHz, checked with assertions if DEV_ASSERT is enabled. After calibration, the ADC is left in Powerup state (PWDN bit is clear).

Parameters

in	*u32Instance*	- ADC instance number

Returns

the calibration result

- ADC_SAR_IP_STATUS_SUCCESS: calibration successful
- ADC_SAR_IP_STATUS_TIMEOUT: calibration step timed out
- ADC_SAR_IP_STATUS_ERROR: calibration failed

6.2.6.17 Adc_Sar_Ip_Powerup()

```
Adc_Sar_Ip_StatusType Adc_Sar_Ip_Powerup (
            const uint32 u32Instance )
```

Power up the ADC.

This function enables the ADC (disables the Power Down feature).

Parameters

in	*u32Instance*	- ADC instance number

Returns

status:

- ADC_SAR_IP_STATUS_SUCCESS: power up successful
- ADC_SAR_IP_STATUS_TIMEOUT: power up failed

6.2.6.18 Adc_Sar_Ip_Powerdown()

```
Adc_Sar_Ip_StatusType Adc_Sar_Ip_Powerdown (
            const uint32 u32Instance )
```

Power down the ADC.

This function disables the ADC (enables the Power Down feature).

Parameters

in	*u32Instance*	- ADC instance number

Returns

status:

- ADC_SAR_IP_STATUS_SUCCESS: power down successful
- ADC_SAR_IP_STATUS_TIMEOUT: power down failed

6.2.6.19 Adc_Sar_Ip_EnableNotifications()

```
void Adc_Sar_Ip_EnableNotifications (
            const uint32 u32Instance,
            const uint32 u32NotificationMask )
```

Enable ADC interrupts.

This function enables ADC interrupts.

Parameters

in	*u32Instance*	- ADC instance number
in	*u32NotificationMask*	- mask of interrupts to enable (of status flags)

Returns

void

6.2.6.20 Adc_Sar_Ip_DisableNotifications()

```
void Adc_Sar_Ip_DisableNotifications (
            const uint32 u32Instance,
            const uint32 u32NotificationMask )
```

Disable ADC interrupts.

This function disables ADC interrupts.

Parameters

in	*u32Instance*	- ADC instance number
in	*u32NotificationMask*	- mask of interrupts to disable (of status flags)

Returns

void

6.2.6.21 Adc_Sar_Ip_EnableChannelNotifications()

```
void Adc_Sar_Ip_EnableChannelNotifications (
            const uint32 u32Instance,
            const uint32 u32ChnIdx,
            const uint32 u32Mask )
```

Enable ADC interrupt for a channel.

This function enables interrupt generation on End of Conversion and watchdog events for a single channel. The mask parameter can be set using the defines provided in the driver header file that have the pattern ADC_SAR_IP_C↩ HAN_NOTIF_... e.g. ADC_SAR_IP_CHAN_NOTIF_EOC.

Parameters

in	*u32Instance*	- ADC instance number
in	*u32ChnIdx*	- channel 0
in	*u32Mask*	- mask selecting targeted events

Returns

void

6.2.6.22 Adc_Sar_Ip_DisableChannelNotifications()

```
void Adc_Sar_Ip_DisableChannelNotifications (
            const uint32 u32Instance,
            const uint32 u32ChnIdx,
            const uint32 u32Mask )
```

Disable ADC interrupt for a channel.

This function disables interrupt generation on End of Conversion and watchdog events for a single channel. The mask parameter can be set using the defines provided in the driver header file that have the pattern ADC_SAR_↩ IP_CHAN_NOTIF_... e.g. ADC_SAR_IP_CHAN_NOTIF_EOC.

Parameters

in	*u32Instance*	- ADC instance number
in	*u32ChnIdx*	- channel 0
in	*u32Mask*	- mask selecting targeted events

Returns

void

6.2.6.23 Adc_Sar_Ip_SetClockMode()

```
Adc_Sar_Ip_StatusType Adc_Sar_Ip_SetClockMode (
            const uint32 u32Instance,
            const Adc_Sar_Ip_ClockConfigType *const pConfig )
```

Set the ADC clocks.

This function initializes the ADC clock configuration.

Parameters

in	*u32Instance*	- ADC instance number
in	*pConfig*	- the clock configuration

Returns

status:

- ADC_SAR_IP_STATUS_SUCCESS: set successful
- ADC_SAR_IP_STATUS_TIMEOUT: power up or down sequence timed out

6.2.6.24 Adc_Sar_Ip_SetSampleTimes()

```
void Adc_Sar_Ip_SetSampleTimes (
            const uint32 u32Instance,
            const uint8 *const aSampleTimes )
```

Set the sample times.

This function sets the sample times for each channnel group. Note: aSampleTimes must contain the sample times in order of the channel groups, e.g. sample time for the channel group 1 must be on the first position.

Parameters

in	*u32Instance*	- ADC instance number
in	*aSampleTimes*	- array with size ADC_SAR_IP_NUM_GROUP_CHAN containing sample times for each channel group

Returns

void

6.2.6.25 Adc_Sar_Ip_AbortConversion()

```
void Adc_Sar_Ip_AbortConversion (
            const uint32 u32Instance )
```

Abort ongoing conversion.

This function aborts an ongoing conversion.

Parameters

in	*u32Instance*	- ADC instance number

Returns

void

6.2.6.26 Adc_Sar_Ip_AbortChain()

```
Adc_Sar_Ip_StatusType Adc_Sar_Ip_AbortChain (
            const uint32 u32Instance,
            const boolean bBlocking,
            const boolean bAllowRestart )
```

Abort ongoing chain conversion.

This function aborts an ongoing chain of conversions. If scan(continuos) mode is selected, then bAllowRestart selects whether the abort command restarts the chain or stops it. For one shot conversion, this should be FALSE.

Parameters

in	*u32Instance*	- ADC instance number
in	*bBlocking*	- wait and check that the ongoing conversion has stopped
in	*bAllowRestart*	- restart continuos conversion instead of aborting

Returns

status:

- ADC_SAR_IP_STATUS_SUCCESS: abort successful
- ADC_SAR_IP_STATUS_TIMEOUT: ongoing conversion could not be stopped

void

6.2.6.27 Adc_Sar_Ip_SetPresamplingSource()

```
void Adc_Sar_Ip_SetPresamplingSource (
            const uint32 u32Instance,
            const Adc_Sar_Ip_ChanGroupType pChanGroup,
            const Adc_Sar_Ip_PresamplingSourceType pPresampleSource )
```

Set the Presampling Source for the channel group.

This function configures the Presampling Source for a channel group.

Parameters

in	*u32Instance*	- ADC instance number
in	*pChanGroup*	- the channel group
in	*pPresampleSource*	- the presampling source

Returns

void

6.2.6.28 Adc_Sar_Ip_EnableChannelPresampling()

```
void Adc_Sar_Ip_EnableChannelPresampling (
            const uint32 u32Instance,
            const uint32 u32ChnIdx )
```

Enable Presampling on one channel.

This function enables the Presampling on one channel of the ADC.

Parameters

in	*u32Instance*	- ADC instance number
in	*u32ChnIdx*	- channel 0

Returns

void

6.2.6.29 Adc_Sar_Ip_DisableChannelPresampling()

```
void Adc_Sar_Ip_DisableChannelPresampling (
            const uint32 u32Instance,
            const uint32 u32ChnIdx )
```

Disable Presampling on one channel.

This function disables the Presampling on one channel of the ADC.

Parameters

in	*u32Instance*	- ADC instance number
in	*u32ChnIdx*	- channel 0

Returns

void

6.2.6.30 Adc_Sar_Ip_EnablePresampleConversion()

```
void Adc_Sar_Ip_EnablePresampleConversion (
            const uint32 u32Instance )
```

Enable Conversion Presampled Data.

This function enables bypass of the Sampling Phase, resulting in a conversion of the presampled data. This is available only for channels that have presampling enabled.

Parameters

in	*u32Instance*	- ADC instance number

Returns

void

6.2.6.31 Adc_Sar_Ip_DisablePresampleConversion()

```
void Adc_Sar_Ip_DisablePresampleConversion (
            const uint32 u32Instance )
```

Disable Conversion of Presampled Data.

This function disables Sampling Phase bypass.

Parameters

in	*u32Instance*	- ADC instance number

Returns

void

6.2.6.32 Adc_Sar_Ip_EnableDma()

```
void Adc_Sar_Ip_EnableDma (
            const uint32 u32Instance )
```

Enable DMA Requests.

This function enables requests to DMA from ADC

Parameters

in	*u32Instance*	- ADC instance number

Returns

void

6.2.6.33 Adc_Sar_Ip_DisableDma()

```
void Adc_Sar_Ip_DisableDma (
            const uint32 u32Instance )
```

Disable DMA Requests.

This function disables requests to DMA from ADC

Parameters

in	*u32Instance*	- ADC instance number

Returns

void

6.2.6.34 Adc_Sar_Ip_EnableChannelDma()

```
void Adc_Sar_Ip_EnableChannelDma (
            const uint32 u32Instance,
            const uint32 u32ChnIdx )
```

Enable DMA on one channel.

This function enables DMA requests triggered by End of Conversion event from a selected channel.

Parameters

in	*u32Instance*	- ADC instance number
in	*u32ChnIdx*	- channel 0

Returns

void

6.2.6.35 Adc_Sar_Ip_DisableChannelDma()

```
void Adc_Sar_Ip_DisableChannelDma (
            const uint32 u32Instance,
            const uint32 u32ChnIdx )
```

Disable DMA on one channel.

This function disables DMA requests triggered by End of Conversion event from a selected channel.

Parameters

in	*u32Instance*	- ADC instance number
in	*u32ChnIdx*	- channel 0

Returns

void

6.2.6.36 Adc_Sar_Ip_DisableChannelDmaAll()

```
void Adc_Sar_Ip_DisableChannelDmaAll (
            const uint32 u32Instance )
```

Disable DMA on all channels.

This function disables DMA requests triggered by End of Conversion event from all channels.

Parameters

in	*u32Instance*	- ADC instance number

Returns

void

6.2.6.37 Adc_Sar_Ip_SetDmaClearSource()

```
void Adc_Sar_Ip_SetDmaClearSource (
            const uint32 u32Instance,
            const Adc_Sar_Ip_ClearSourceType pDmaClear )
```

Set DMA Request Clear Source.

This function selects the DMA Request Flag Clear Source.

Parameters

in	*u32Instance*	- ADC instance number
in	*pDmaClear*	- the clear source for DMA Requests (Ack from DMA Controller or read of data registers)

Returns

void

6.2.6.38 Adc_Sar_Ip_SetWdgThreshold()

```
void Adc_Sar_Ip_SetWdgThreshold (
             const uint32 u32Instance,
             const uint8 u8RegisterIdx,
             const Adc_Sar_Ip_WdgThresholdType *const pThresholdValues )
```

Configure watchdog threshold register.

This function configures the high/low thresholds for a certain register.

Parameters

in	*u32Instance*	- ADC instance number
in	*u8RegisterIdx*	- the index of the register
in	*pThresholdValues*	- the threshold values

Returns

void

6.2.6.39 Adc_Sar_Ip_SetConversionMode()

```
void Adc_Sar_Ip_SetConversionMode (
             const uint32 u32Instance,
             const Adc_Sar_Ip_ConvModeType eConvMode )
```

Configure conversion mode.

This function configures the used conversion mode. Note: The selected ADC instance must be in the IDLE state.

Parameters

in	*u32Instance*	- ADC instance number
in	*eConvMode*	- selected conversion mode

Returns

void

6.2.6.40 Adc_Sar_Ip_SetExternalTrigger()

```
void Adc_Sar_Ip_SetExternalTrigger (
             const uint32 u32Instance,
             const Adc_Sar_Ip_ExtTriggerEdgeType eTriggerEdge,
             const Adc_Sar_Ip_ExtTriggerSourceType eTrggerSrc )
```

Configure external trigger.

This function configures the external trigger.

Parameters

in	*u32Instance*	- ADC instance number
in	*eTriggerEdge*	- selected external trigger type
in	*eTrggerSrc*	- selects normal, auxiliary normal or injected trigger

Returns

void

6.3 Bctu IPL

6.3.1 Detailed Description Bctu HW module.

Bctu IP layer module.

Data Structures

- struct Bctu_Ip_AdcNotificationType

  Structure for configuring Adc notification. More...
- struct Bctu_Ip_ListItemConfigType

  Structure for storing channel information. More...
- struct Bctu_Ip_ListItemType

  Structure for storing channel information at runtime. More...
- struct Bctu_Ip_FifoConfigType

  Structure for configuring BCTU FIFO. More...
- struct Bctu_Ip_FifoNotificationType

  Store BCTU FIFO notifications. More...
- struct Bctu_Ip_TrigConfigType

  Structure for configuring a trigger configuration register. More...
- struct Bctu_Ip_ResultType

  Structure for storing full information of the conversion result. More...
- struct Bctu_Ip_FifoResultType

  Structure for storing full information of the conversion result stored in the FIFO. More...
- struct Bctu_Ip_ConfigType

  Structure for configuring global BCTU functionalities. More...
- struct Bctu_Ip_StateStructType

  Structure used to store runtime info. More...

Macros

- #define BCTU_IP_NOTIF_FIFO1

  Macros for status and notification flags.

Enum Reference

- enum Bctu_Ip_StatusType

  BCTU status return type.
- enum Bctu_Ip_TrigDataDestType

  Enumeration for selecting data destination.
- enum Bctu_Ip_TrigType

  Enumeration for selecting trigger type.

Function Reference

- void Bctu_Ip_Init (const uint32 u32Instance, const Bctu_Ip_ConfigType *const pConfig)

  Configure global functionalities of a BCTU instance.
- Bctu_Ip_StatusType Bctu_Ip_Deinit (const uint32 u32Instance)

  Reset the BCTU registers to default values.
- void Bctu_Ip_SetGlobalTriggerEn (const uint32 u32Instance, const boolean bState)

  Set the state of the Global Trigger Enable flag.
- void Bctu_Ip_SetLowPowerMode (const uint32 u32Instance, const boolean bState)

  Set the BCTU operating mode: normal or low power.
- uint32 Bctu_Ip_GetStatusFlags (const uint32 u32Instance)

  Get the value of all BCTU status flags.
- void Bctu_Ip_ClearStatusFlags (const uint32 u32Instance, const uint32 u32Mask)

  Clear selected BCTU status flags.
- Bctu_Ip_StatusType Bctu_Ip_ConfigTrigger (const uint32 u32Instance, const Bctu_Ip_TrigConfigType *const pTrigConfig)

  Configures a trigger configuration register for an individual (non-list) conversion.
- Bctu_Ip_StatusType Bctu_Ip_SetTriggerChnListAddr (const uint32 u32Instance, const uint8 u8TrigIdx, const boolean bList, const uint8 u8ChnOrListAddr)

  Configures trigger resolution and either the channel or first address of the command list associated with the trigger.
- void Bctu_Ip_EnableHwTrigger (const uint32 u32Instance, const uint8 u8TrigIdx)

  Clears any eventual pending hw input signals already occurred on the selected trigger and enables new occurrences of the hw input signals to start new conversions.
- void Bctu_Ip_DisableHwTrigger (const uint32 u32Instance, const uint8 u8TrigIdx)

  Prevent the selected BCTU trigger to start new conversions on the occurrence of an external input signal or new conversions in an on-going conversion loop (loop flag enabled).
- void Bctu_Ip_StopLoopConversions (const uint32 u32Instance, const uint8 u8TrigIdx)

  Stop a series of loop conversions initiated by the selected BCTU trigger.
- void Bctu_Ip_ConfigChanList (const uint32 u32Instance, const uint8 u8StartListElemIdx, const Bctu_Ip_ListItemType *const pConvListItems, const uint8 u8NumItems)

  Set an array of ADC channel structures in the conversion list.
- void Bctu_Ip_ConfigChanListSimple (const uint32 u32Instance, const uint8 u8StartListElemIdx, const uint8 *const pConvListItems, const uint8 u8NumItems)

  Set an array of ADC channel numbers in the conversion list.
- void Bctu_Ip_SwTriggerConversion (const uint32 u32Instance, const uint8 u8TrigIdx)

  Triggers a conversion or list of conversions associated with a trigger index, if write protection is disabled.
- uint16 Bctu_Ip_GetConvData (const uint32 u32Instance, const uint8 u8AdcIdx)

  Return the current conversion result data corresponding to an ADC instance.
- void Bctu_Ip_GetConvResult (const uint32 u32Instance, const uint8 u8AdcIdx, Bctu_Ip_ResultType *const pResult)

  Return the full conversion result information corresponding to an ADC instance.
- uint16 Bctu_Ip_GetFifoData (const uint32 u32Instance, const uint8 u8FifoIdx)

  Return the conversion result data stored in the index in a FIFO.
- void Bctu_Ip_GetFifoResult (const uint32 u32Instance, const uint8 u8FifoIdx, Bctu_Ip_FifoResultType *const pResult)

  Return the full conversion result information stored in a FIFO.
- void Bctu_Ip_EnableNotifications (const uint32 u32Instance, const uint32 u32NotificationMask)

  Enables notifications.

- void Bctu_Ip_DisableNotifications (const uint32 u32Instance, const uint32 u32NotificationMask)
  Disables notifications.
- void Bctu_Ip_SetFifoWatermark (const uint32 u32Instance, const uint8 u8FifoIdx, const uint8 u8Watermark)
  Sets watermark value for the selected FIFO.
- uint8 Bctu_Ip_GetFifoCount (const uint32 u32Instance, const uint8 u8FifoIdx)
  Get the counter value of the given FIFO.

6.3.2 Data Structure Documentation

6.3.2.1 struct Bctu_Ip_AdcNotificationType

Structure for configuring Adc notification.

This structure is used for configuring Adc notification

Implements : Bctu_Ip_AdcNotificationType_Class

Definition at line 142 of file Bctu_Ip_Types.h.

6.3.2.2 struct Bctu_Ip_ListItemConfigType

Structure for storing channel information.

This structure is used for storing channel information.

Implements : Bctu_Ip_ListItemConfigType_Class

Definition at line 155 of file Bctu_Ip_Types.h.

6.3.2.3 struct Bctu_Ip_ListItemType

Structure for storing channel information at runtime.

This structure is used for storing channel information at runtime.

Implements : Bctu_Ip_ListItemType_Class

Definition at line 168 of file Bctu_Ip_Types.h.

6.3.2.4 struct Bctu_Ip_FifoConfigType

Structure for configuring BCTU FIFO.

This structure is used for configuring the BCTU FIFO.

Implements : Bctu_Ip_FifoConfigType_Class

Definition at line 180 of file Bctu_Ip_Types.h.

6.3.2.5 struct Bctu_Ip_FifoNotificationType

Store BCTU FIFO notifications.

This structure is used for storing BCTU FIFO notifications

Implements : Bctu_Ip_FifoNotificationType_Class

Definition at line 203 of file Bctu_Ip_Types.h.

6.3.2.6 struct Bctu_Ip_TrigConfigType

Structure for configuring a trigger configuration register.

This structure is used for configuring a trigger configuration register (TRGCFG)

Implements : Bctu_Ip_TrigConfigType_Class

Definition at line 216 of file Bctu_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	TrigIndex	
boolean	LoopEn	Select if a trigger (hw or sw) starts a loop of conversions.
Bctu_Ip_TrigDataDestType	DataDest	
boolean	HwTriggersEn	Enable/disable initiation of ADC conversions by hw input trigger: true - enabled; false - disabled.
Bctu_Ip_TrigType	TrigType	
uint8	AdcTargetMask	Bitmask to select target ADC instance(s): (1 << n) corresponds to n-th ADC instance.
uint8	AdcChanOrListStart	

6.3.2.7 struct Bctu_Ip_ResultType

Structure for storing full information of the conversion result.

This structure is used for storing full information of the conversion result

Implements : Bctu_Ip_ResultType_Class

Definition at line 233 of file Bctu_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	TriggerIdx	Index of the trigger which started the conversion.
uint8	ChanIdx	Index of the ADC channel which executed the conversion.
uint16	AdcData	Conversion result data.
boolean	ListFlag	Conversion initiated from a LIST: true / false.
boolean	LastFlag	Last conversion in a LIST: true / false

6.3.2.8 struct Bctu_Ip_FifoResultType

Structure for storing full information of the conversion result stored in the FIFO.

This structure is used for storing full information of the conversion result stored in the FIFO

Implements : Bctu_Ip_FifoResultType_Class

Definition at line 248 of file Bctu_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	TriggerIdx	Index of the trigger which started the conversion.
uint8	ChanIdx	Index of the ADC channel which executed the conversion.
uint8	AdcNum	Index of the ADC instance which executed the conversion.
uint16	AdcData	Conversion result data.

6.3.2.9 struct Bctu_Ip_ConfigType

Structure for configuring global BCTU functionalities.

This structure is used for configuring global BCTU functionalities

Implements : Bctu_Ip_ConfigType_Class

Definition at line 262 of file Bctu_Ip_Types.h.

Data Fields

- boolean LowPowerModeEn
- boolean GlobalHwTriggersEn
- uint8 NewDataDmaEnMask
- void(* TriggerNotification)(void)

6.3.2.9.1 Field Documentation

6.3.2.9.1.1 LowPowerModeEn `boolean LowPowerModeEn`

Low power operating mode: true - low power; false - normal operating mode.

Definition at line 263 of file Bctu_Ip_Types.h.

6.3.2.9.1.2 GlobalHwTriggersEn `boolean GlobalHwTriggersEn`

HW triggers globally enabled/disabled: true - enabled; false - all hw triggers disabled (BCTU in freeze mode).

Definition at line 264 of file Bctu_Ip_Types.h.

6.3.2.9.1.3 NewDataDmaEnMask `uint8 NewDataDmaEnMask`

Bitmask for DMA transfer enable per ADC instance: (1 << n) corresponds to n-th ADC instance.

Definition at line 265 of file Bctu_Ip_Types.h.

6.3.2.9.1.4 TriggerNotification `void(* TriggerNotification) (void)`

TRGF

Definition at line 267 of file Bctu_Ip_Types.h.

6.3.2.10 struct Bctu_Ip_StateStructType

Structure used to store runtime info.

This structure is used to store BCTU runtime info

Implements : Bctu_Ip_StateStructType_Class

Definition at line 288 of file Bctu_Ip_Types.h.

Data Fields

- boolean InitStatus
- void(* TriggerNotification)(void)

6.3.2.10.1 Field Documentation

6.3.2.10.1.1 InitStatus `boolean InitStatus`

Check if the driver was initialized.

Definition at line 289 of file Bctu_Ip_Types.h.

6.3.2.10.1.2 TriggerNotification `void(* TriggerNotification) (void)`

TRGF

Definition at line 290 of file Bctu_Ip_Types.h.

6.3.3 Macro Definition Documentation

6.3.3.1 BCTU_IP_NOTIF_FIFO1

```
#define BCTU_IP_NOTIF_FIFO1
```

Macros for status and notification flags.

These flags map to internal hardware flags in different registers, but are grouped together for convenience.

Definition at line 113 of file Bctu_Ip.h.

6.3.4 Enum Reference

6.3.4.1 Bctu_Ip_StatusType

```
enum Bctu_Ip_StatusType
```

BCTU status return type.

This structure is used as return type

Implements : Bctu_Ip_StatusType_Class

Enumerator

BCTU_IP_STATUS_SUCCESS	Function completed successfully
BCTU_IP_STATUS_ERROR	Function didn't complete successfully
BCTU_IP_STATUS_TIMEOUT	Function timed out
BCTU_IP_STATUS_BUSY	Functionality busy

Definition at line 102 of file Bctu_Ip_Types.h.

6.3.4.2 Bctu_Ip_TrigDataDestType

```
enum Bctu_Ip_TrigDataDestType
```

Enumeration for selecting data destination.

This enumeration selects the data destination

Implements : Bctu_Ip_TrigDataDestType_Class

Definition at line 117 of file Bctu_Ip_Types.h.

6.3.4.3 Bctu_Ip_TrigType

```
enum Bctu_Ip_TrigType
```

Enumeration for selecting trigger type.

This enumeration selects the trigger type

Implements : Bctu_Ip_TrigType_Class

Definition at line 130 of file Bctu_Ip_Types.h.

6.3.5 Function Reference

6.3.5.1 Bctu_Ip_Init()

```
void Bctu_Ip_Init (
            const uint32 u32Instance,
            const Bctu_Ip_ConfigType *const pConfig )
```

Configure global functionalities of a BCTU instance.

This function configures the global functionalities of a BCTU instance.

Parameters

in	*u32Instance*	- BCTU instance number
in	*pConfig*	- pointer to the input configuration structure

Returns

void

6.3.5.2 Bctu_Ip_Deinit()

```
Bctu_Ip_StatusType Bctu_Ip_Deinit (
            const uint32 u32Instance )
```

Reset the BCTU registers to default values.

This function attempts to reset the BCTU registers to default values, in the given timeout interval. Trigger config registers cannot be modified while there is an ongoing conversion, so the function waits for the conversion(s) to finish and returns an error code if timeout occurres. It globally disables all triggers, leaving the BCTU in configuration mode.

Parameters

in	*u32Instance*	- BCTU instance number

Returns

status:
- BCTU_IP_STATUS_SUCCESS: reset succeeded
- BCTU_IP_STATUS_TIMEOUT: reset did not complete because an active conversion did not finish execution within the provided timeout interval

6.3.5.3 Bctu_Ip_SetGlobalTriggerEn()

```
void Bctu_Ip_SetGlobalTriggerEn (
            const uint32 u32Instance,
            const boolean bState )
```

Set the state of the Global Trigger Enable flag.

This function sets the state of the Global Trigger Enable flag. When triggers are disabled globally, the BCTU is in configuration mode, which allows configuring the trigger config registers freely, without generating any spurious ADC conversion triggers. Enabling the flag will move BCTU out of configuration mode and will enable all trigger inputs which are already individually enabled from the trigger config registers.

Parameters

in	*u32Instance*	- BCTU instance number
in	*bState*	- true: globally enable triggers false: globally disable triggers - enter configuration mode

Returns

void

6.3.5.4 Bctu_Ip_SetLowPowerMode()

```
void Bctu_Ip_SetLowPowerMode (
            const uint32 u32Instance,
            const boolean bState )
```

Set the BCTU operating mode: normal or low power.

This function sets the BCTU operating mode: normal or low power. In low power mode all trigger inputs are ignored and any pending triggers remain active.

Parameters

in	*u32Instance*	- BCTU instance number
in	*bState*	- true: enable low power mode false: enable normal mode

Returns

void

6.3.5.5 Bctu_Ip_GetStatusFlags()

```
uint32 Bctu_Ip_GetStatusFlags (
            const uint32 u32Instance )
```

Get the value of all BCTU status flags.

This function returns the value of all BCTU status flags.

Parameters

in	*u32Instance*	- BCTU instance number

Returns

state of the selected flags

6.3.5.6 Bctu_Ip_ClearStatusFlags()

```
void Bctu_Ip_ClearStatusFlags (
            const uint32 u32Instance,
            const uint32 u32Mask )
```

Clear selected BCTU status flags.

This function clears selected BCTU status flags. The mask parameter should be set using BCTU_IP_STATUS_... defines.

Parameters

in	*u32Instance*	- BCTU instance number
in	*u32Mask*	- selected flags mask

Returns

void

6.3.5.7 Bctu_Ip_ConfigTrigger()

```
Bctu_Ip_StatusType Bctu_Ip_ConfigTrigger (
            const uint32 u32Instance,
            const Bctu_Ip_TrigConfigType *const pTrigConfig )
```

Configures a trigger configuration register for an individual (non-list) conversion.

This function attempts to configure a trigger configuration register for an individual (non-list) conversion. If there is an on-going conversion started via target trigger, it only disables the trigger (will ignore any new triggers) and returns error code.
If succeeds, implicitly clears any pending previous input trigger event.
Note: when executing a list, the conversion is considered active until the conversion marked as "last" completes.

Parameters

in	*u32Instance*	- BCTU instance number
in	*pTrigConfig*	- BCTU trigger configuration

Returns

status:
- BCTU_IP_STATUS_SUCCESS: configured successfully
- BCTU_IP_STATUS_BUSY: configuration was not written because there is a triggered active conversion

6.3.5.8 Bctu_Ip_SetTriggerChnListAddr()

```
Bctu_Ip_StatusType Bctu_Ip_SetTriggerChnListAddr (
            const uint32 u32Instance,
            const uint8 u8TrigIdx,
            const boolean bList,
            const uint8 u8ChnOrListAddr )
```

Configures trigger resolution and either the channel or first address of the command list associated with the trigger.

This function attempts to configure trigger resolution and the register for either the channel or the first address of the command list associated with the trigger. If there is an on-going conversion started via target trigger, it only disables the trigger (will ignore any new triggers) and returns error code.
If succeeds, implicitly clears any pending previous input trigger event.

Parameters

in	*u32Instance*	- BCTU instance number
in	*u8TrigIdx*	- index of the trigger register
in	*bList*	- value of trigger resolution register: false for single conversion and true for conversion list
in	*u8ChnOrListAddr*	- channel index if bList is false or conversion list start address if bList is true

Returns

status:
- BCTU_IP_STATUS_SUCCESS: configured successfully
- BCTU_IP_STATUS_BUSY: configuration was not written because there is a triggered active conversion

6.3.5.9 Bctu_Ip_EnableHwTrigger()

```
void Bctu_Ip_EnableHwTrigger (
            const uint32 u32Instance,
            const uint8 u8TrigIdx )
```

Clears any eventual pending hw input signals already occurred on the selected trigger and enables new occurrences of the hw input signals to start new conversions.

This function clears any eventual pending hw input signals already occurred on the selected trigger and enables new occurrences of the hw input signals to start new conversions.

Parameters

in	*u32Instance*	- BCTU instance number
in	*u8TrigIdx*	- index of the trigger register

Returns

void

6.3.5.10 Bctu_Ip_DisableHwTrigger()

```
void Bctu_Ip_DisableHwTrigger (
            const uint32 u32Instance,
            const uint8 u8TrigIdx )
```

Prevent the selected BCTU trigger to start new conversions on the occurrence of an external input signal or new conversions in an on-going conversion loop (loop flag enabled).

This function prevents the selected BCTU trigger to start new conversions on the occurrence of an hw input signal or new conversions in an on-going conversion loop (loop flag enabled). After calling this function, new conversions may still be started by BCTU software trigger. Note: the function does NOT stop an on-going conversion.

Parameters

in	*u32Instance*	- BCTU instance number
in	*u8TrigIdx*	- index of the trigger register

Returns

void

6.3.5.11 Bctu_Ip_StopLoopConversions()

```
void Bctu_Ip_StopLoopConversions (
            const uint32 u32Instance,
            const uint8 u8TrigIdx )
```

Stop a series of loop conversions initiated by the selected BCTU trigger.

This function stops a series of loop conversions initiated by the selected BCTU trigger. Note0: the function does NOT stop an on-going conversion. Note1: the function assumes that no HW trigger occurs on the selected trigger during its execution - the reset & restore operation on the LOOP flag is not atomic! Occurrence of a HW trigger may start a new conversion with an inconsistent value for the LOOP flag. For stopping HW triggered loops of conversions consider also using Bctu_Ip_DisableHwTrigger.

Parameters

in	*u32Instance*	- BCTU instance number
in	*u8TrigIdx*	- index of the trigger register

Returns

void

6.3.5.12 Bctu_Ip_ConfigChanList()

```
void Bctu_Ip_ConfigChanList (
            const uint32 u32Instance,
            const uint8 u8StartListElemIdx,
            const Bctu_Ip_ListItemType *const pConvListItems,
            const uint8 u8NumItems )
```

Set an array of ADC channel structures in the conversion list.

This function sets the conversion list using an array of ListItemType structures. Also marks the element corresponding to the last channel in the array, as last element in the list. Implicitly wrap-around of the array onto the list, if (listSize - startElemIdx) < numArrayElems.

Parameters

in	*u32Instance*	- BCTU instance number
in	*u8StartListElemIdx*	- index in the list corresponding to the first element in the array
in	*pConvListItems*	- pointer to the first element in the ADC channel structure input array
in	*u8NumItems*	- number of elements in the ADC channel numbers input array (required to be >0)

Returns

void

6.3.5.13 Bctu_Ip_ConfigChanListSimple()

```
void Bctu_Ip_ConfigChanListSimple (
            const uint32 u32Instance,
            const uint8 u8StartListElemIdx,
            const uint8 *const pConvListItems,
            const uint8 u8NumItems )
```

Set an array of ADC channel numbers in the conversion list.

This function sets an array of ADC channel numbers in the conversion list and marks the element corresponding to the last channel in the array, as last element in the list. Other configurable fields of a conversion element, like NEXT_CH_WAIT_ON_TRIG_y is set to false/0. Implicitly wrap-around of the array onto the list, if (listSize - startElemIdx) < numArrayElems.

Parameters

in	*u32Instance*	- BCTU instance number
in	*u8StartListElemIdx*	- index in the list corresponding to the first element in the array
in	*pConvListItems*	- pointer to the first element in the ADC channel array
in	*u8NumItems*	- number of elements in the ADC channel numbers input array (required to be >0)

Returns

void

6.3.5.14 Bctu_Ip_SwTriggerConversion()

```
void Bctu_Ip_SwTriggerConversion (
            const uint32 u32Instance,
            const uint8 u8TrigIdx )
```

Triggers a conversion or list of conversions associated with a trigger index, if write protection is disabled.

This function triggers a conversion or list of conversions associated with a trigger index, if write protection is disabled. If write protection is enabled, the function will not trigger the conversion.
It is recommended that this function is used with HW triggers disabled for the target trigger index, to avoid ambiguity regarding source of trigger (hw or sw) for conversion results.
The BCTU HW will ignore attempts to trigger a new conversion or list, while there is a pending conversion or, in case of an active conversion list, before the last conversion is started.

Parameters

in	*u32Instance*	- BCTU instance number
in	*u8TrigIdx*	- index of the trigger register

Returns

void

6.3.5.15 Bctu_Ip_GetConvData()

```
uint16 Bctu_Ip_GetConvData (
            const uint32 u32Instance,
            const uint8 u8AdcIdx )
```

Return the current conversion result data corresponding to an ADC instance.

This function returns the current conversion result data corresponding to an ADC instance. Note: In order to return the result with the correct resolution, the ADC_SAR instance used must still be initialized when calling this function.

Parameters

in	*u32Instance*	- BCTU instance number
in	*u8AdcIdx*	- index of the target ADC module

Returns

uint16 - conversion result

6.3.5.16 Bctu_Ip_GetConvResult()

```
void Bctu_Ip_GetConvResult (
            const uint32 u32Instance,
            const uint8 u8AdcIdx,
            Bctu_Ip_ResultType *const pResult )
```

Return the full conversion result information corresponding to an ADC instance.

This function returns the full conversion result information corresponding to an ADC instance. Note: In order to return the result with the correct resolution, the ADC_SAR instance used must still be initialized when calling this function.

Parameters

in	*u32Instance*	- BCTU instance number
in	*u8AdcIdx*	- index of the target ADC module
out	*pResult*	- pointer to the structure to be populated with the conversion result info

Returns

void

6.3.5.17 Bctu_Ip_GetFifoData()

```
uint16 Bctu_Ip_GetFifoData (
            const uint32 u32Instance,
            const uint8 u8FifoIdx )
```

Return the conversion result data stored in the index in a FIFO.

This function returns the conversion result data that is stored in the BCTU result FIFO at the given index. Note: In order to return the result with the correct resolution, the ADC_SAR instance used must still be initialized when calling this function.

Parameters

in	*u32Instance*	- BCTU instance number
in	*u8FifoIdx*	- index of the selected FIFO

Returns

uint16 - FIFO content

6.3.5.18 Bctu_Ip_GetFifoResult()

```
void Bctu_Ip_GetFifoResult (
            const uint32 u32Instance,
            const uint8 u8FifoIdx,
            Bctu_Ip_FifoResultType *const pResult )
```

Return the full conversion result information stored in a FIFO.

This function returns the full conversion result information that is stored in the BCTU result FIFO at the given index. Note: In order to return the result with the correct resolution, the ADC_SAR instance used must still be initialized when calling this function.

Parameters

in	*u32Instance*	- BCTU instance number
in	*u8FifoIdx*	- index of the selected FIFO
out	*pResult*	- pointer to the structure to be populated with the FIFO result info

Returns

void

6.3.5.19 Bctu_Ip_EnableNotifications()

```
void Bctu_Ip_EnableNotifications (
            const uint32 u32Instance,
            const uint32 u32NotificationMask )
```

Enables notifications.

This function enables notifications selected by the given mask on the selected BCTU instance. The notifications mask parameter should be set using BCTU_IP_NOTIF_... defines.

Parameters

in	*u32Instance*	- BCTU instance number
in	*u32NotificationMask*	- mask selecting targeted events

Returns

void

6.3.5.20 Bctu_Ip_DisableNotifications()

```
void Bctu_Ip_DisableNotifications (
            const uint32 u32Instance,
            const uint32 u32NotificationMask )
```

Disables notifications.

This function disables notifications selected by the given mask on the selected BCTU instance. The notifications mask parameter should be set using BCTU_IP_NOTIF_... defines.

Parameters

in	*u32Instance*	- BCTU instance number
in	*u32NotificationMask*	- mask selecting targeted events

Returns

void

6.3.5.21 Bctu_Ip_SetFifoWatermark()

```
void Bctu_Ip_SetFifoWatermark (
            const uint32 u32Instance,
            const uint8 u8FifoIdx,
            const uint8 u8Watermark )
```

Sets watermark value for the selected FIFO.

This function sets the watermark value for the selected FIFO.

Parameters

in	*u32Instance*	- BCTU instance number
in	*u8FifoIdx*	- index of the selected FIFO
in	*u8Watermark*	- watermark value

Returns

void

6.3.5.22 Bctu_Ip_GetFifoCount()

```
uint8 Bctu_Ip_GetFifoCount (
             const uint32 u32Instance,
             const uint8 u8FifoIdx )
```

Get the counter value of the given FIFO.

This function gets the counter value of the given FIFO.

Parameters

in	*u32Instance*	- BCTU instance number
in	*u8FifoIdx*	- index of the selected FIFO

Returns

uint8 - FIFO counter value

How to Reach Us:

Home Page:
nxp.com

Web Support:
nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, AltiVec, C–5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C–Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2022 NXP B.V.